



Universidad de Valladolid

ESCUELA DE INGENIERÍA INFORMÁTICA

GRADO EN INGENIERÍA INFORMÁTICA
Mención en Computación

Desarrollo de un sistema accesible para la
gestión de trámites online a través de una IA
telefónica

Alumno: Hugo Martín Alonso

Tutora: María Aránzazu Simón Hurtado

Resumen

Hoy en día se considera indispensable la conexión a Internet para realizar tareas cotidianas, aunque todavía existe un sector de la población que permanece al margen de estas tecnologías.

Este desarrollo presenta una solución accesible que puede ser utilizada por cualquier persona sin necesidad de conocimientos técnicos y disponiendo únicamente de una línea telefónica. Para ello, se han simulado distintos procedimientos administrativos que se llevan a cabo automáticamente, mediante un agente conversacional de inteligencia artificial que responde la llamada, recopila la información necesaria e inicia la automatización del proceso.

Este trabajo demuestra que es posible reducir la brecha digital ofreciendo un servicio alternativo, que actúa como intermediario entre las personas y la Administración digital. De esta manera se sientan las bases para futuras propuestas inclusivas capaces de integrar a colectivos excluidos del entorno digital.

Abstract

Nowadays, Internet access is considered essential for carrying out everyday tasks, although a segment of the population still remains excluded from these technologies.

This development presents an accessible solution that can be used by anyone without technical knowledge and requiring only a telephone line. To achieve this, various administrative procedures have been simulated and are carried out automatically by a conversational AI agent that answers the call, collects the necessary information and initiates the automation of the process.

This work demonstrates that it is possible to reduce the digital divide by offering an alternative service that acts as an intermediary between individuals and digital administration. In this way, it lays the groundwork for future inclusive proposals capable of integrating groups excluded from the digital environment.

Índice general

Resumen	III
Abstract	V
Lista de figuras	XI
Lista de tablas	XV
1. Introducción	1
1.1. Motivación del tema	1
1.2. Estado de la cuestión	2
1.3. Objetivos	3
1.4. Estructura de la memoria	3
2. Análisis	5
2.1. Público objetivo	5
2.2. Alcance del sistema	5
2.2.1. Trámites	6
2.3. Requisitos	6
2.4. Casos de uso	8
2.4.1. Diagrama de casos de uso:	8
2.4.2. Actores:	9
2.4.3. Solicitud del certificado de empadronamiento:	9
2.4.4. Solicitud de cita en la Agencia Tributaria:	9
2.4.5. Solicitud de la tarjeta sanitaria de SACYL por pérdida o robo:	10
2.4.6. Iniciar sesión:	11

2.4.7. Consultar detalles de una petición:	11
2.4.8. Petición revisable:	11
2.4.9. Consultar datos personales y cambiar contraseña:	12
2.4.10. Crear petición:	12
2.4.11. Añadir empleados:	12
2.4.12. Modificar o desactivar empleados	13
2.4.13. Añadir trámites:	13
2.4.14. Modificar o desactivar trámites:	14
2.5. Modelo de dominio:	15
3. Planificación	17
3.1. Metodología	17
3.2. Planificación temporal	17
3.3. Estimación de costes	18
3.3.1. Coste en un entorno profesional	18
3.3.2. Coste real del proyecto	18
3.4. Análisis de riesgos	19
3.5. Seguimiento del proyecto	19
4. Diseño	21
4.1. Decisiones técnicas	21
4.1.1. Frontend	21
4.1.2. Backend	21
4.1.3. Base de datos	21
4.1.4. Servicios	22
4.1.5. Herramientas de desarrollo y documentación	23
4.2. Diseño de la interfaz de usuario	24
4.3. Arquitectura del sistema	28
4.4. Diseño de la base de datos	30
4.5. Diseño de la inteligencia artificial	30
4.6. Diagramas de secuencia	32
4.7. Seguridad	42

5. Implementación	45
5.1. Implementación del frontend	45
5.2. Implementación de la base de datos	48
5.3. Implementación del backend	49
5.4. Implementación e integración de servicios	51
5.4.1. Implementación e integración del servicio de correo electrónico	51
5.4.2. Implementación e integración de los servicios de Google Cloud	52
5.4.3. Entrenamiento del agente de Dialogflow	54
5.4.4. Implementación e integración del bot de Discord	57
5.4.5. Implementación e integración de las funciones de RPA	61
6. Pruebas	63
6.0.1. Pruebas del sistema web	63
6.0.2. Pruebas del agente conversacional	69
6.0.3. Pruebas del bot de Discord	70
7. Conclusiones y trabajo futuro	73
7.1. Conclusiones	73
7.2. Trabajo futuro	74
Bibliografía	78
A. Manuales	79
A.1. Manual de instalación y despliegue	79
A.1.1. Dependencias del sistema	79
A.1.2. Instalación del proyecto	80
A.1.3. Configuración del agente de Dialogflow	80
A.1.4. Configuración de Google Cloud	80
A.1.5. Configuración de Vosk	81
A.1.6. Configuración del bot de Discord	81
A.1.7. Configuración de Ngrok	81
A.1.8. Despliegue del sistema	81

A.2. Manual de usuario	82
A.2.1. Login en la aplicación	82
A.2.2. Funcionalidades del administrador	83
A.2.3. Funcionalidades del secretario	87
A.2.4. Funcionalidades comunes	91
B. Resumen de enlaces adicionales	93

Lista de Figuras

2.1. Diagrama de casos de uso.	8
2.2. Modelo de dominio.	15
3.1. Diagrama de Gantt del proyecto.	18
4.1. Prototipo visual de la página de login.	24
4.2. Prototipo visual de la página de peticiones.	25
4.3. Prototipo visual de la página con detalles de una petición.	26
4.4. Prototipo visual de la página de una petición revisable.	26
4.5. Prototipo visual de la página de empleados.	27
4.6. Prototipo visual de la página de estadísticas.	28
4.7. Diagrama de descomposición y uso.	29
4.8. Diagrama Entidad-Relación.	30
4.9. Diagrama de flujo del agente conversacional.	32
4.10. Diagrama de secuencia de una llamada telefónica al sistema.	33
4.11. Diagrama de secuencia de inicio de sesión de un empleado.	34
4.12. Diagrama de secuencia de consulta de los detalles de una petición.	35
4.13. Diagrama de secuencia de asignación de una petición revisable.	36
4.14. Diagrama de secuencia de cambio de contraseña de un empleado.	37
4.15. Diagrama de secuencia de creación de una petición por un empleado.	38
4.16. Diagrama de secuencia de creación de un empleado.	39
4.17. Diagrama de secuencia de edición de un empleado.	40
4.18. Diagrama de secuencia de creación de un trámite.	41
4.19. Diagrama de secuencia de edición de un trámite.	42

5.1. Código de la plantilla base layout.jinja.	45
5.2. Fragmento de código de la construcción dinámica de la tabla de peticiones.	46
5.3. Fragmento de código de la plantilla resumen de una petición.	47
5.4. Fragmento de código de carga dinámica de las plantillas de cada trámite.	47
5.5. Código de la función de inicialización de la base de datos.	48
5.6. Fragmento de código de la consulta de peticiones.	49
5.7. JSON con los permisos de cada rol.	50
5.8. Código de las funciones para la gestión de permisos en la aplicación.	50
5.9. Código de la función de envío de correos electrónicos.	52
5.10. Código de la función de envío de transcripciones a Dialogflow.	53
5.11. Configuración del webhook en la consola de Dialogflow.	54
5.12. Establecimiento de frases de entrenamiento en Dialogflow.	54
5.13. Página de entrenamiento en Dialogflow.	55
5.14. Contexto de un intent en Dialogflow.	56
5.15. Parámetros de un intent en Dialogflow.	56
5.16. Entidad personalizada en Dialogflow.	57
5.17. Página de desarrolladores de Discord.	57
5.18. Servidor de Discord.	58
5.19. Código de la clase OpusDecoder.	59
5.20. Resumen de la lógica del bot de Discord.	60
5.21. Código de la función de RPA para solicitar el certificado de empadronamiento.	61
A.1. Formulario de inicio de sesión.	82
A.2. Página con la tabla de peticiones para el rol de administrador.	83
A.3. Página con la tabla de empleados.	84
A.4. Formulario para añadir un nuevo empleado al sistema.	84
A.5. Formulario para editar la información de un empleado existente.	85
A.6. Página con la tabla de trámites.	85
A.7. Formulario para añadir un nuevo trámite al sistema.	86
A.8. Formulario para editar la información de un trámite existente.	86
A.9. Página con la tabla de peticiones para el rol de secretario.	87

A.10. Formulario para la creación de peticiones manualmente.	87
A.11. Formulario para la creación de una petición del trámite de solicitud del certificado de empadronamiento.	88
A.12. Formulario para la creación de una petición del trámite de cita en la AEAT.	89
A.13. Formulario para la creación de una petición del trámite de solicitud de la tarjeta de SACYL.	89
A.14. Página con la información de una petición pendiente para el rol de secretario.	90
A.15. Página con la información de una petición asignada para el rol de secretario.	90
A.16. Cuadro de detalles del perfil del empleado.	91

Lista de Tablas

2.1. Descripción general del caso de uso - Solicitud del certificado de empadronamiento.	9
2.2. Descripción general del caso de uso - Solicitud de cita en la Agencia Tributaria.	10
2.3. Descripción general del caso de uso - Solicitud de la tarjeta sanitaria de SACYL por pérdida o robo.	11
2.4. Descripción general del caso de uso - Iniciar sesión.	11
2.5. Descripción general del caso de uso - Consultar detalles de una petición.	11
2.6. Descripción general del caso de uso - Petición revisable.	12
2.7. Descripción general del caso de uso - Consultar datos personales y cambiar contraseña. . .	12
2.8. Descripción general del caso de uso - Crear petición.	12
2.9. Descripción general del caso de uso - Añadir empleados.	13
2.10. Descripción general del caso de uso - Modificar o desactivar empleados.	13
2.11. Descripción general del caso de uso - Añadir trámites.	14
2.12. Descripción general del caso de uso - Modificar o desactivar trámites.	14
3.1. Planificación temporal inicial del proyecto.	17
3.2. Análisis de riesgos del proyecto.	19
3.3. Seguimiento del proyecto.	19
6.1. Caso de prueba WEB01 - Inicio de sesión exitoso	63
6.2. Caso de prueba WEB02 - Inicio de sesión denegado	63
6.3. Caso de prueba WEB03 - Buscar una petición	64
6.4. Caso de prueba WEB04 - Crear una petición manualmente	64
6.5. Caso de prueba WEB05 - Asignarse una petición	64
6.6. Caso de prueba WEB06 - Desasignarse una petición	64
6.7. Caso de prueba WEB07 - Editar los datos de una petición	64

6.8. Caso de prueba WEB08 - Cancelar una petición	65
6.9. Caso de prueba WEB09 - Completar una petición de certificado de empadronamiento . . .	65
6.10. Caso de prueba WEB10 - Completar una petición de cita en la AEAT	65
6.11. Caso de prueba WEB11 - Completar una petición de tarjeta de SACYL	66
6.12. Caso de prueba WEB12 - Buscar un empleado	66
6.13. Caso de prueba WEB13 - Crear un empleado	66
6.14. Caso de prueba WEB14 - Modificar información de un empleado	67
6.15. Caso de prueba WEB15 - Desactivar una cuenta de empleado	67
6.16. Caso de prueba WEB16 - Activar una cuenta de empleado	67
6.17. Caso de prueba WEB17 - Reestablecer contraseña de un empleado	67
6.18. Caso de prueba WEB18 - Buscar un trámite	67
6.19. Caso de prueba WEB19 - Crear un trámite	68
6.20. Caso de prueba WEB20 - Modificar el título de un trámite	68
6.21. Caso de prueba WEB21 - Desactivar un trámite	68
6.22. Caso de prueba WEB22 - Activar un trámite	68
6.23. Caso de prueba WEB23 - Cambiar la foto de perfil	68
6.24. Caso de prueba WEB24 - Cambiar la contraseña de la cuenta	68
6.25. Caso de prueba WEB25 - Cerrar sesión	69
6.26. Caso de prueba AC01 - Detección del trámite	69
6.27. Caso de prueba AC02 - Extracción de datos para el certificado de empadronamiento . . .	69
6.28. Caso de prueba AC03 - Extracción de datos para la cita de la AEAT	69
6.29. Caso de prueba AC04 - Extracción de datos para la tarjeta de SACYL	69
6.30. Caso de prueba AC05 - Confirmar los datos extraídos	70
6.31. Caso de prueba AC06 - Modificar uno de los datos extraídos	70
6.32. Caso de prueba AC07 - Responder preguntas sin perder el flujo	70
6.33. Caso de prueba DISC01 - Conexión al canal de voz	70
6.34. Caso de prueba DISC02 - Recepción de audio del canal de voz	70
6.35. Caso de prueba DISC03 - Reproducción de audio en el canal de voz	71
6.36. Caso de prueba DISC04 - Colgar la llamada	71

Capítulo 1

Introducción

Este documento constituye la memoria del Trabajo de Fin de Grado de la titulación de Grado en Ingeniería Informática, mención en Computación, de la Universidad de Valladolid. A través de este informe se exponen las fases y decisiones tomadas a lo largo del desarrollo de un sistema computacional orientado a la gestión accesible de trámites burocráticos.

En este capítulo se presentan los motivos que justifican el valor de esta plataforma para determinados sectores de la sociedad y las soluciones similares existentes que pretenden reducir la brecha digital, destacando las diferencias con el proyecto propuesto. También se indican los objetivos que se pretenden alcanzar con esta herramienta y, finalmente, un resumen estructural del resto de la memoria.

1.1. Motivación del tema

La brecha digital se define como la desigualdad existente entre personas por causa de su exclusión a las tecnologías de la información y comunicación. Esta exclusión se manifiesta desde tres dimensiones diferentes.

- En primer lugar, en la falta de acceso a dispositivos tecnológicos o a una conexión a Internet. Esto puede deberse tanto a factores geográficos como a limitaciones económicas. En España, un 20 % de la población no dispone de un ordenador, y en hogares con ingresos inferiores a 1100€, el porcentaje asciende al 50 % con tan solo el 79,1 % con acceso a Internet [1].
- En segundo lugar, en el uso de estas tecnologías. Una parte de la población carece de las competencias digitales necesarias para desenvolverse con facilidad en este entorno. Esto influye en gran medida cuando pretenden afrontar tareas más complejas, como la realización de trámites relacionados con la Administración pública, hasta el punto de que un 20 % de la población necesita ayuda para completar estas gestiones [1].
- Por último, hay una brecha de aprovechamiento referida a la dificultad de obtener beneficios de las tecnologías, por ejemplo, para acceder a mejores oportunidades educativas o mejores puestos de trabajo. No obstante, este proyecto no se centra en esta dimensión.

Aunque se trata de un porcentaje reducido de la población, resulta evidente la necesidad de ofrecer soluciones que permitan a estas personas integrarse en el entorno digital [1].

Teniendo en cuenta estas limitaciones, uno de los sectores más afectados por la brecha digital es el de las personas mayores, que no suelen estar tan familiarizados con las tecnologías y les afecta en gran medida la falta de acceso y de uso. Tanto es así que un 27,3 % de los españoles mayores de 65 años nunca han utilizado Internet [2]. Por ello, es fundamental desarrollar una solución universal e intuitiva,

que no requiera de ningún conocimiento previo. En este sentido, una llamada telefónica es una opción muy adecuada, ya que es un medio de comunicación ampliamente utilizado por todos los sectores de la población, no requiere explicación previa, funciona incluso en zonas rurales de baja cobertura y no supone una barrera económica adicional, ya que el 99,8 % de los hogares españoles dispone de un teléfono ya sea móvil o fijo [3]. De esta manera, se garantiza la accesibilidad del servicio y no se imponen nuevas limitaciones tecnológicas.

Además, se opta por centrar el proyecto en las diligencias de la Administración pública, ya que la digitalización de estos servicios ha supuesto una barrera adicional, dificultando su realización para los sectores vulnerables. Tal es el caso que el 60 % de la población opina que la informatización de los trámites administrativos vulnera sus derechos y el 74,5 % desearía disponer de un portal único para realizar todos los procedimientos [4].

1.2. Estado de la cuestión

A lo largo de los últimos años se han desarrollado múltiples iniciativas orientadas a reducir la brecha digital. Sin embargo, ninguna de ellas ha logrado beneficiar a la mayoría de los sectores de la población, independientemente del grado en que se ven afectados por dicha brecha. El proyecto presentado busca dar respuesta a esta desigualdad, abordándola desde la perspectiva de la falta de acceso y la dificultad de uso.

La mayoría de las soluciones existentes se centran en la formación y capacitación de los usuarios para introducirlos en el entorno digital. Existen múltiples cursos, talleres e incluso plataformas digitales como Cóginiti [5] que cumplen esta función. No obstante, presentan varias desventajas como la necesidad de disponer de un dispositivo y una conexión a Internet, lo que sigue suponiendo una barrera de acceso. Además, el aprendizaje requiere de un tiempo y esfuerzo que no todos los usuarios están dispuestos a invertir. A esto se suma que no todas las personas aprenden con la misma facilidad y, en muchos casos, los resultados obtenidos son limitados, ya que se adquieren únicamente conceptos básicos que no siempre se traducen en saber realizar tareas más complejas.

En cuanto a iniciativas orientadas a reducir la brecha de acceso, existen distintos proyectos para ampliar la cobertura en zonas desconectadas y volver las tecnologías más asequibles para personas con recursos limitados. Asimismo, están surgiendo diversas tecnologías, como la red 5G o el Internet satelital, que contribuyen a mejorar la conectividad en áreas donde las infraestructuras tradicionales, como la fibra óptica, resultan insuficientes o inviables. Sin embargo, estas soluciones no abordan la brecha de uso, que sigue siendo un obstáculo para muchos usuarios.

En lo relativo a las páginas oficiales del gobierno, ha habido un esfuerzo por reducir la complejidad de sus trámites y hacerlos más accesibles mediante aplicaciones móviles. Pese a este esfuerzo, las aplicaciones móviles no eximen a los usuarios con competencias digitales limitadas de necesitar ayuda externa para completar las gestiones.

Por último, en cuanto a soluciones que involucren llamadas telefónicas, existen líneas como el 060 para la atención administrativa general del Estado, que se utiliza principalmente para ofrecer información o resolver dudas. Además, existen sistemas más avanzados que incorporan reconocimiento del lenguaje natural para gestiones específicas, como la solicitud de citas médicas telefónicas de SACYL. No obstante, estas soluciones son bastante limitadas, ya que generalmente obligan al usuario a interactuar mediante menús predefinidos y suelen necesitar también de un operador humano para completar gestiones más complejas.

Del mismo modo, existen servicios con bots de atención automatizada, que al igual que en el caso anterior, tienen una funcionalidad muy limitada y no permiten la realización de trámites complejos. A

diferencia de estos sistemas, una inteligencia artificial conversacional avanzada permite mantener una interacción más natural, similar a una conversación humana, sin exigir respuestas rígidas. Asimismo, ofrecen una mayor personalización y flexibilidad, ya que pueden entrenarse para realizar cualquier tarea o gestión necesaria, así como responder a cualquier duda sin interrumpir la conversación. Por ejemplo, un usuario puede estar solicitando un trámite y mientras la IA está recopilando los datos necesarios, el usuario puede interrumpir con una pregunta relacionada, y la IA responderá adecuadamente y continuará el trámite sin romper el flujo de la conversación.

Esta tecnología de inteligencia artificial conversacional está en constante evolución y mejora, ya que es relativamente reciente. Sin embargo, ya empiezan a aparecer soluciones comerciales que la integran en sus modelos de negocio para automatizar procesos de alto valor y reducir costes.

En conclusión, la mayoría de las soluciones existentes se centran únicamente en un aspecto de la brecha digital y no logran integrar de manera efectiva a todos los sectores de la población. Si bien se obtiene un buen resultado con la combinación de varias iniciativas, sigue sin ser equiparable a un sistema integral como el presentado.

1.3. Objetivos

El objetivo principal de este proyecto consiste en la reducción de la brecha digital mediante un sistema accesible que ofrece una alternativa para la realización de trámites administrativos online a cualquier persona, independientemente de su nivel de acceso o sus competencias digitales, a través de una llamada telefónica. Para ello, se han establecido los siguientes objetivos específicos:

1. Entrenamiento y configuración de un agente conversacional de inteligencia artificial, capaz de interactuar de forma natural y fluida con el usuario, con el fin de orientar y recopilar la información necesaria del beneficiario para iniciar los trámites administrativos.
2. Desarrollo de una interfaz web administrativa para la gestión y supervisión de las solicitudes realizadas a través del sistema.
3. Simulación de un conjunto representativo de procedimientos administrativos de diferentes entidades y tipologías.
4. Implementación de un sistema de llamadas Voice over IP (VoIP) para la recepción de llamadas y la integración del agente conversacional.

1.4. Estructura de la memoria

Este documento se estructura en seis capítulos principales, además de la introducción, donde se presenta el proyecto. Dichos capítulos siguen la estructura propia del desarrollo de un sistema computacional. A continuación se resume su contenido:

Capítulo 2. Análisis Se estudian las necesidades del público objetivo del sistema. En base a este estudio, se establecen los requisitos del sistema y los trámites y procesos que se deben implementar, así como el alcance y las limitaciones de la solución.

Capítulo 3. Planificación Se organiza y estructura el trabajo a realizar. En este capítulo se define la metodología empleada, se establece la planificación temporal del proyecto y se realiza una estimación de recursos y costes.

Capítulo 4. Diseño Se explican las decisiones tecnológicas tomadas, se detalla la arquitectura general y los distintos componentes del sistema. Asimismo, se presentan los diagramas y patrones de diseño elegidos, que servirán de base para la implementación.

Capítulo 5. Implementación Se describe el proceso de desarrollo del sistema, detallando la implementación de la aplicación web, el entrenamiento del agente conversacional, el desarrollo del sistema de llamadas VoIP y la integración de todos los componentes del sistema.

Capítulo 6. Pruebas Se exponen las distintas pruebas realizadas para verificar el correcto funcionamiento de cada uno de los componentes del sistema.

Capítulo 7. Conclusiones y trabajo futuro Se resumen los resultados obtenidos y se proponen distintas líneas de mejora y ampliación del proyecto para su desarrollo futuro.

Capítulo 2

Análisis

2.1. Público objetivo

La audiencia objetivo de este proyecto se enfoca principalmente en los usuarios finales, pero también incluye a las entidades administrativas y a sus trabajadores que se beneficiarán de este sistema.

Los usuarios finales son los colectivos más afectados por la brecha digital, principalmente las personas con falta de competencias digitales o sin acceso a la tecnología. En particular, los principales beneficiarios serán las personas mayores, que suelen presentar mayores dificultades con la tecnología, especialmente en lo respectivo a la realización de trámites administrativos online. Aunque también, podrán verse favorecidos otros colectivos vulnerables de este sistema, como personas con discapacidades físicas o cognitivas que les dificulten el uso de dispositivos tecnológicos, personas con bajos recursos económicos que no puedan permitirse las tecnologías, personas que vivan en zonas rurales o aisladas de la conexión a la red.

Por otro lado, las entidades administrativas también forman parte de la audiencia objetivo, ya que se ven beneficiadas de la automatización de los trámites, reduciendo la carga de trabajo de sus empleados y liberándolos de tareas más engorrosas. Al disponer de una línea telefónica atendida por inteligencia artificial, los empleados pueden dejar de atender multitud de peticiones diarias y liberar tiempo para otras tareas. Además, con la ayuda de la plataforma web administrativa, pueden gestionar y supervisar las solicitudes de una manera eficiente.

Por último, cualquier persona fuera de estas características puede utilizar el sistema sin ningún problema, en caso de que no tengan un ordenador a mano o que simplemente les resulte más cómodo para ahorrar tiempo.

2.2. Alcance del sistema

El proyecto contempla el desarrollo de un sistema de gestión de trámites administrativos simulado capaz de hacer una representación fiel de la implementación futura real. Para ello se han definido unos límites, quedando incluidos en el alcance del proyecto los siguientes aspectos:

- La gestión de trámites administrativos de organismos públicos, limitados a la provincia de Valladolid, con el fin de acotar el desarrollo, evitando una generalización excesiva que dificulte la implementación del prototipo y manteniendo la representatividad del sistema.
- Una plataforma web limitada a la administración y supervisión de los empleados, los trámites y las solicitudes realizadas a través del sistema.

- Un sistema de llamadas VoIP para la iniciación y recepción de llamadas, que simule la red telefónica.
- Un agente conversacional de inteligencia artificial capaz de informar y recopilar los datos necesarios de los trámites seleccionados.

Y quedando fuera del alcance del proyecto los siguientes aspectos:

- La integración con sistemas reales de las Administraciones públicas, ya que el objetivo es simular con datos ficticios el inicio y cómo se llevaría a cabo el trámite, pero no se pretende completar ningún proceso real.
- El uso del agente de inteligencia artificial para otros fines distintos a la gestión de los trámites seleccionados.
- La utilización de la infraestructura de telefonía tradicional, dado que una solución VoIP reduce los costes asociados al uso y alquiler de líneas y ofrece una experiencia de interacción equivalente para el usuario.
- El proyecto contempla solo fases de desarrollo y pruebas; no entra en el alcance del proyecto fases de producción, envío real de correos electrónicos ni la publicación de la plataforma web.

2.2.1. Trámites

La selección de los trámites incluidos en el sistema es uno de los pasos fundamentales en el análisis, ya que determina el realismo del sistema. La importancia recae en incorporar distintos tipos de gestiones de diferentes organizaciones que incluyan distintos tipos de cumplimiento, de tal forma que la simulación sea variada y representativa.

Además, no sirve con elegir cualquier trámite, tienen que ser de valor para las personas objetivo, ofreciéndoles una alternativa útil que vayan a utilizar frente a las otras soluciones existentes. Los trámites seleccionados son los siguientes:

- Trámite 1. Solicitud del certificado de empadronamiento en el ayuntamiento de Valladolid. Este documento es necesario para la realización de otros trámites, como la solicitud de ayudas o beneficios exclusivos para residentes locales. Es una gestión factible que se realiza online mediante un formulario, frente a la alternativa de acudir al ayuntamiento presencialmente para obtenerlo.
- Trámite 2. Solicitar una cita en la Agencia Tributaria en la provincia de Valladolid. Aunque ya se ha comentado que hay algunas Administraciones que ya implementan bots de llamadas para solicitar citas previas, esta tecnología los mejora. Para nuestro sistema simulado nos vamos a centrar en la AEAT ya que ofrece servicios de valor para el público objetivo, y la cita telefónica que disponen actualmente solo es atendida por personal durante unas horas concretas. Con esta implementación se ampliaría el soporte a cualquier hora y día y se liberaría trabajo a los operadores de la agencia.
- Trámite 3. Solicitar la tarjeta sanitaria de SACYL por pérdida o robo en la provincia de Valladolid. Este procedimiento cumple también con los requisitos de ser un trámite útil, ya que, el usuario objetivo requiere de los servicios de salud pública con mayor frecuencia y la tarjeta sanitaria es necesaria para acceder a estos servicios. Además, se realiza también mediante un formulario en línea, evitando la necesidad de acudir presencialmente al centro de salud, evitando desplazamientos y esperas innecesarias.

2.3. Requisitos

A continuación, se elicitán los requisitos funcionales, no funcionales y de información que debe cumplir el sistema:

- RF1. El sistema deberá permitir a los usuarios finales interactuar con él a través de una conversación telefónica.
- RF2. El usuario podrá solicitar su certificado de empadronamiento de la ciudad de Valladolid.
- RF3. El usuario podrá reservar una cita en la agencia tributaria para la provincia de Valladolid.
- RF4. El usuario podrá obtener su tarjeta sanitaria de SACYL en caso de pérdida o robo en la provincia de Valladolid.
- RF5. El sistema deberá permitir a los empleados identificarse con su usuario y contraseña en la plataforma web administrativa.
- RF6. El sistema deberá limitar las funcionalidades de la aplicación web en función del rol del empleado (administrador o secretario).
- RF7. El sistema deberá almacenar todas las peticiones registradas a través de la IA.
- RF8. El sistema mostrará a los empleados un listado con todas las peticiones del sistema y permitirá seleccionarlas individualmente para mostrar información de esa petición concreta.
- RF9. El sistema deberá permitir a los secretarios de la Administración correspondiente asignarse y resolver manualmente las peticiones revisables que requieran asistencia humana.
- RF10. El sistema deberá permitir a los secretarios crear una petición manualmente en la página web con el objetivo de subsanar situaciones excepcionales derivadas de errores humanos o fallos del sistema.
- RF11. El sistema deberá permitir a los administradores añadir otros empleados al sistema.
- RF12. El sistema mostrará a los administradores una lista con los empleados registrados en el sistema, permitiendo desactivarlos o modificar su información.
- RF13. El sistema deberá permitir a los administradores añadir trámites al sistema.
- RF14. El sistema mostrará a los administradores una lista con los trámites del sistema, permitiendo desactivarlos o modificar su información.
- RF15. El sistema permitirá a los empleados consultar su información personal y modificar su contraseña.
- RFI1. El sistema deberá almacenar la siguiente información con respecto a las peticiones:
 - ID (identificador único de cada petición)
 - Tipo (Trámite 1, 2 o 3)
 - Teléfono (número del llamante)
 - Información (datos extraídos por la IA en la llamada)
 - Estado (creada, pendiente, asignada, modificada, completada o cancelada)
 - Fecha de creación
 - Fecha de modificación (empleado que se asignó la tarea)
- RFI2. El sistema deberá almacenar la siguiente información con respecto a los empleados:
 - Usuario
 - Contraseña
 - Nombre
 - Primer apellido
 - Segundo apellido
 - Email
 - Rol (administrador o secretario)
 - Foto de perfil
 - Estado (activo o inactivo)
- RFI3. El sistema deberá almacenar la siguiente información con respecto a los trámites:
 - ID (identificador único de cada trámite)
 - Nombre del trámite
 - Estado (activo o inactivo)
- RNF1. Los errores registrados por la IA, por falta de comprensión o entrenamiento, no deben superar el 10 % de las peticiones.
- RNF2. El tiempo de respuesta del agente conversacional en llamada no debe superar los 3 segundos para dar sensación de fluidez en la conversación.
- RNF3. La interfaz de la web de empleados será intuitiva y fácil de usar, de forma que el 95 % de los

empleados puedan completar tareas clave con menos de 2 errores y en menos de 5 minutos por tarea.

2.4. Casos de uso

2.4.1. Diagrama de casos de uso:

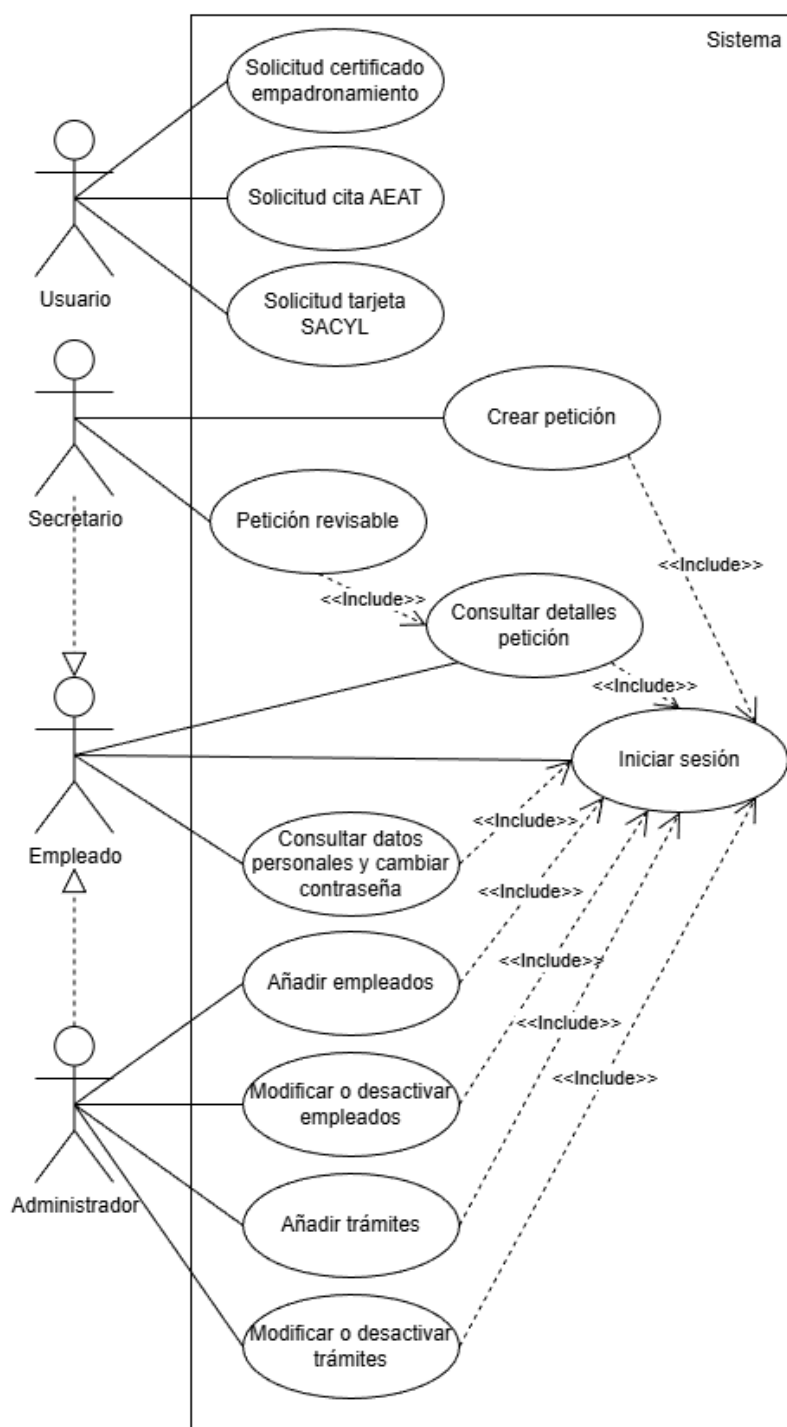


Figura 2.1: Diagrama de casos de uso.

2.4.2. Actores:

1. Usuario: persona ajena a la Administración pública que necesita un servicio de la misma. Se comunica con el sistema mediante llamadas telefónicas.
2. Empleados: representa a un trabajador de los organismos públicos. Tienen acceso a la web interna y desempeñan distintas acciones en función de su rol.
 - Administrador: su función principal es la gestión de empleados y trámites en la plataforma interna.
 - Secretario: se encargan de supervisar y completar las peticiones. También pueden crear peticiones manualmente en caso de que sea necesario, ya sea por subsanar algún error o por cualquier otra razón.

2.4.3. Solicitud del certificado de empadronamiento:

Solicitud del certificado de empadronamiento
Actor principal: Usuario
Precondición: El usuario ha iniciado la llamada al sistema
Secuencia normal: 1. El usuario solicita obtener el certificado de empadronamiento. 2. El sistema solicita el DNI. 3. El usuario proporciona el DNI. 4. El sistema solicita el nombre. 5. El usuario proporciona su nombre. 6. El sistema solicita los dos apellidos. 7. El usuario proporciona sus dos apellidos. 8. El sistema solicita el teléfono. 9. El usuario proporciona su teléfono. 10. El sistema solicita el motivo de la petición. 11. El usuario responde el motivo de la petición. 12. El sistema repite la información obtenida y solicita confirmación. 13. El usuario confirma los datos. 14. El sistema almacena la petición. 15. El sistema rellena el formulario de solicitud del certificado de empadronamiento con los datos obtenidos.
Flujos alternativos: 11.a Si el usuario no confirma los datos, el sistema repite las preguntas correspondientes y se continúa con el paso siguiente a la pregunta.

Tabla 2.1: Descripción general del caso de uso - Solicitud del certificado de empadronamiento.

2.4.4. Solicitud de cita en la Agencia Tributaria:

Solicitud de cita en la Agencia Tributaria
Actor principal: Usuario.
Precondición: El usuario ha iniciado una llamada al sistema.
Secuencia normal:

1. El usuario solicita agendar una cita en la Agencia Tributaria. 2. El sistema solicita el DNI. 3. El usuario proporciona su DNI. 4. El sistema solicita el nombre y el primer apellido. 5. El usuario proporciona su nombre y sus primer apellido. 6. El sistema solicita el tipo de servicio. 7. El usuario proporciona el tipo de servicio. 8. El sistema pregunta si prefiere cita telefónica o presencial. 9. El usuario responde que presencial. 10. El sistema solicita la oficina. 11. El usuario responde la oficina. 12. El sistema solicita fecha y hora preferible para la cita. 13. El usuario responde la fecha y hora. 14. El sistema comprueba que la fecha y hora es válida. 15. El sistema repite la información obtenida y solicita confirmación. 16. El usuario confirma los datos. 17. El sistema almacena la petición. 18. El sistema rellena el formulario para agendar la cita.
Flujos alternativos: 9.a. Si el usuario responde cita telefónica se continúa con el paso 15. 14.a. Si la fecha u hora no es válida se vuelve al paso 12. 16.a Si el usuario no confirma los datos, el sistema repite las preguntas correspondientes y se continúa con el paso siguiente a la pregunta.

Tabla 2.2: Descripción general del caso de uso - Solicitud de cita en la Agencia Tributaria.

2.4.5. Solicitud de la tarjeta sanitaria de SACYL por pérdida o robo:

Solicitud de la tarjeta sanitaria de SACYL por pérdida o robo
Actor principal: Usuario.
Precondición: el usuario ha iniciado una llamada al sistema.
Secuencia normal: 1. El usuario solicita la tarjeta sanitaria. 2. El sistema solicita el motivo (pérdida o robo). 3. El usuario proporciona el motivo (pérdida o robo). 4. El sistema solicita el nombre. 5. El usuario proporciona su nombre. 6. El sistema solicita el primer apellido. 7. El usuario proporciona su primer apellido. 8. El sistema solicita la fecha de nacimiento. 9. El usuario proporciona su fecha de nacimiento. 10. El sistema pregunta el centro de salud. 11. El usuario responde su centro de salud. 12. El sistema solicita la dirección. 13. El usuario responde la dirección. 14. El sistema repite la información obtenida y solicita confirmación. 15. El usuario confirma los datos. 16. El sistema almacena la petición. 17. El sistema rellena el formulario para solicitar la tarjeta sanitaria.

Flujos alternativos:

- 15.a Si el usuario no confirma los datos, el sistema repite las preguntas correspondientes y se continúa con el paso siguiente a la pregunta.

Tabla 2.3: Descripción general del caso de uso - Solicitud de la tarjeta sanitaria de SACYL por pérdida o robo.

2.4.6. Iniciar sesión:

Iniciar sesión
Actor principal: Empleado.
Secuencia normal: 1. El sistema solicita el usuario o correo electrónico y la contraseña. 2. El empleado introduce su usuario o correo electrónico y su contraseña. 3. El sistema comprueba que los datos coinciden con un empleado e inicia sesión.
Flujos alternativos: 3.a. Si los datos no coinciden con un empleado, el sistema informa de datos incorrectos y vuelve al paso 1. 3.b. Si los datos coinciden pero el usuario está desactivado, el sistema lo informa y vuelve al paso 1.
Postcondiciones: el empleado está identificado en el sistema.

Tabla 2.4: Descripción general del caso de uso - Iniciar sesión.

2.4.7. Consultar detalles de una petición:

Consultar detalles de una petición
Actor principal: Empleado.
Precondición: caso de uso 2.4.5. Inicia sesión.
Secuencia normal: 1. El empleado va a la página con la lista de peticiones. 2. El empleado clicla en el ID de la petición. 3. El sistema muestra la información de la petición.

Tabla 2.5: Descripción general del caso de uso - Consultar detalles de una petición.

2.4.8. Petición revisable:

Petición revisable
Actor principal: Secretario.
Precondición: caso de uso 2.4.6. Consultar detalles de una petición.
Secuencia normal:

1. El secretario se asigna la petición. 2. El secretario modifica la información necesaria. 3. El secretario completa la petición. 4. El sistema inicia el trámite correspondiente a la petición. 5. El sistema actualiza el estado de la petición a completada.
Flujos alternativos: 3.a. Si el secretario anula la petición, cambia y actualiza el estado a cancelada y finaliza el caso de uso.

Tabla 2.6: Descripción general del caso de uso - Petición revisable.

2.4.9. Consultar datos personales y cambiar contraseña:

Consultar datos personales y cambiar contraseña
Actor principal: Empleado.
Precondición: caso de uso 2.4.5. Inicia sesión.
Secuencia normal: 1. El empleado clic en su perfil. 2. El sistema muestra su información personal. 3. El empleado clic en cambiar contraseña. 4. El sistema solicita la contraseña actual y la contraseña nueva. 5. El empleado introduce la información. 6. El sistema comprueba que la contraseña actual es verídica. 7. El sistema actualiza la contraseña del empleado.
Flujos alternativos: 6.a. Si la contraseña actual es incorrecta, el sistema lo informa y vuelve al paso 4.

Tabla 2.7: Descripción general del caso de uso - Consultar datos personales y cambiar contraseña.

2.4.10. Crear petición:

Crear petición
Actor principal: Secretario.
Precondición: caso de uso 2.4.5. Inicia sesión.
Secuencia normal: 1. El secretario va a la página con la lista de peticiones. 2. El secretario clic en crear petición. 3. El sistema solicita el teléfono y el trámite. 4. El secretario introduce el teléfono y el trámite. 5. El sistema solicita la información correspondiente al trámite. 6. El secretario introduce la información. 7. El sistema crea la petición e inicia el trámite.

Tabla 2.8: Descripción general del caso de uso - Crear petición.

2.4.11. Añadir empleados:

Añadir empleados
Actor principal: Administrador.
Precondición: caso de uso 2.4.5. Inicia sesión.
Secuencia normal: 1. El administrador va a la página con la lista de empleados. 2. El administrador selecciona añadir empleado. 3. El sistema solicita los datos del nuevo empleado. 4. El administrador rellena los datos. 5. El sistema comprueba que el email no está registrado ya en el sistema. 6. El sistema crea un usuario y una contraseña temporal. 7. El sistema envía al nuevo usuario su contraseña temporal. 8. El sistema almacena el nuevo empleado.
Flujos alternativos: 5.a. Si el email ya está registrado, el sistema lo informa y continúa con el paso 3.

Tabla 2.9: Descripción general del caso de uso - Añadir empleados.

2.4.12. Modificar o desactivar empleados

Modificar o desactivar empleados
Actor principal: Administrador.
Precondición: caso de uso 2.4.5. Inicia sesión.
Secuencia normal: 1. El administrador va a la página con la lista de empleados. 2. El administrador clic en el botón de la columna activo del empleado correspondiente. 3. El sistema muestra los campos con la información del empleado. 4. El administrador modifica la información correspondiente y clic en editar. 5. El sistema actualiza la información del empleado.
Flujos alternativos: 3.a. El administrador clic en resetear contraseña, el sistema crea una contraseña temporal y se la envía al email del empleado. 4.a. El administrador modifica el email, el sistema comprueba que el email ya está registrado en el sistema, lo informa y vuelve al paso 3. 4.b. El administrador modifica el email, el sistema comprueba que el email no está registrado en el sistema y continúa con el paso 5. 4.c. El administrador activa o desactiva el empleado, el sistema se lo avisa al email del empleado y continúa en el paso 5.

Tabla 2.10: Descripción general del caso de uso - Modificar o desactivar empleados.

2.4.13. Añadir trámites:

Añadir trámites
Actor principal: Administrador.
Precondición: caso de uso 2.4.5. Inicia sesión.
Secuencia normal:

1. El administrador va a la página con la lista de trámites. 2. El administrador clic en añadir trámite. 3. El sistema solicita el nombre y estado del trámite. 4. El administrador introduce el nombre y estado del trámite. 5. El sistema comprueba que no hay otro trámite con el mismo nombre. 6. El sistema actualiza los datos del trámite.
Flujos alternativos: 5.a. El sistema comprueba que hay otro trámite con el mismo nombre, lo informa y vuelve al paso 3.

Tabla 2.11: Descripción general del caso de uso - Añadir trámites.

2.4.14. Modificar o desactivar trámites:

Modificar o desactivar trámites
Actor principal: Administrador.
Precondición: caso de uso 2.4.5. Inicia sesión.
Secuencia normal: 1. El administrador va a la página con la lista de trámites. 2. El administrador clic en el botón de la columna activo del trámite correspondiente. 3. El sistema muestra los campos con la información del trámite. 4. El administrador modifica la información correspondiente y clic en editar. 5. El sistema actualiza la información del trámite.
Flujos alternativos: 4.a. El administrador modifica el título del trámite, el sistema comprueba que el título ya está registrado en el sistema, lo informa y vuelve al paso 3. 4.b. El administrador modifica el título del trámite, el sistema comprueba que el título no está registrado en el sistema y continúa con el paso 5. 4.c. Si el administrador desactiva un trámite activo, el sistema cancela todas las peticiones no completadas de ese trámite y va al paso 5.

Tabla 2.12: Descripción general del caso de uso - Modificar o desactivar trámites.

2.5. Modelo de dominio:

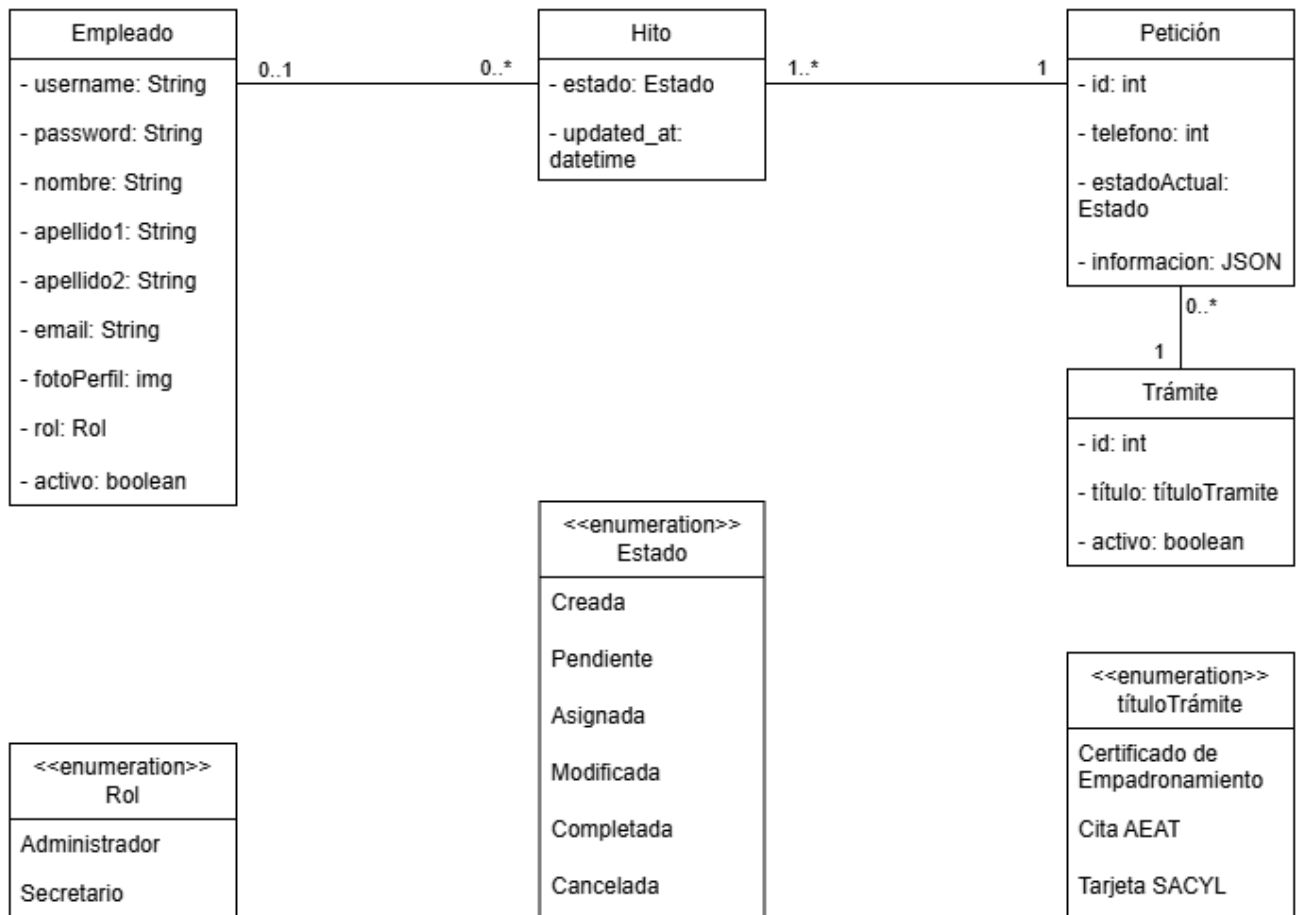


Figura 2.2: Modelo de dominio.

Capítulo 3

Planificación

3.1. Metodología

Para el desarrollo del proyecto se ha seguido un modelo de desarrollo estructural en cascada. Este enfoque divide el proceso de desarrollo en fases claramente diferenciadas que se suceden de forma secuencial, comenzando cada fase una vez finalizada la anterior.

Las fases típicas de este modelo incluyen el análisis, el diseño, la implementación, las pruebas y el mantenimiento. No obstante, al tratarse de un proyecto académico, la fase de mantenimiento no es necesaria y se sustituye por la documentación final de la memoria.

Se emplea esta metodología debido a que el objetivo final del proyecto está claramente definido desde el inicio, con unos requisitos bien establecidos y sin necesidad de iteraciones incrementales. Por ello, la planificación del proyecto se ha realizado tomando como base el análisis inicial, permitiendo organizar de forma coherente y estructural el desarrollo del trabajo.

3.2. Planificación temporal

La organización temporal del proyecto se ha realizado siguiendo la dedicación exigida por la guía docente de la asignatura [6]. El proyecto ha sido planificado para ser elaborado en un total de 300 horas, distribuidas en 15 semanas, dedicando aproximadamente 20 horas semanales al desarrollo del mismo. Se ha repartido el trabajo en 6 fases, estructuradas temporalmente según la Tabla 3.1 y representadas en el diagrama de Gantt de la Figura 3.1.

Fase	Semanas	Horas	Entregables
Análisis	1	10	Estudio del alcance de la aplicación, definición de requisitos y casos de uso
Planificación	1	10	Planificación temporal y seguimiento del proyecto
Diseño	2	20	Decisiones técnicas, diagramas UML, diseño de la base de datos y mockups
Implementación	3-8	120	Sistema funcional
Pruebas	8-9	40	Validación del sistema
Documentación	10-15	100	Memoria final y presentación

Tabla 3.1: Planificación temporal inicial del proyecto.

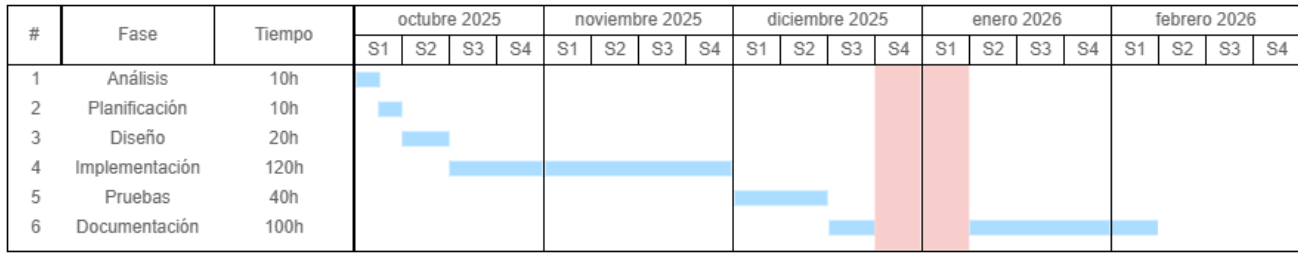


Figura 3.1: Diagrama de Gantt del proyecto.

3.3. Estimación de costes

La estimación de costes se ha realizado considerando dos escenarios: un entorno profesional teórico y el coste real del desarrollo académico.

3.3.1. Coste en un entorno profesional

En un entorno profesional, el principal coste asociado a la elaboración del proyecto sería el coste del personal. Dado que el desarrollo se ha planificado siguiendo un modelo en cascada, donde cada fase se ejecuta de forma secuencial, se asume que todas las tareas podrían ser realizadas por un único profesional.

Para la estimación económica, se ha tomado como referencia el salario medio de un desarrollador fullstack junior en España, estimado en torno a 16€/h, lo que supone un coste total aproximado de 4800€ para las 300 horas de trabajo planificadas.

Respecto al coste material, no se prevén gastos adicionales, ya que se utilizan herramientas gratuitas y servicios con planes gratuitos. Podría considerarse el coste de una estación de trabajo para el empleado, estimado en torno a 1000€, no obstante, este coste se considera amortizado entre múltiples proyectos.

Aunque queda fuera del alcance de este proyecto, y por tanto no se considera en esta estimación, en un entorno profesional sería necesario tener en cuenta el despliegue del sistema. Esto implicaría gastos en plataformas como Google Cloud, donde los costes dependerían del uso de servicios como Dialogflow y Google Cloud Speech-to-Text, siguiendo un modelo de pago por uso, así como los costes mensuales asociados al almacenamiento de información y al alojamiento de la aplicación web.

3.3.2. Coste real del proyecto

El coste real del proyecto es de 0€ debido a los siguientes factores:

- No se consideran costes de personal, ya que se trata de un proyecto académico desarrollado de forma individual.
- Se utilizan herramientas y tecnologías completamente gratuitas o con planes gratuitos con créditos suficientes para cubrir las necesidades del proyecto.
- No es necesario adquirir hardware adicional, ya que se trabaja con un equipo disponible previamente.

3.4. Análisis de riesgos

En esta sección se muestran los principales riesgos identificados durante la planificación del proyecto, junto con su probabilidad de ocurrencia, su impacto y las posibles medidas para mitigarlos, Talba 3.2.

Riesgo	Probabilidad	Impacto	Salvaguarda
Créditos insuficientes en servicios en la nube	Media	Alto	Monitorización del gasto de créditos y evitar el uso innecesario de recursos.
Incremento inesperado del coste de las tecnologías utilizadas	Baja	Medio	Evaluación continua de los costes asociados y búsqueda de alternativas gratuitas o de bajo coste.
Indisponibilidad temporal de los servicios externos	Baja	Alto	Uso de servicios consolidados con buen soporte.
Aparición de imprevistos que afecten al tiempo de desarrollo	Alta	Medio	Planificación de tiempo adicional para imprevistos y margen con la fecha límite de entrega.
Complejidad técnica en la integración de los distintos componentes del sistema	Media	Medio	Formación y estudio previo de las tecnologías utilizadas.

Tabla 3.2: Análisis de riesgos del proyecto.

3.5. Seguimiento del proyecto

A lo largo del desarrollo del proyecto, se ha llevado un seguimiento detallado para controlar el gasto temporal real frente al planificado. En la Tabla 3.3 se muestra la comparativa entre la planificación temporal inicial y el progreso real del proyecto, permitiendo identificar desajustes y calcular el coste real del proyecto.

Fase	Horas Programadas	Horas Reales
Análisis	10	15
Planificación	10	8
Diseño	20	25
Implementación	120	180
Pruebas	40	35
Documentación	100	90

Tabla 3.3: Seguimiento del proyecto.

El gasto temporal real del proyecto asciende a 353 horas por imprevistos en la fase de implementación.

Inicialmente, estaba previsto el uso de la red telefónica para la comunicación entre los usuarios y el sistema, lo que no aumentaba el gasto temporal ya que venía integrado con los servicios de la IA. Sin embargo, debido al elevado coste de esta solución en desarrollo, se optó por una solución basada en llamadas VoIP, que resulta completamente gratuita, pero implica un desarrollo adicional para integrar esta funcionalidad en el sistema.

En cuanto al coste real del proyecto, el gasto ascendería a 5295€, superando el presupuesto inicial en 795€. El gasto de la red telefónica hubiera supuesto un coste adicional de 1,50€ de establecimiento de

llamada más 1€ adicional por cada minuto. Además, el coste de alquiler de una línea telefónica virtual hubiera implicado un gasto mensual de entre 5-10€/mes. Estos gastos hubieran complicado el desarrollo del proyecto, limitando el número de llamadas que se podrían realizar durante la fase de implementación y pruebas, lo que hubiera afectado negativamente a la calidad del sistema.

Capítulo 4

Diseño

4.1. Decisiones técnicas

En esta sección se plantean las herramientas y tecnologías seleccionadas para llevar a cabo el desarrollo del proyecto, justificando su elección en función de las necesidades y limitaciones del sistema.

4.1.1. Frontend

Para programar el frontend de la aplicación web orientada a empleados se utilizan HTML, CSS y JavaScript.

Adicionalmente, se utiliza Jinja2 [7], un motor de plantillas integrado con Flask que permite generar contenido HTML de manera dinámica, facilitando la interacción con el backend.

4.1.2. Backend

Para la implementación de la lógica del sistema se utiliza Python, que actúa como conexión entre el agente conversacional, la base de datos y la aplicación web.

Este lenguaje se selecciona por su facilidad y versatilidad para integrar todos los servicios. Algunas de las librerías elegidas para llevar a cabo el desarrollo son:

- Flask [8]: es un framework web de Python, se emplea para desarrollar el backend de la aplicación.
- SQLAlchemy [9]: herramienta para interactuar con la base de datos de manera sencilla y eficiente, permitiendo una gestión efectiva de los datos mediante un modelo orientado a objetos.
- Discord.py: librería empleada para interactuar con la plataforma de Discord y desarrollar el bot encargado de atender las llamadas VoIP.
- Google-Cloud-Dialogflow y Google-Cloud-Speech: librerías utilizadas para la integración con los servicios de inteligencia artificial de Google Cloud.
- Selenium [10]: herramienta utilizada para automatizar la interacción con páginas web externas, permitiendo la simulación de los trámites en las webs oficiales.

4.1.3. Base de datos

Para la gestión de la información del sistema se utiliza SQLite como sistema de base de datos.

Esta elección se debe a su simplicidad de uso y a su integración directa con Python, por lo que no requiere un servidor independiente. Además, resulta suficiente para almacenar la información necesaria en el contexto de un sistema simulado y en fase de desarrollo.

4.1.4. Servicios

Para complementar el sistema, se integran diversos servicios externos que proporcionan funcionalidades clave como la comunicación con el usuario, el procesamiento del lenguaje natural y la gestión del audio. La selección de estas tecnologías se basa en un estudio previo de las alternativas disponibles, considerando las limitaciones del proyecto, especialmente en términos de coste, dificultad de integración y adecuación a los objetivos y requisitos planteados.

Agente conversacional

Para la gestión de la conversación, se analizan diferentes enfoques para la implementación del agente.

Por un lado, existen soluciones basadas en modelos de lenguaje de gran tamaño (LLM), como Vapi [11], que ofrecen interacciones más naturales y flexibles. Sin embargo, el objetivo del proyecto exige conversaciones guiadas orientadas a la recopilación precisa de la información necesaria para cada trámite. Por ello, se descartó esta solución, ya que mantener un flujo estricto y limitado, evitando desviaciones temáticas o respuestas fuera del contexto del trámite, resulta más complejo con este tipo de modelos generativos.

Por este motivo, se opta por el otro enfoque, basado en la definición de intenciones, entidades y flujos de conversación estructurados, utilizando Dialogflow ES [12] de Google Cloud. Esta solución se adapta mejor a los objetivos del proyecto, facilitando el control del diálogo y la recopilación de información con un formato específico, resultando especialmente adecuada para un número limitado de trámites bien definidos. Asimismo, se valoran opciones similares a Dialogflow, como Synthflow [13], pero sus planes gratuitos o sistemas de créditos resultan más restrictivos e insuficientes para la cantidad de pruebas necesarias durante el desarrollo, haciéndolas menos viables para el proyecto.

Canal de comunicación con el usuario

Uno de los aspectos más relevantes del sistema es el medio a través del cual los usuarios interactúan con el agente conversacional para realizar sus trámites. En este sentido, se plantean dos enfoques principales, el uso de la red telefónica tradicional frente a la comunicación mediante llamadas VoIP.

Inicialmente, se considera la posibilidad de utilizar la red telefónica tradicional mediante la integración con Dialogflow Phone Gateway, así como con otras opciones como Twilio [14]. Estas plataformas permiten alquilar una línea telefónica y gestionar la conexión entre la llamada y el agente. Esta alternativa se ajusta mejor al planteamiento original del proyecto, permitiendo realizar llamadas reales a un número de teléfono, lo que ofrece una experiencia más realista e intuitiva para el usuario. No obstante, esta opción se descarta ya que presenta limitaciones importantes en el contexto del proyecto. En particular, la disponibilidad de números telefónicos españoles en este tipo de plataformas es reducida, lo que implica el uso de números internacionales. Esto conlleva costes por minuto de llamada significativamente elevados, lo que incrementa considerablemente el coste del desarrollo.

Como alternativa, se considera el uso de tecnologías VoIP, que permiten realizar llamadas a través de Internet sin necesidad de una línea telefónica tradicional. Dentro de este enfoque, se opta por el uso

de Discord como plataforma de comunicación. Esta solución permite simular llamadas de voz de manera gratuita, manteniendo una experiencia de usuario similar a la de una llamada telefónica, pero sin los costes asociados. Además, Discord facilita la creación de aplicaciones y bots programables en Python, lo que simplifica su integración con el resto de servicios del sistema y con los servicios de inteligencia artificial utilizados. De este modo, se consigue una solución funcional, económica y adecuada a los requisitos del proyecto.

Procesamiento de voz

El sistema requiere la conversión del audio extraído de las llamadas VoIP en texto para que pueda ser procesado por el agente conversacional. Para ello, se utiliza el servicio de Google Speech-to-Text, que permite convertir el audio en texto de manera precisa e inmediata. Además, esta solución está integrada con el ecosistema de Google Cloud, facilitando su uso y compartiendo el sistema de créditos con Dialogflow.

Adicionalmente, se contempla la implementación de un servicio de reconocimiento de voz local utilizando Vosk, que es menos preciso que el servicio de Google, pero permite realizar consultas durante el desarrollo sin incurrir en el consumo de créditos.

Servicios y herramientas utilizadas

En base a las decisiones anteriores, se seleccionan los siguiente servicios externos para la implementación del sistema:

- Google Cloud: plataforma en la nube que centraliza los servicios de inteligencia artificial que se utilizan en el proyecto, incluyendo Dialogflow ES y Google Speech-to-Text.
- Dialogflow ES: es la herramienta de Google Cloud para crear agentes conversacionales capaces de interpretar el lenguaje natural y las intenciones de los usuarios mediante algoritmos de machine learning. Se utiliza para definir las intenciones y los flujos de conversación que podrán tener los usuarios con el agente, recopilar la información requerida para los trámites y devolver el audio a reproducir en la llamada.
- Google Speech-to-Text: servicio de reconocimiento de voz de Google Cloud que permite convertir el audio de las llamadas VoIP en texto para que pueda ser procesado por Dialogflow.
- Discord: plataforma de comunicación que se emplea para simular llamadas VoIP entre el usuario y el agente. El usuario inicia una llamada uniéndose a un canal de Discord y el bot comenzará una conversación recopilando el audio de la llamada y reproduciendo la respuesta del agente.
- Ngrok [15]: herramienta que permite exponer el servidor local a Internet para tener un acceso público en desarrollo. Esta acción es necesaria para poder comunicar el backend de la aplicación web con el agente conversacional de Dialogflow y así permitir que el agente pueda realizar acciones en el sistema, como crear las peticiones. Es una herramienta segura que evita los costes de mantener un servidor propio para la implementación del proyecto, manteniéndonos dentro del alcance definido.

4.1.5. Herramientas de desarrollo y documentación

Para el desarrollo del proyecto y la elaboración de la memoria se utilizan diversas herramientas que facilitan la implementación y organización del trabajo:

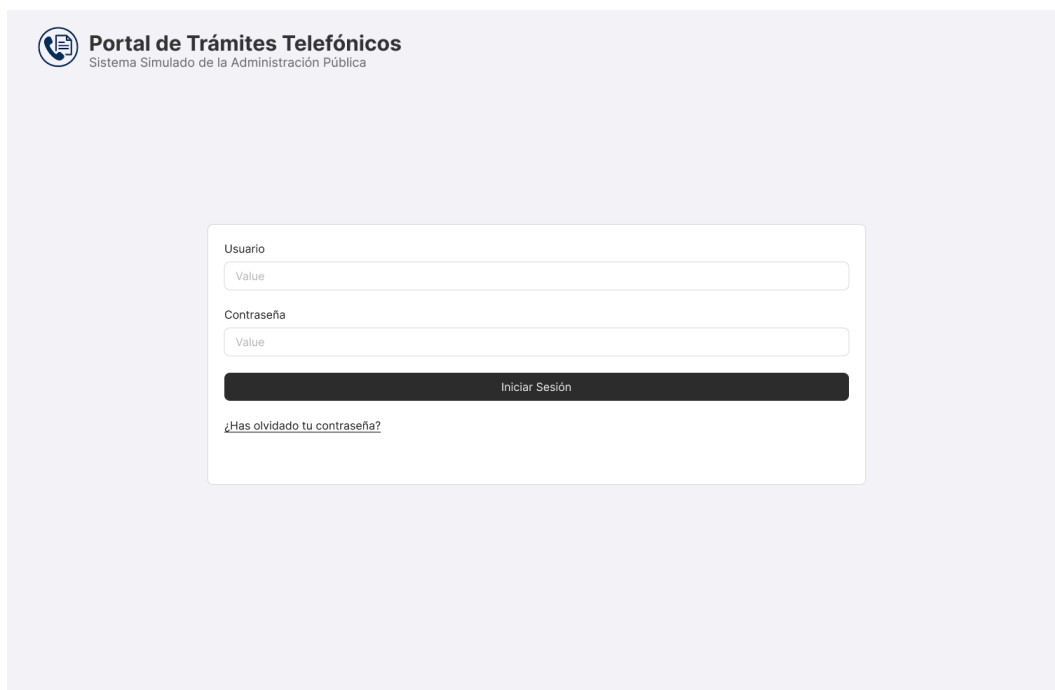
- Visual Studio Code: es un editor de código que se emplea tanto para el desarrollo del sistema como para la redacción de la memoria en LaTeX. Se selecciona por su integración con múltiples lenguajes de programación y su capacidad para manejar y organizar proyectos complejos.

- LaTeX: es un sistema de composición de textos que se utiliza para la redacción de la memoria, permitiendo generar documentos estructurados.
- GitHub: es una plataforma de control de versiones que se emplea para gestionar los cambios en el código y la documentación del proyecto, facilitando el seguimiento de cambios y la recuperación de versiones anteriores.
- Draw.io: es una herramienta de diseño que se utiliza para la creación de diagramas y esquemas explicativos del proyecto.
- Figma: es una herramienta de diseño que se emplea para la creación de prototipos visuales y mockups interactivos de la interfaz de usuario.

4.2. Diseño de la interfaz de usuario

En esta sección se muestra el diseño adoptado en la interfaz de la aplicación web destinada a trabajadores de las Administraciones públicas. El concepto general de la interfaz se ha centrado en mostrar únicamente la información y funcionalidades necesarias en pantalla, manteniendo un estilo simple y claro que favorece la eficiencia y eficacia en el uso de la aplicación.

A continuación se presentan los mockups de las páginas principales del sistema. En cada prototipo se describen los elementos que lo componen, explicando las funcionalidades que disponen los empleados.

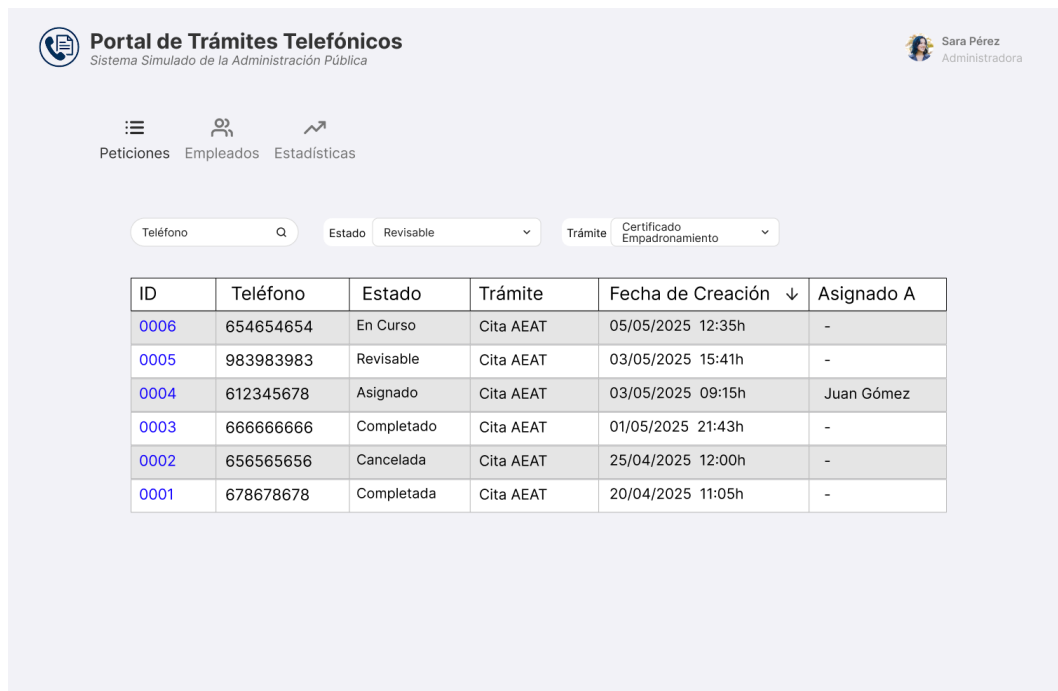


El mockup muestra la interfaz de inicio de sesión de un sistema web. En la parte superior izquierda, hay un icono de un teléfono dentro de un círculo azul, seguido del texto "Portal de Trámites Telefónicos" y "Sistema Simulado de la Administración Pública". En el centro, hay un formulario con dos campos de entrada: "Usuario" y "Contraseña", cada uno con un placeholder "Value". Debajo de los campos, hay un botón negro con el texto "Iniciar Sesión". En la parte inferior del formulario, hay un enlace que dice "¿Has olvidado tu contraseña?".

Figura 4.1: Prototipo visual de la página de login.

La Figura 4.1 muestra la pantalla de inicio de sesión del sistema. Se trata de la página inicial de la aplicación y la única accesible públicamente para usuarios ajenos al sistema. Por ello, presenta únicamente un formulario de acceso en la parte central, que permite a los empleados identificarse fácilmente mediante sus credenciales.

Para acceder a cualquier otra página del sistema es necesario autenticarse para garantizar la confidencialidad de la información.



ID	Teléfono	Estado	Trámite	Fecha de Creación ↓	Asignado A
0006	654654654	En Curso	Cita AEAT	05/05/2025 12:35h	-
0005	983983983	Revisable	Cita AEAT	03/05/2025 15:41h	-
0004	612345678	Asignado	Cita AEAT	03/05/2025 09:15h	Juan Gómez
0003	666666666	Completado	Cita AEAT	01/05/2025 21:43h	-
0002	656565656	Cancelada	Cita AEAT	25/04/2025 12:00h	-
0001	678678678	Completada	Cita AEAT	20/04/2025 11:05h	-

Figura 4.2: Prototipo visual de la página de peticiones.

La Figura 4.2 muestra un listado con todas las peticiones del sistema. Se trata de la página principal de la aplicación, ya que es visible por todos los empleados.

El componente principal es una tabla con la información básica de cada petición, incluyendo en la parte superior de la tabla una serie de filtros para localizar rápidamente la gestión deseada. En cada fila de la tabla se incluye un link, situado en la columna del ID, para navegar a una página con detalles específicos de cada petición.

Además, en la parte superior izquierda, debajo del logotipo, se incluye un menú para navegar entre las distintas páginas de la aplicación. Dependiendo del rol del empleado, el menú mostrará unas opciones u otras, restringiendo el acceso a ciertas páginas. Por ejemplo, los secretarios no tienen permiso para acceder a la sección de empleados, por lo que no se muestra en su menú, mientras que los administradores sí pueden acceder a ella.

Por último, todas las páginas incluyen el perfil del empleado identificado en la parte superior derecha. Clicando sobre el perfil se puede navegar hasta la página con detalles del empleado.

Portal de Trámites Telefónicos
Sistema Simulado de la Administración Pública

Sara Pérez
Administradora

< Petición 0006, Cita AEAT

Teléfono: 654654654

05/05/2025 12:35h **Creada**

05/05/2025 12:36h **Pendiente**

05/05/2025 12:41h **Asignada**
Juan Fernández

05/05/2025 12:43h **Completada**
Juan Fernández

Información

DNI: 71111111A

☒ Presencial ☐ Telefónica

Nombre: Ana Sanz

Oficina de la Aeat de Rgto. Medina Campo
Cl López Flores, 1, 47400, Medina Del Campo, Valladolid

Servicio: Presentación de documentos en registro

27/11/2025 09:00h

Figura 4.3: Prototipo visual de la página con detalles de una petición.

Portal de Trámites Telefónicos
Sistema Simulado de la Administración Pública

Juan Fernández
Secretario

< Petición 0005, Cita AEAT

Teléfono: 983983983

Asignar

03/05/2025 15:41h **Creada**

03/05/2025 15:41h **Pendiente**

Información

DNI: 69834521J

☐ Presencial ☒ Telefónica

Nombre: Fran García

Servicio: IVA

Cancelar **Guardar**

Figura 4.4: Prototipo visual de la página de una petición revisable.

Al clicar en el ID de una petición en la tabla de la Figura 4.2, se muestra la página con los detalles, Figura 4.3. Esta pantalla expone la información más relevante de cada petición, incluyendo una línea temporal con los estados de la gestión, para que los empleados puedan consultar su evolución, y un formulario con los datos recogidos por el agente conversacional, necesarios para completar el trámite.

En caso de ser una petición revisable, Figura 4.4, todos los secretarios verán en la parte superior derecha un botón, el secretario que lo pulse se asignará dicha petición. Una vez asignada, los campos del

formulario se volverán modificables, permitiendo al secretario editar la información o cancelar la petición.

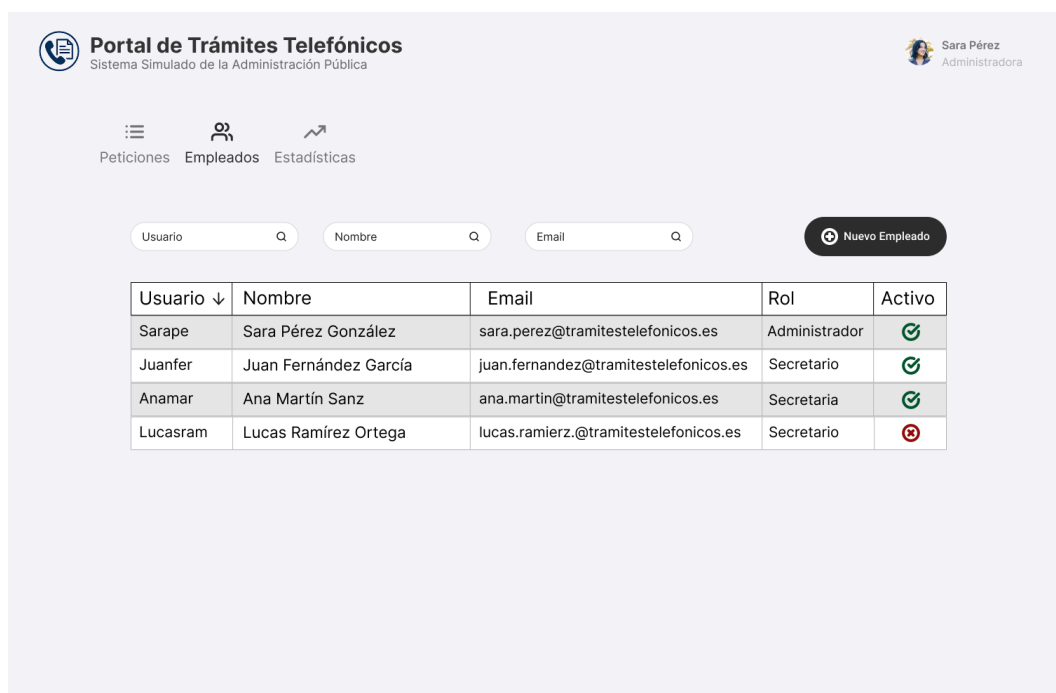


Figura 4.5: Prototipo visual de la página de empleados.

Desde el menú superior, los administradores pueden navegar hasta la sección de empleados, Figura 4.5. El componente principal es una tabla similar a la de las peticiones, que incluye la información de todos los empleados registrados en el sistema y los filtros necesarios para localizarlos rápidamente.

A diferencia de la otra tabla, se añade una columna que indica si la cuenta de cada trabajador está activa. Esta columna cumple una doble función, ya que, clicando sobre el valor correspondiente a un empleado, se navegará hasta una página donde se muestra su información, permitiendo modificarla.

La posibilidad de desactivar cuentas impide el acceso al sistema de un empleado sin necesidad de eliminar su cuenta, preservando así la trazabilidad de la información.

Por último, se incluye a la derecha de los filtros un botón para poder agregar nuevos empleados al sistema, cualquier administrador podrá utilizarlo, pudiendo añadir nuevas cuentas de administradores y secretarios.

La página que muestra la información de los trámites será exactamente igual a esta, manteniendo los mismos componentes, disposición y funcionalidad.



Figura 4.6: Prototipo visual de la página de estadísticas.

Desde el menú de navegación, también se puede acceder a una sección de estadísticas, Figura 4.6. Esta página está diseñada para mostrar información relevante sobre las peticiones y las llamadas recibidas. Dado que se trata de un sistema simulado, no habrá datos suficientes para que la información sea representativa. Sin embargo, la sección queda incluida en el diseño como un desarrollo previsto a futuro.

4.3. Arquitectura del sistema

La arquitectura principal del sistema se organiza como un modelo de capas relajadas, separando la funcionalidad en cuatro capas independientes. Esta arquitectura implica que cada capa no se comunica únicamente con el nivel inmediatamente inferior, sino que habrá una capa superior que se comunique con las demás. Esto se debe a que se ha adaptado la arquitectura al framework de Flask; por ello, el sistema no sigue un patrón modelo-vista-controlador estricto.

La capa principal es la capa de presentación, que incluye las vistas de la aplicación web y los controladores. Flask funciona con rutas que se definen con una función, por lo que cada controlador puede incluir múltiples rutas. Es decir, se usan controladores ligeros ya que cada controlador puede gestionar múltiples vistas. Y al ser un sistema moderno, desde los mismos controladores se comunican con las otras tres capas, lo que implica que no puedan ser estrictas.

La capa de negocio incluye únicamente un archivo desde el que se controlan los permisos de los empleados. El resto de la lógica de negocio está definida implícitamente en las rutas.

En la capa de persistencia, se incluyen únicamente los modelos ORM, ya que sirven para representar la estructura de las entidades de la base de datos y encapsular su acceso. Con esta herramienta, no es necesario crear un singleton propio para la conexión con la base de datos, ya que esa funcionalidad ya viene incluida.

Por último, hay una capa de servicios que incluye los archivos del sistema que se utilizan para comunicarse con los servicios externos a la aplicación. Principalmente, se usan las prestaciones de Google

Cloud y Discord, aunque también incluye el servicio de correo interno y los procesos de automatización para ejecutar los trámites en las webs oficiales.

La comunicación y dependencias entre las distintas capas del sistema y sus componentes se representan en la Figura 4.7.

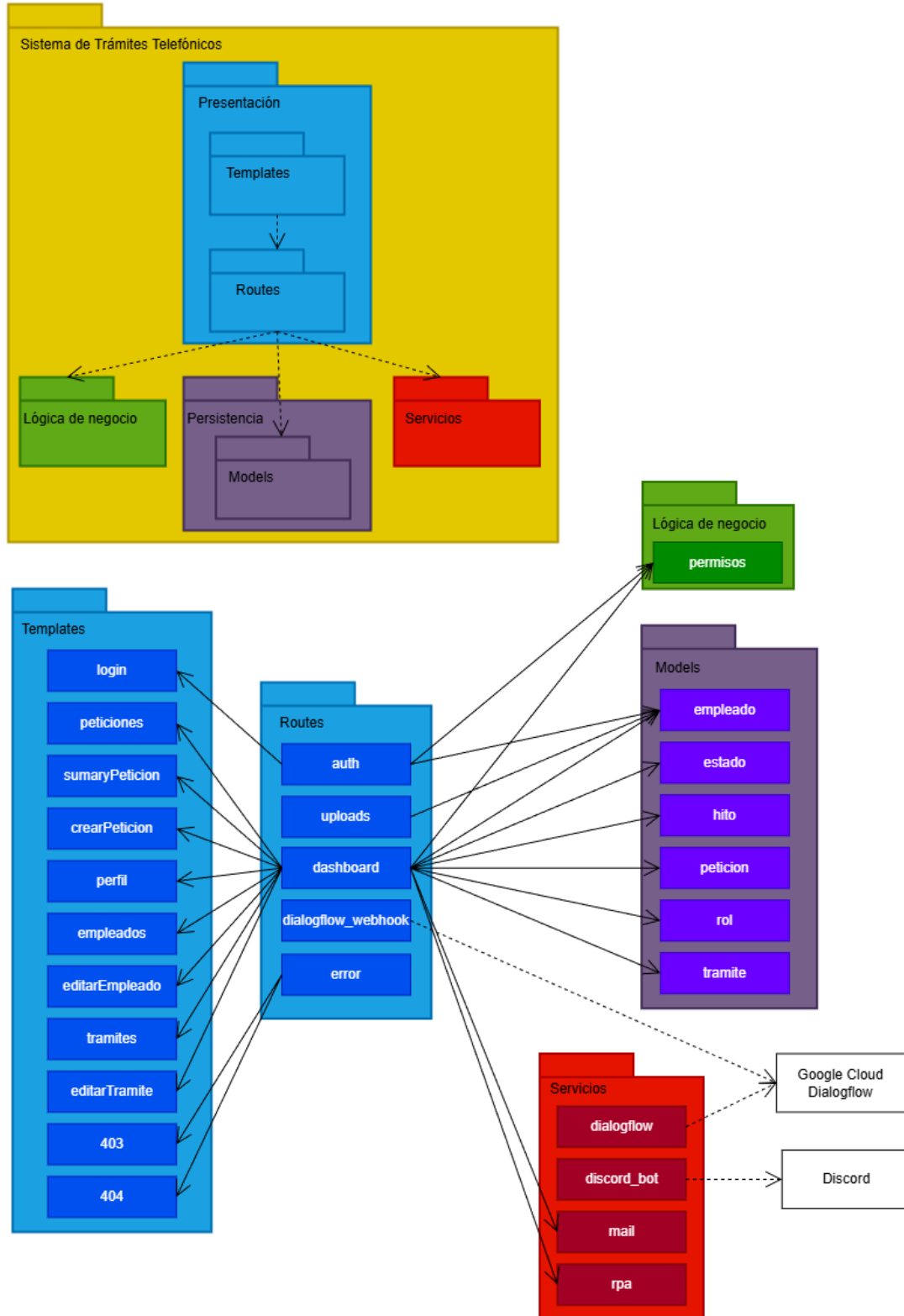


Figura 4.7: Diagrama de descomposición y uso.

4.4. Diseño de la base de datos

En esta sección se presenta el diseño de la base de datos del sistema, estructurada para almacenar toda la información necesaria para el correcto funcionamiento de la aplicación. En la Figura 4.8 se representa el diagrama entidad relación, siguiendo una representación fiel al modelo de dominio.

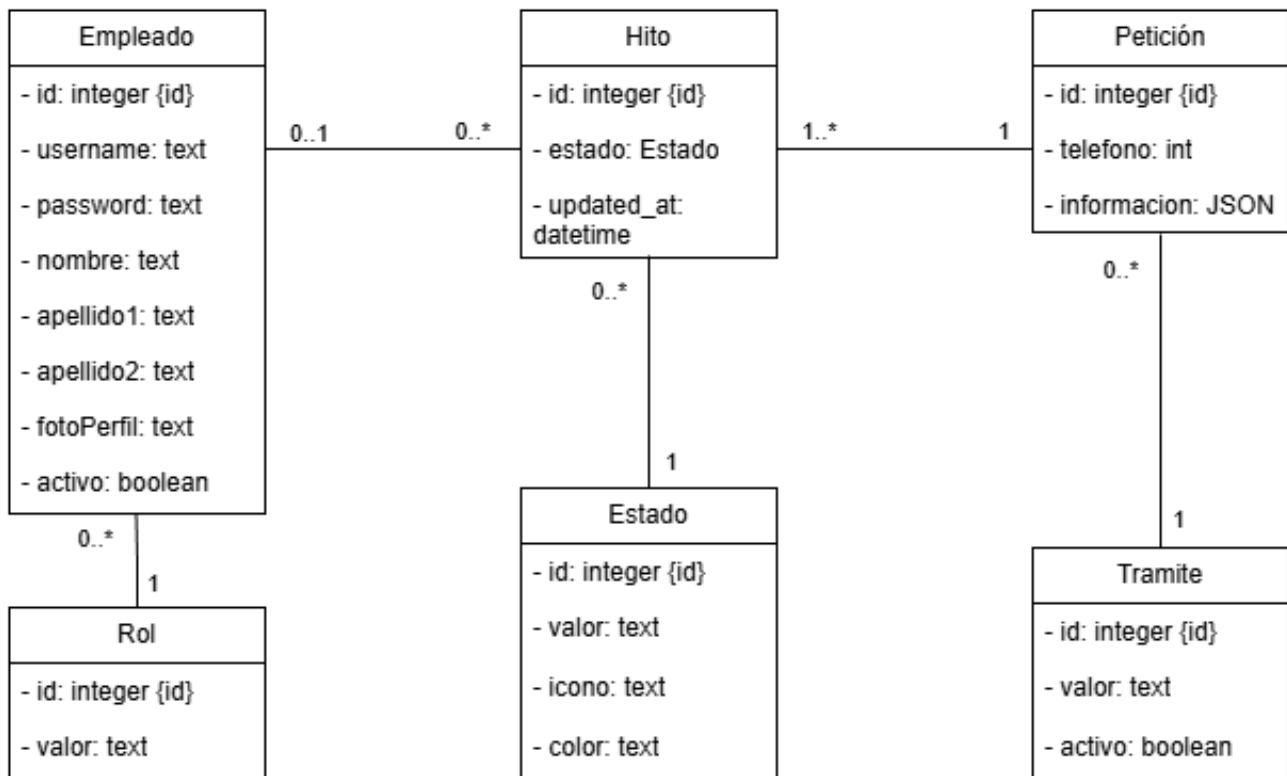


Figura 4.8: Diagrama Entidad-Relación.

4.5. Diseño de la inteligencia artificial

El agente conversacional de Dialogflow funciona mediante la definición de intents y entities, implementando tecnologías de machine learning para detectar y analizar la intención que tiene el usuario.

Las posibles intenciones de los usuarios se definen en los intents. Se describen las frases de entrada o eventos que activan el intent, las contestaciones que debe dar el bot y las entities necesarias que se deben obtener. Cuando el usuario dice una frase de entrada, la inteligencia artificial calcula la probabilidad de que coincida con un intent, y si el porcentaje supera el umbral, activa la intención.

Las entities sirven para definir los tipos de datos o la forma en la que se extrae la información de las frases. En otras palabras, sirven para categorizar o etiquetar las palabras. Por ejemplo, en la siguiente solicitud: “Quiero agendar una cita en la AEAT para mañana a las 10”, podríamos extraer agendar una cita en la AEAT como una entidad de tipo trámite, mañana como fecha y las 10 como hora. De esta forma, podemos determinar qué tipos de datos necesitamos al recopilar información, si un número, un texto, una dirección, etc. Además, las entidades pueden definirse como listas de valores con sinónimos asociados, para reconocer diferentes formas de referirse a un mismo concepto. Asimismo, es posible utilizar expresiones regulares para identificar patrones concretos, así como técnicas de coincidencia aproximada basadas en machine learning para identificar valores similares.

Para la resolución de los trámites escogidos se han definido los siguientes intents:

- Bienvenida: se activará automáticamente cuando el usuario inicia la llamada y activará un evento de espera de trámites.
- Trámite certificado empadronamiento, trámite cita AEAT y trámite tarjeta SACYL: se activarán cuando el usuario los solicite y el evento de espera de trámite esté activo, y activarán un intent para solicitar los datos necesarios para cada trámite.
- Obtener datos: cada uno de los intents de detección de trámites activarán un intent específico para solicitar los datos necesarios para cada gestión. Estos intents se encargan de recopilar los datos que faltan y, en caso de que haya algún dato erróneo, permite su modificación.
- Cita AEAT Modalidad: una vez recuperada la información inicial del trámite cita AEAT, se activa este intent desde el backend solo si el servicio escogido ofrece también cita telefónica.
- Cita AEAT Presencial: si el usuario solicita una cita presencial, el bot recupera la información correspondiente sobre la oficina y la fecha.
- Confirmar datos: una vez recuperada la información necesaria, se activará este intent para solicitar al usuario la veracidad de los datos, finalizando la llamada generando una solicitud.
- Cancelar: se activará si el usuario solicita cancelar la gestión, finalizando la llamada sin generar una solicitud.

También será necesario estipular los siguientes entities para facilitar la captura de la información:

- Centros de salud de Valladolid: una lista con todos los centros de salud de Valladolid.
- DNI o pasaporte: una expresión regular para restringir la entrada al formato del DNI o pasaporte.
- Motivo tarjeta SACYL: una variable para identificar el motivo por el que se solicita la tarjeta sanitaria, con opciones de pérdida o deterioro.
- Municipios Valladolid: una lista con todos los municipios de Valladolid.
- Oficina AEAT Valladolid: una lista con las oficinas disponibles de la agencia tributaria en Valladolid.
- Servicios AEAT: una lista de los servicios de la agencia tributaria para agendar citas.
- Teléfono: una expresión regular con el formato de los números telefónicos españoles.
- Tipo de cita: una variable para identificar si la cita en la AEAT es telefónica o presencial.
- Trámite: una lista con los trámites disponibles.

Por último, se muestra en la Figura 4.9 un diagrama de flujo que define la secuencia prevista de una llamada al sistema.

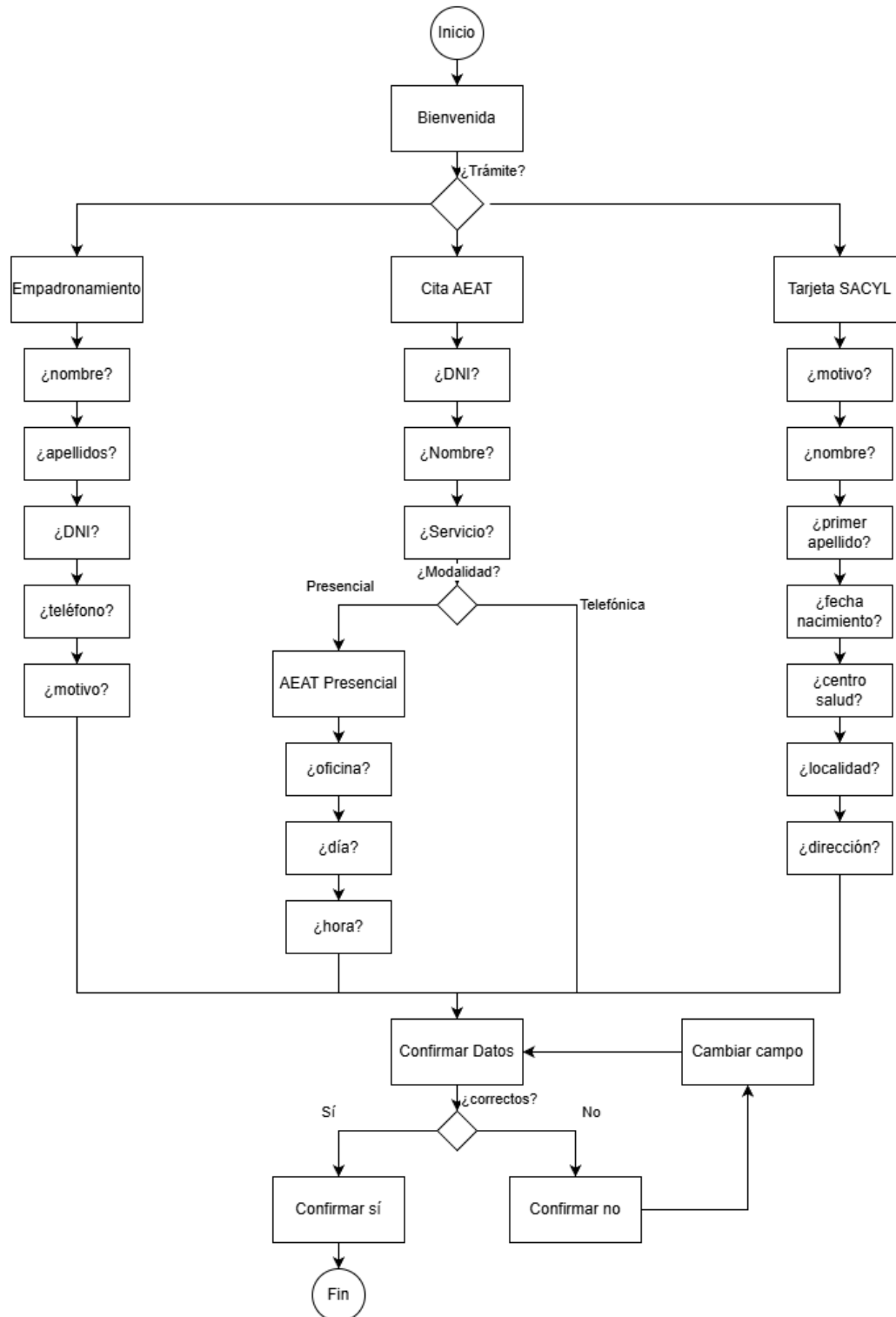


Figura 4.9: Diagrama de flujo del agente conversacional.

4.6. Diagramas de secuencia

A continuación se van a mostrar los diagramas de secuencia de los casos de uso más relevantes del sistema.

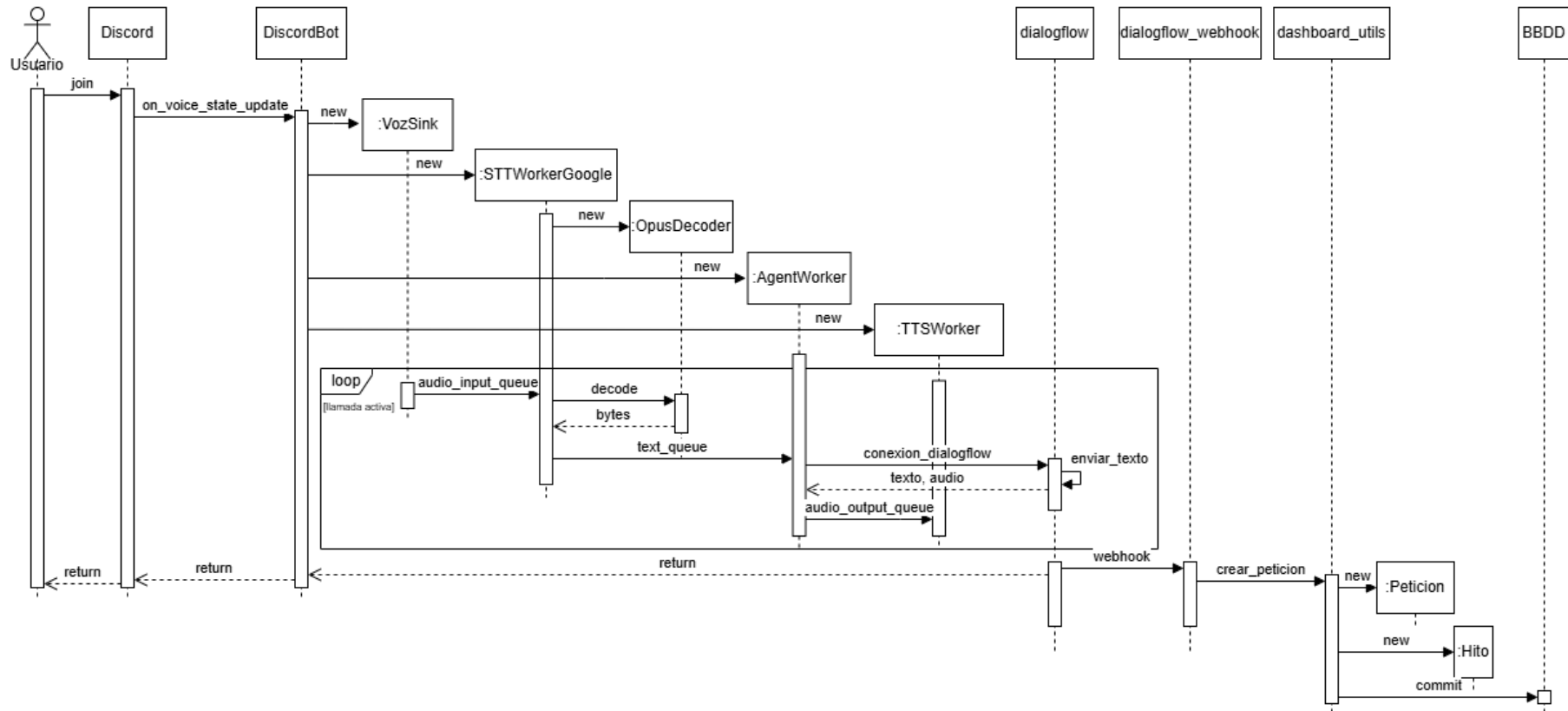


Figura 4.10: Diagrama de secuencia de una llamada telefónica al sistema.

La Figura 4.10 muestra el diagrama de secuencia base de una llamada VoIP entre un usuario y el sistema para realizar cualquier trámite. El flujo inicia cuando el usuario entra en el canal de voz de Discord, lo que activa el bot y da comienzo a la conversación. Cada conversación se gestiona con cuatro hilos independientes:

1. VozSink: se encarga de capturar el audio de entrada del usuario en el canal de voz y añadirlo a una cola de entrada de audio.
2. STTWorkerGoogle: este hilo procesa la cola de entrada de audio, enviando los fragmentos al servicio de Google Speech-to-Text para convertirlos en texto y añadirlos a una cola de entrada de texto.
3. AgentWorker: procesa la cola de entrada de texto, enviando cada mensaje al agente de Dialogflow para obtener directamente el audio de respuesta, que agregará a una cola de salida de audio.
4. TTSTWorker: gestiona la cola de salida de audio, reproduciendo cada fragmento en el canal de voz para que el usuario lo escuche.

Este proceso se mantiene en bucle mientras la llamada permanezca activa, aunque, para cada trámite concreto, la conversación será diferente y estará guiada por el agente siguiendo el diagrama definido en la figura 4.9. Cuando Dialogflow procesa el mensaje final, se finaliza la llamada y se envía la información recopilada al backend para generar la petición y almacenarla en la base de datos.

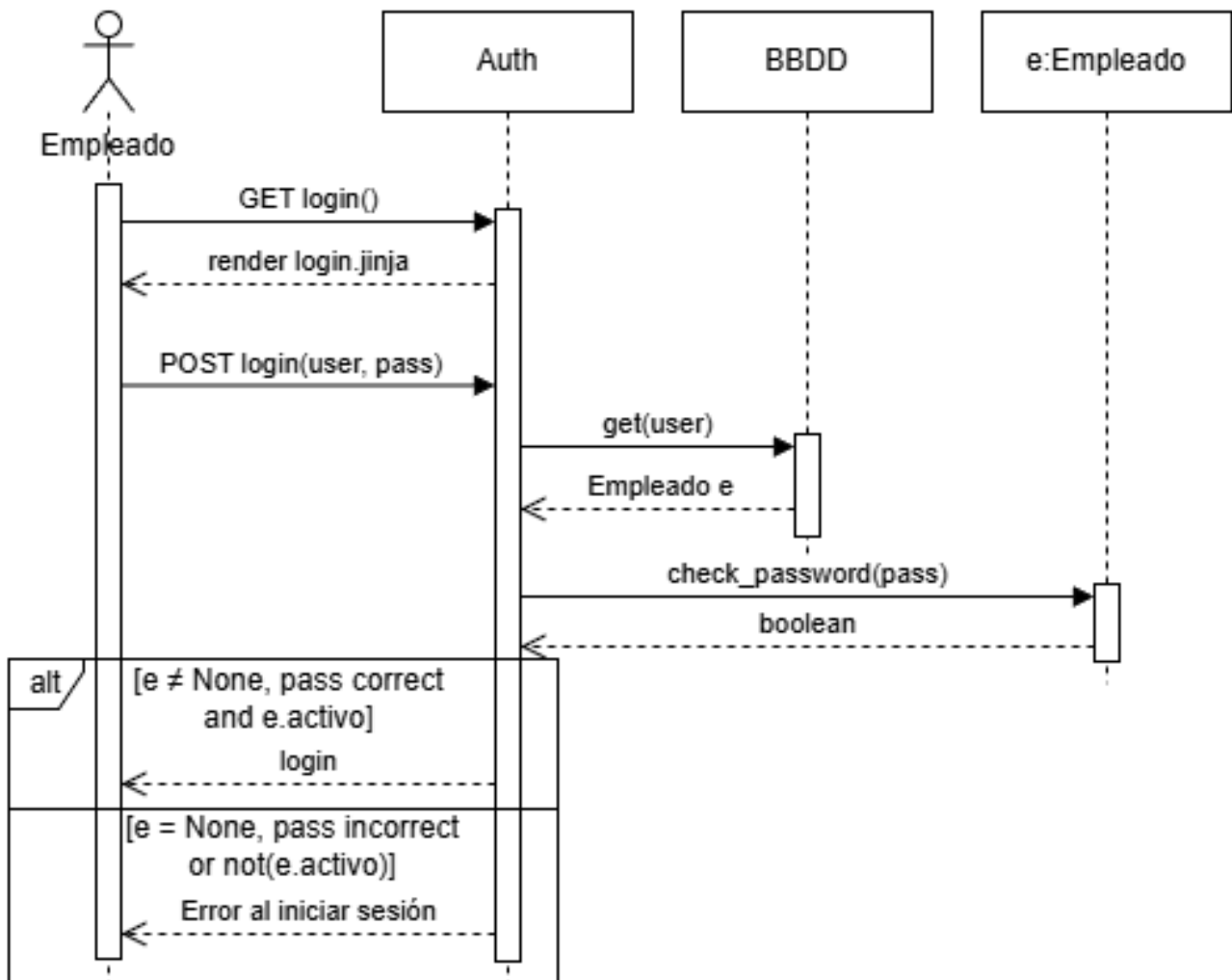


Figura 4.11: Diagrama de secuencia de inicio de sesión de un empleado.

En la Figura 4.11 se muestra el flujo de inicio de sesión en la aplicación web de empleados. El empleado introduce sus credenciales en el formulario de login, que se enviarán al backend para su verificación.

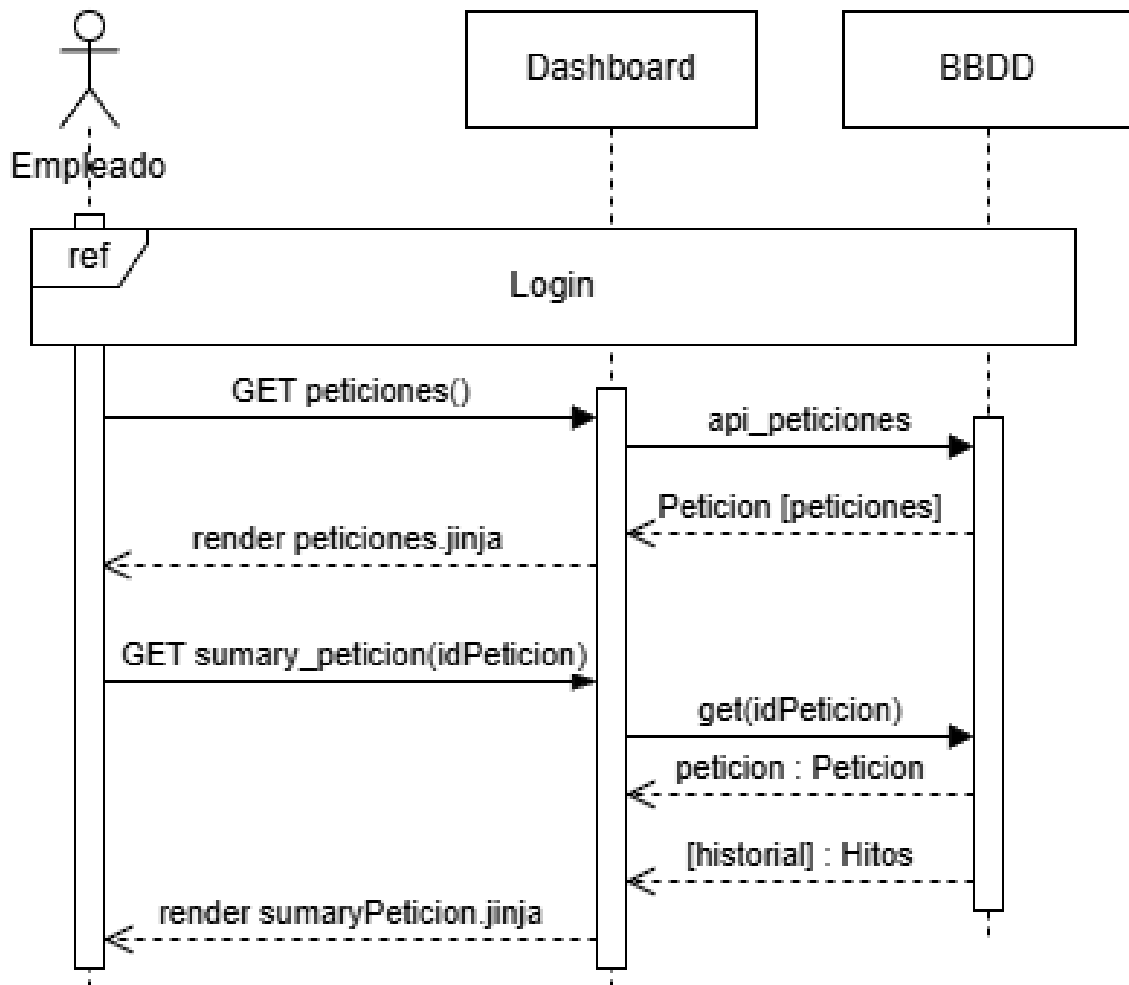


Figura 4.12: Diagrama de secuencia de consulta de los detalles de una petición.

La Figura 4.12 representa el diagrama correspondiente a la consulta de los detalles específicos de una petición. Una vez en la vista con el listado de peticiones, el empleado busca la gestión deseada y clicca en el ID para navegar a la página con los detalles. El sistema recuperará la información de la base de datos y la mostrará en pantalla.

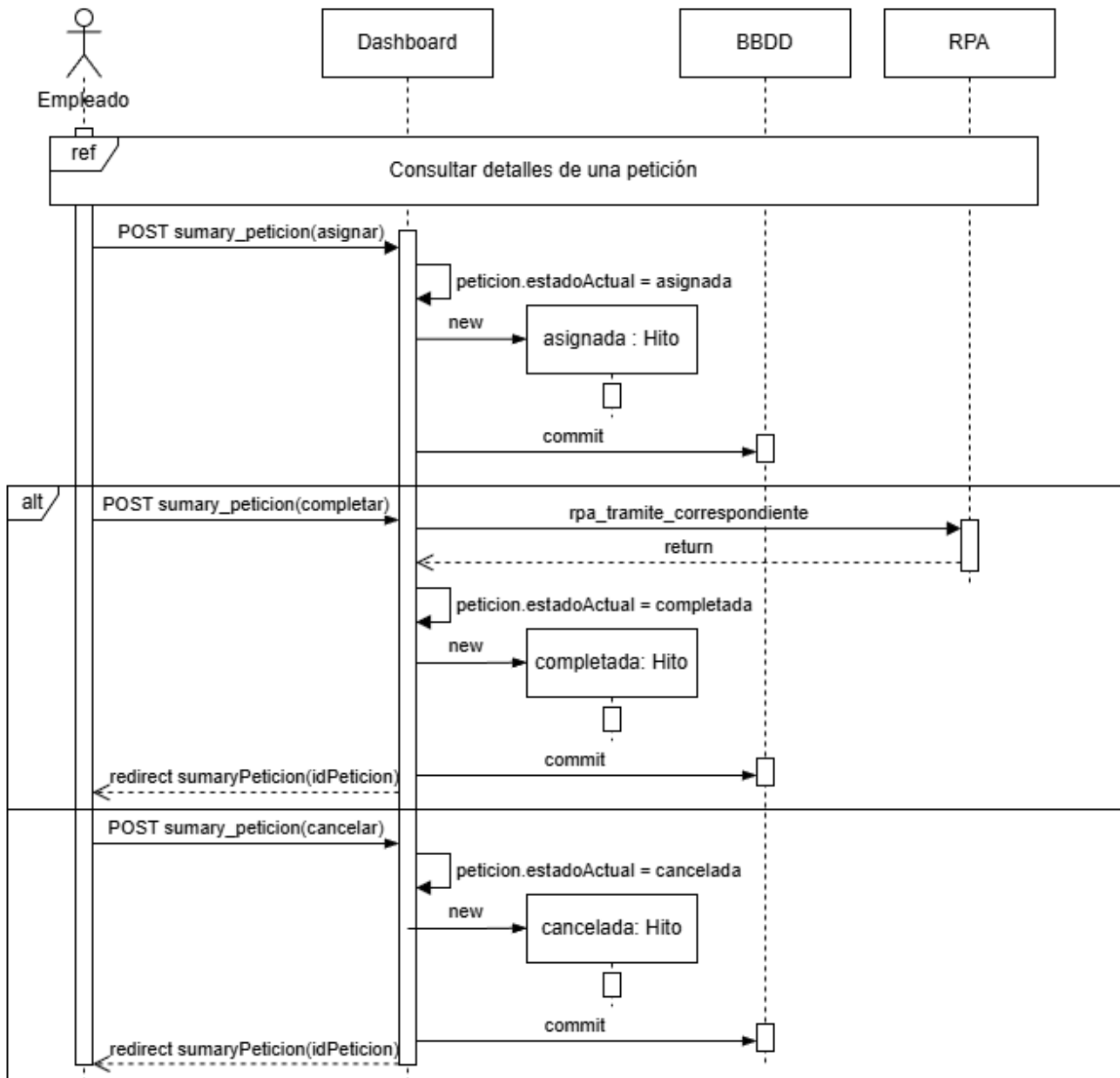


Figura 4.13: Diagrama de secuencia de asignación de una petición revisable.

La Figura 4.13 corresponde al diagrama de secuencia para la asignación de una petición por un empleado. Desde la vista con los detalles de una petición revisable, el empleado puede clicar en un botón para asignársela. El sistema actualizará el estado de la petición en la base de datos y permitirá al empleado editar los campos y completar o cancelar la gestión. Si el empleado decide completar el trámite, se accederá al servicio de automatización para ejecutarlo en la web oficial correspondiente, para posteriormente actualizar su estado en la base de datos. Si la cancela, también se actualizará el estado.

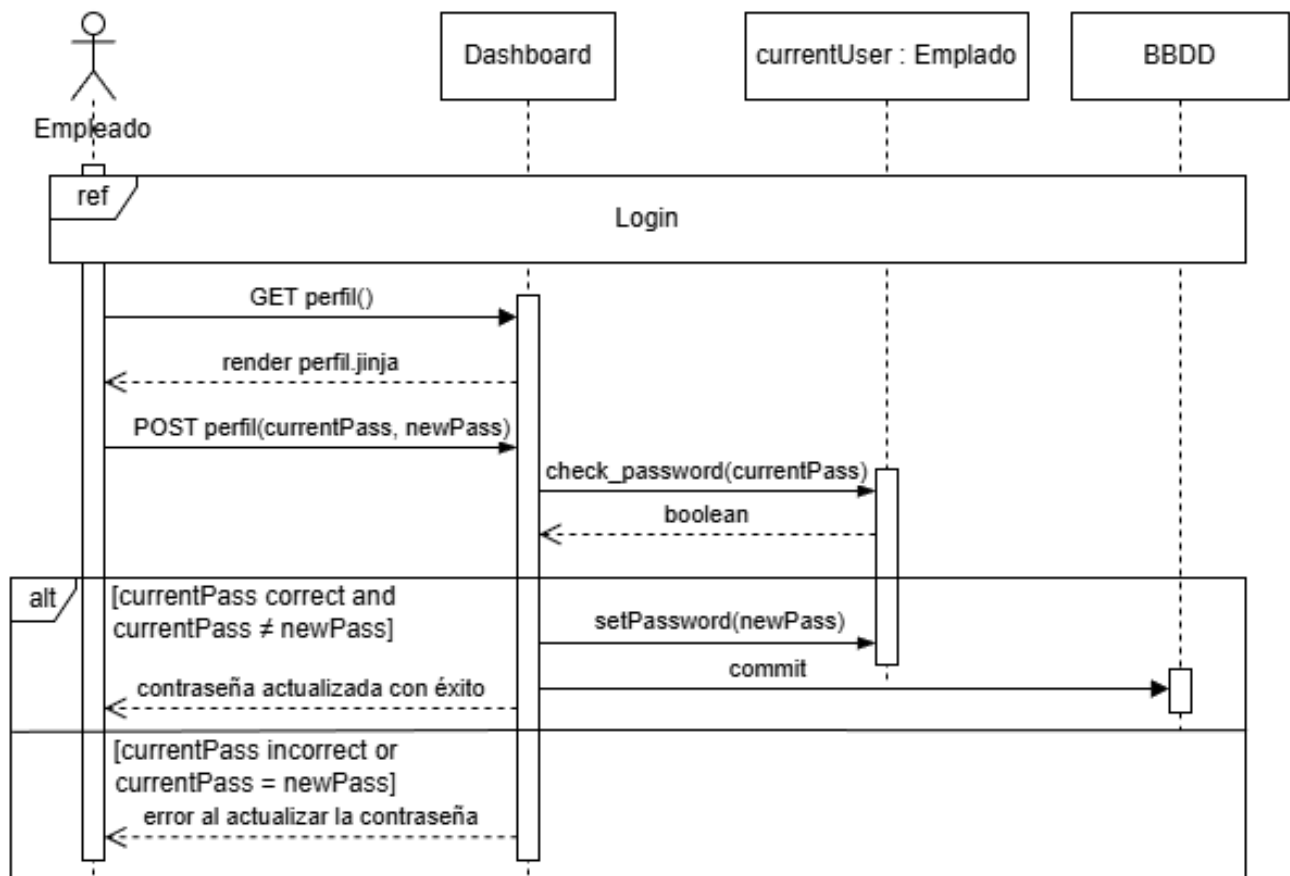


Figura 4.14: Diagrama de secuencia de cambio de contraseña de un empleado.

En la Figura 4.14 se muestra el diagrama de secuencia correspondiente al cambio de contraseña de un empleado. Desde la vista del perfil se rellena el formulario de cambio de contraseña y se envía al backend. Si la contraseña actual es correcta, se actualizará la nueva en la base de datos.

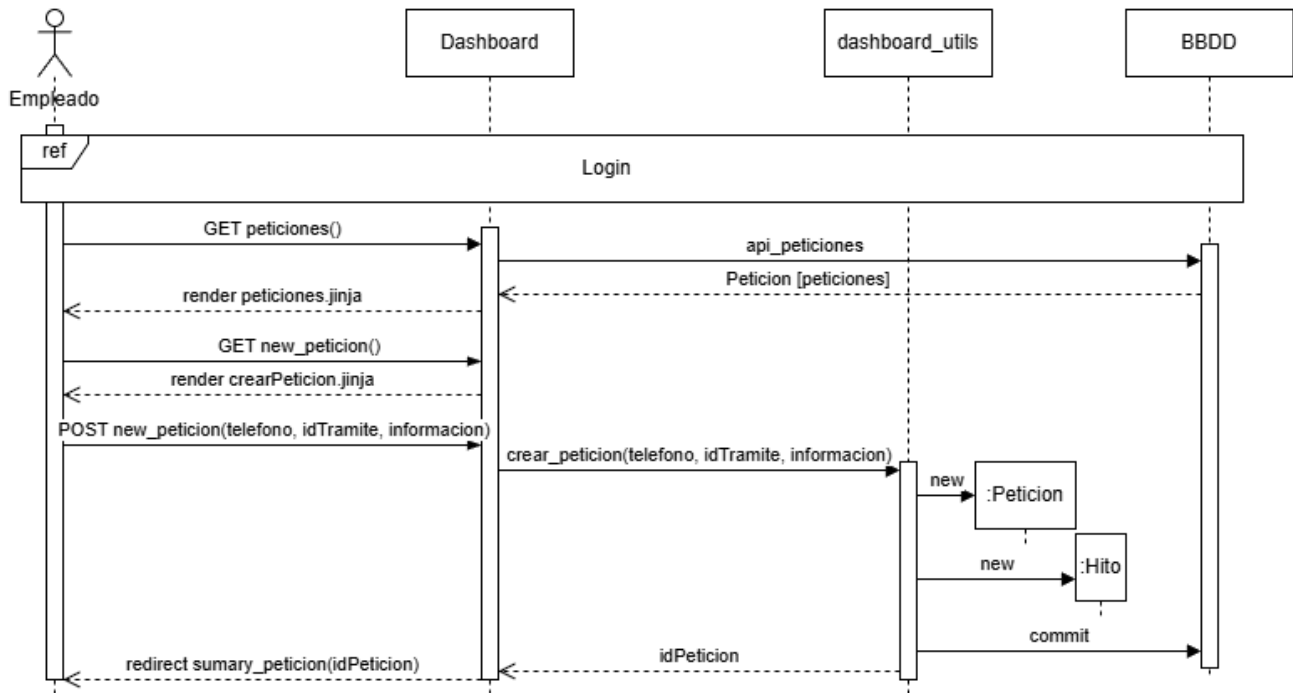


Figura 4.15: Diagrama de secuencia de creación de una petición por un empleado.

La Figura 4.15 representa el flujo de creación de una nueva petición por un empleado. Desde la vista con el listado de peticiones, el empleado clic en el botón para agregar una gestión nueva. Se mostrará un formulario que se rellenará con los datos necesarios, y al enviarlo, el sistema generará la petición y la almacenará en la base de datos.

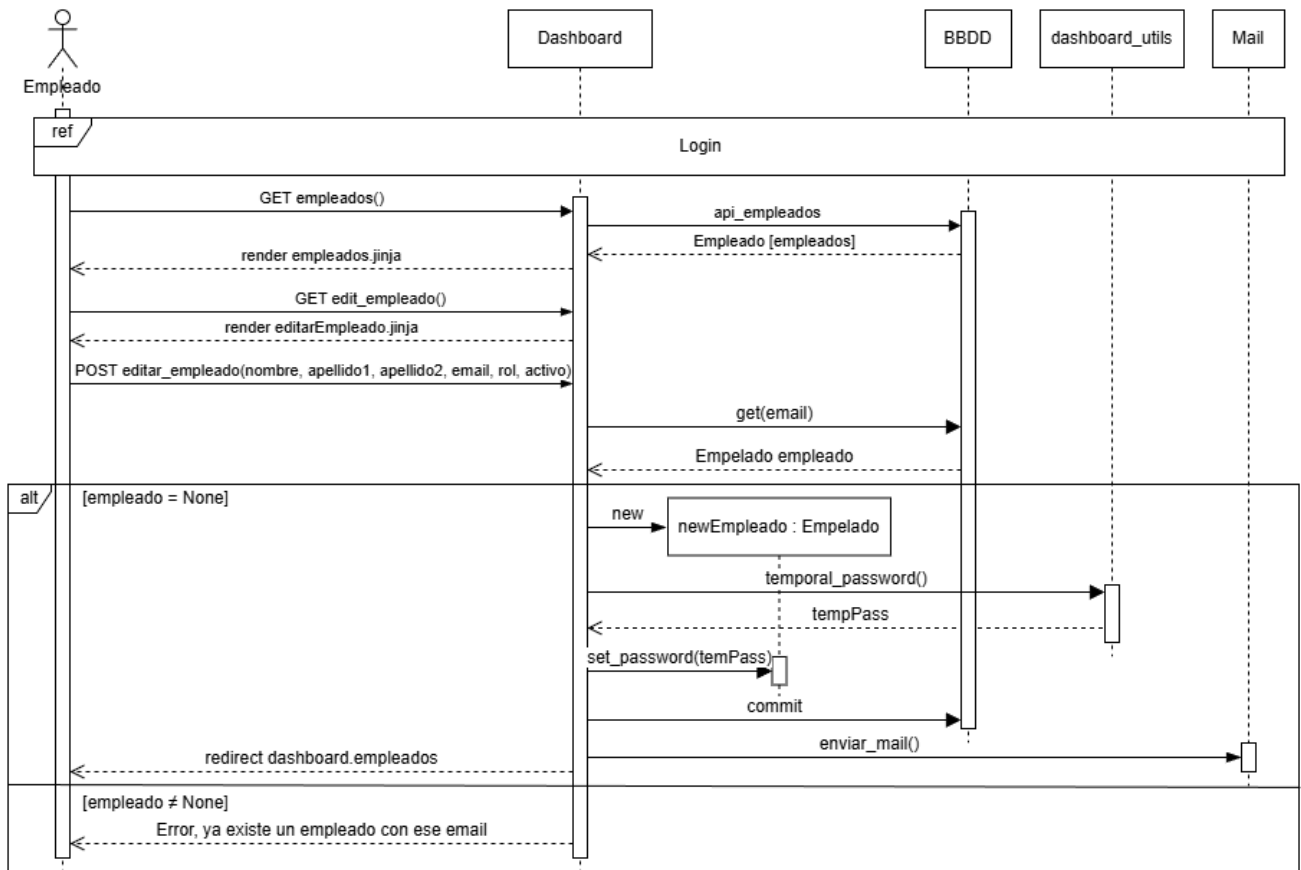


Figura 4.16: Diagrama de secuencia de creación de un empleado.

La Figura 4.16 se corresponde con el diagrama de secuencia para el caso de uso de añadir un nuevo empleado al sistema. El flujo es similar al de la creación de una petición, pero se inicia desde la vista con el listado de empleados. Cuando se envía el formulario, el sistema comprueba que no existe otro empleado con el mismo email, crea la cuenta y envía un correo electrónico al nuevo empleado con su contraseña temporal.

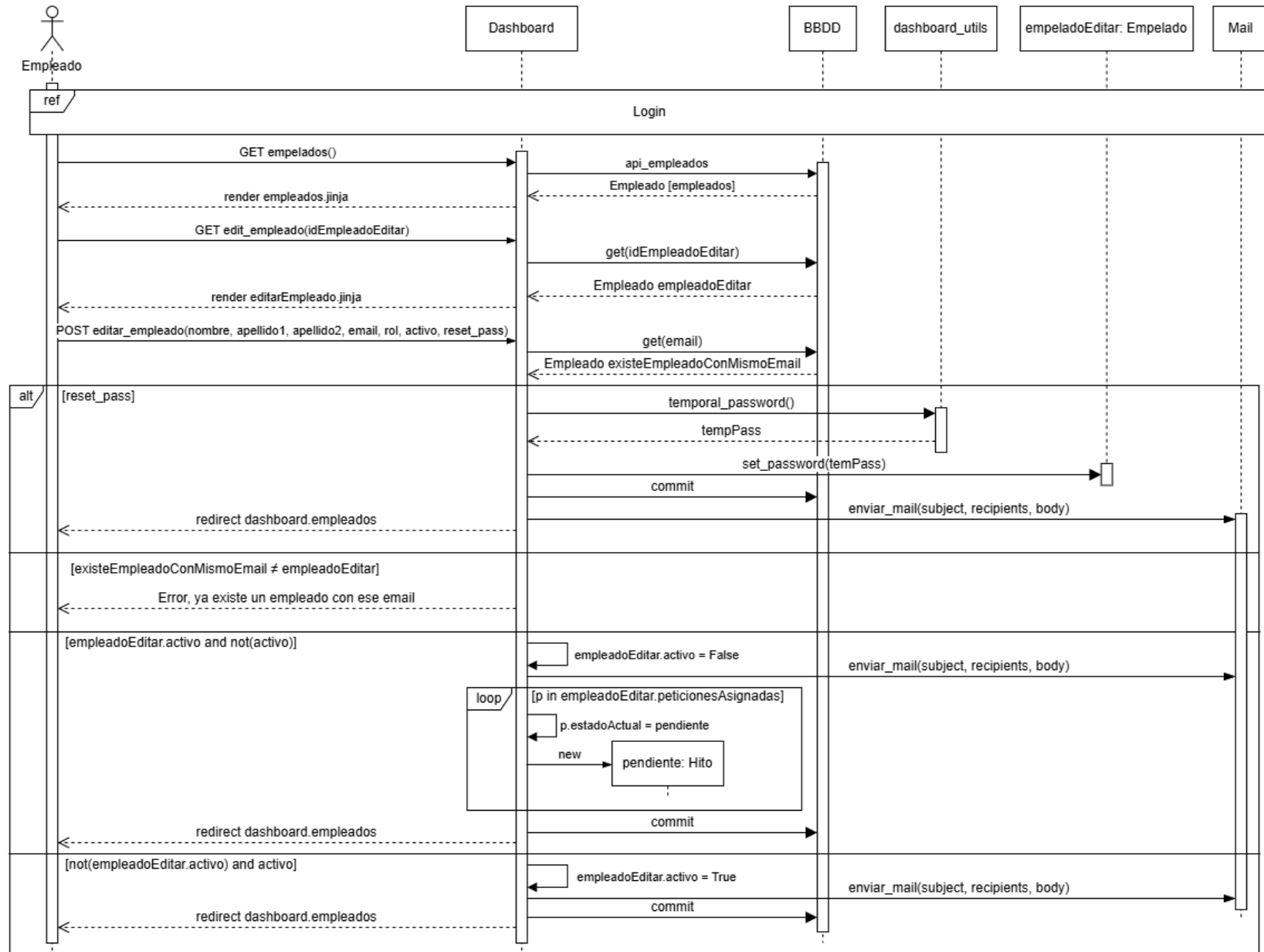


Figura 4.17: Diagrama de secuencia de edición de un empleado.

La Figura 4.17 muestra el diagrama de secuencia para la modificación de la información de un empleado. Desde el mismo formulario de creación, se mostrará la información del empleado seleccionado, permitiendo editar los campos, resetear la contraseña o desactivar la cuenta. Si se modifica el email, el sistema comprobará que no existe otro empleado con el mismo. En caso de reseteo de contraseña o cambio en la activación de la cuenta, se notificará al empleado mediante un correo electrónico. Además, si se desactiva la cuenta de un empleado, el sistema desasignará todas las peticiones que tuviera asignadas para que pueda completarlas otro trabajador.

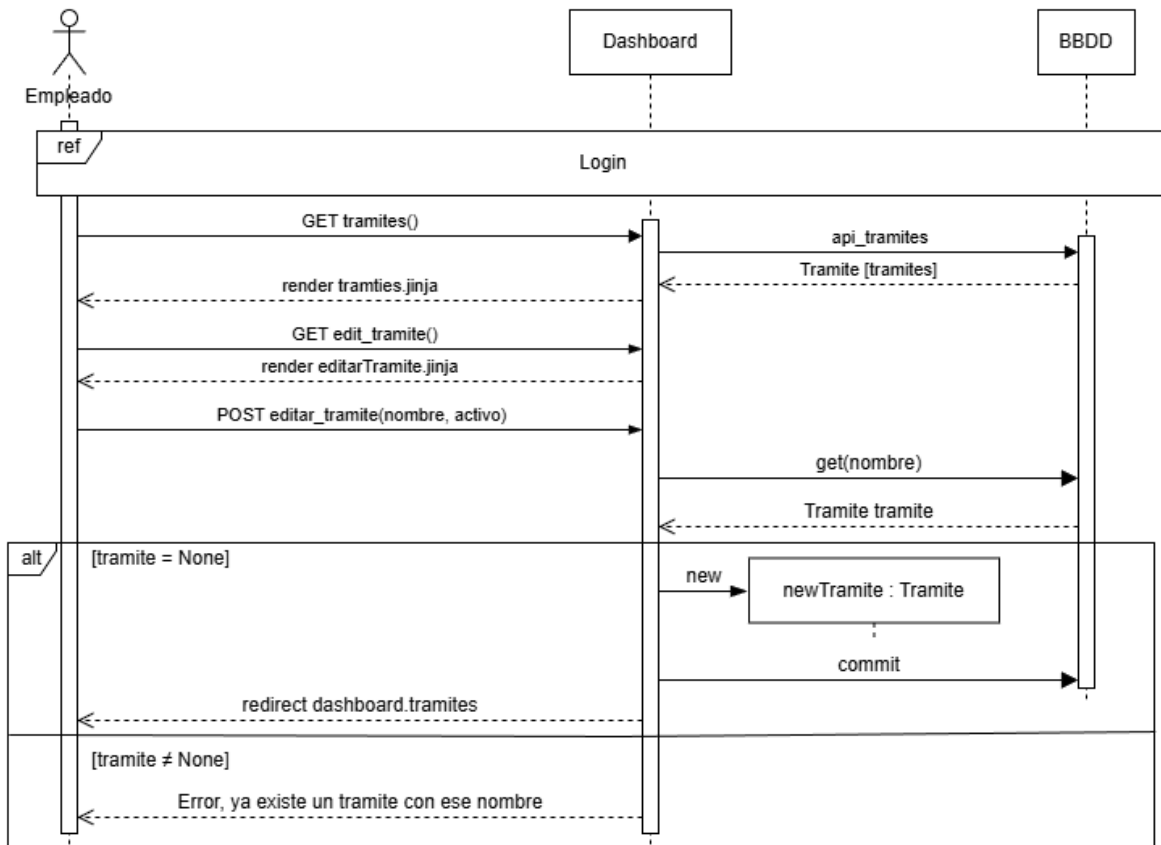


Figura 4.18: Diagrama de secuencia de creación de un trámite.

En la Figura 4.18 se muestra el flujo de creación de un nuevo trámite. Esta secuencia es exactamente igual a la creación de un nuevo empleado, empezando desde la vista con el listado de trámites. En este caso, el sistema comprueba que no existe otro trámite con el mismo nombre antes de crearlo.

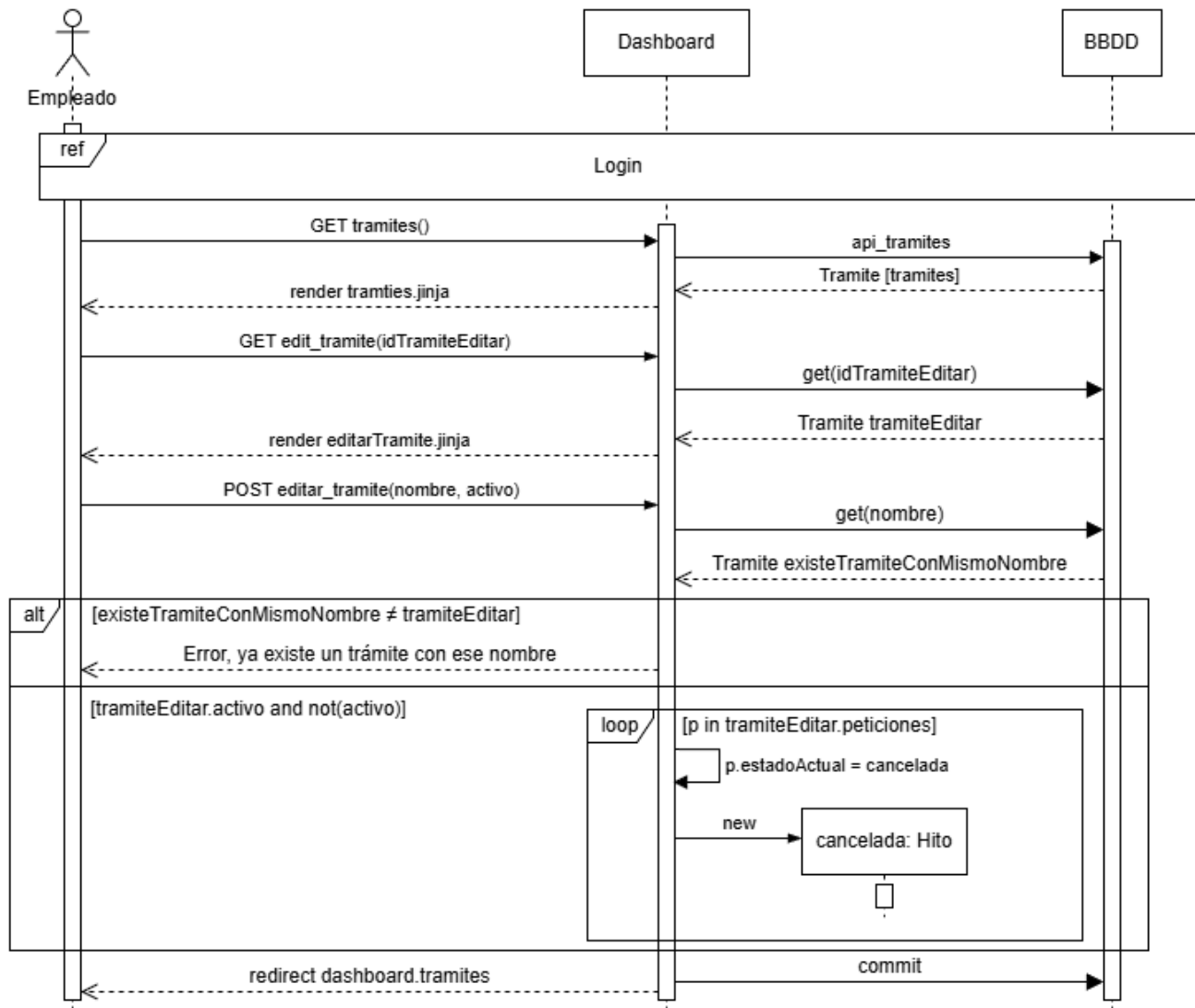


Figura 4.19: Diagrama de secuencia de edición de un trámite.

Por último, se muestra la Figura 4.19, que representa el diagrama de secuencia para la edición de un trámite. Similar a la edición de un empleado, se permite modificar el nombre y la activación de un trámite desde el mismo formulario de creación. Si se cambia el nombre, el sistema comprueba que no existe antes de modificarlo, y si se desactiva, cancela todas las peticiones en curso de ese trámite.

4.7. Seguridad

El diseño de la seguridad constituye un aspecto fundamental del sistema, dado que se maneja información sensible de los usuarios necesaria para completar las gestiones administrativas. Por ello, se han establecido una serie de medidas orientadas a garantizar la confidencialidad, integridad, disponibilidad, autenticidad y trazabilidad de la información.

En primer lugar, se protegerá el acceso de la aplicación web de empleados mediante un sistema de autenticación basado en credenciales. Cada empleado dispondrá de una cuenta con un usuario y correo único, junto con una contraseña que será tratada con una función hash para evitar su almacenamiento en texto plano.

También se empleará un sistema de permisos para restringir el acceso a ciertas funcionalidades. Esta funcionalidad permitirá definir diferentes roles de empleados, otorgando permisos específicos dependiendo de las responsabilidades de cada uno. Para el prototipo solamente hace falta definir dos roles:

- Secretario: podrá visualizar todas las peticiones, crear nuevas y asignárselas.
- Administrador: podrá gestionar empleados y trámites aparte de visualizar las peticiones.

Esta separación de roles garantiza la confidencialidad, integridad y autenticidad de la información, y deja un modelo adaptable a futuro para la fácil inclusión de nuevos roles y permisos, pudiendo limitar los accesos a acciones, registros de la base de datos y vistas de la aplicación, dependiendo de la administración correspondiente, la ciudad o el tipo de trámite.

En segundo lugar, se mantendrá una trazabilidad completa sobre las peticiones en el sistema. Cada gestión dispondrá de un historial de eventos que registrará todos los cambios realizados, incluyendo la fecha y el empleado responsable. Además, no se podrán eliminar trámites ni empleados, sino que se podrán desactivar para mantener la información íntegra.

En cuanto a la disponibilidad, la página web se mantiene independiente del sistema de llamadas, de forma que si los servicios fallan, los empleados podrán seguir utilizándola.

Por último, en relación con la información sensible, la inteligencia artificial emplea los registros de las llamadas únicamente para mejorar la interpretación del agente conversacional, sin almacenar datos personales de manera permanente ni utilizarlos para entrenar modelos de uso general. Google asegura que su herramienta no almacena información personal de forma indefinida. No obstante, al tratarse de un prototipo basado en el uso de información ficticia, no se han implementado aspectos más avanzados de seguridad como el uso de protocolos de comunicación seguros o el cifrado de la información almacenada en la base de datos, quedando estos puntos identificados como posibles mejoras para futuras versiones orientadas a un entorno de producción real.

Capítulo 5

Implementación

En este capítulo se resume el proceso de desarrollo del sistema elaborado, detallando los principales componentes que lo conforman y su funcionamiento. La explicación se estructura siguiendo el orden real de la implementación del proyecto, comenzando por el desarrollo del frontend, seguido de la creación de la base de datos y la construcción del backend. Finalmente, se comentan los distintos servicios integrados en la aplicación, como el envío de correos electrónicos, el bot de Discord, la conexión con la IA mediante Dialogflow y la automatización de los trámites con funciones de Robotic Process Automation (RPA).

5.1. Implementación del frontend

Para el desarrollo del frontend de la aplicación se ha utilizado el sistema de plantillas de Jinja. Se ha definido una plantilla base (layout.jinja), mostrada en la Figura 5.1, que funciona como estructura común para todas las vistas de la web.

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8">
    <link rel="icon" type="image/png" href="{{url_for('static', filename='img/logo.png')}}">
    <title>{%block title%}{%endblock%}</title>
    <link rel="stylesheet" href="{{url_for('static', filename='css/layout.css')}}">
    {%block css%}{%endblock%}
    <script src="https://code.iconify.design/iconify-icon/3.0.1/iconify-icon.min.js"></script>
  </head>
  <body>
    {%include 'components/header.jinja'%}

    {%block body%}{%endblock%}

    {%include 'components/footer.jinja'%}

    {%block js%}{%endblock%}
  </body>
</html>
```

Figura 5.1: Código de la plantilla base layout.jinja.

Esta plantilla incluye componentes reutilizables del sistema, como la cabecera y el pie de página, permitiendo que el resto de vistas definan únicamente el contenido específico. Este enfoque reduce la duplicación de código y mejora la mantenibilidad del sistema, facilitando las modificaciones globales.

Asimismo, se importa una hoja de estilos CSS común para toda la aplicación, garantizando coherencia visual en todas las páginas mediante el uso de la misma paleta de colores, tipografía y estructura.

Por último, se integra la librería de iconos open source Iconify [16], lo que permite incorporar iconos de forma sencilla en las distintas vistas.

Cabe destacar que el frontend incluye múltiples vistas con sus hojas de estilos y scripts dinámicos. Sin embargo, en esta sección se comentan únicamente los elementos más representativos técnicamente, evitando detallar componentes HTML simples o repetitivos, o estilos o validaciones sin importancia.

```
async function actualizarTabla() {
  const params = new URLSearchParams({
    id: idFiltro.value,
    telefono: telefonoFiltro.value,
    estado: estadoFiltro.value,
    tramite: tramiteFiltro.value,
    empleado: empleadoFiltro.value,
    orden: ordenActual,
    direccion: direccionActual,
    per_page: per_pageFiltro.value,
    page: pageActual
  });

  const res = await fetch(`/api/peticiones?${params}`);
  const data = await res.json();

  tabla_peticiones.innerHTML = data.peticiones.map(p =>
    `<tr>
      <td><a href="${urlSummaryPeticion.replace('0', p.id)}">${p.id}</a></td>
      <td>${p.telefono}</td>
      <td>${p.estado}</td>
      <td>${p.tramite}</td>
      <td>${p.creacion}</td>
      <td>${p.asignacion}</td>
    </tr>`
  ).join("");

  pageActual = data.pageActual;
  totalPages = data.totalPages;

  cargarPaginacion();
}
```

Figura 5.2: Fragmento de código de la construcción dinámica de la tabla de peticiones.

Los componentes principales de la web son las tablas que muestran la información correspondiente a las peticiones, empleados o trámites. El contenido de estas tablas se carga dinámicamente en función de los filtros, el orden o la paginación. La Figura 5.2 muestra el fragmento de código en el que se realiza una petición al backend, concretamente al endpoint “/api/peticiones”. En este contexto, un endpoint hace referencia a una ruta que recibe una solicitud y devuelve un recurso, en este caso, recupera la información correspondiente a las peticiones de la base de datos para actualizarla en la tabla.

```
<div id="peticion-resumen">
  <form method="post">
    <div id="peticion-detalles">
      {%include "components/tramites/" ~ petition.idTramite ~ ".jinja" %}
    </div>
    {%if (petition.estadoActual.valor == "Asignada")%}
    <div class="botones">
      <button type="submit" id="peticion-cancelar" name="cancelar" formnovalidate>Cancelar</button>
      <button type="submit" id="peticion-actualizar" name="actualizar">Actualizar</button>
      <button type="submit" id="peticion-completar" name="completar">Completar</button>
    </div>
    {%endif%}
  </form>
</div>
```

Figura 5.3: Fragmento de código de la plantilla resumen de una petición.

```
inputTramite.addEventListener("change", async () => {

  const idTramite = inputTramite.value;

  if(idTramite=='0'){
    templateContainer.innerHTML = "";
    return;
  }

  const response = await fetch(`/cargar_plantilla/${idTramite}`);
  const html = await response.text();

  templateContainer.innerHTML = html;
```

Figura 5.4: Fragmento de código de carga dinámica de las plantillas de cada trámite.

De forma análoga, las plantillas que contienen la información correspondiente a cada trámite se renderizan en tiempo de ejecución, mostrando el formulario correspondiente dependiendo del trámite escogido, Figuras 5.3 y 5.4.

El resto de componentes que abundan en la aplicación son formularios, que no presentan nada especial salvo alguna validación o evento dinámico.

5.2. Implementación de la base de datos

La implementación de la base de datos se ha llevado a cabo mediante tablas SQL, definiendo en cada atributo su tipo de dato, restricciones de unicidad y posibilidad de valores nulos. Asimismo, se han establecido las claves primarias para cada tabla y las claves foráneas para modelar las relaciones entre entidades.

La Figura 5.5 muestra la función encargada de inicializar la base de datos en la aplicación, ejecutando el archivo SQL correspondiente durante el arranque del sistema.

```
def reiniciar_bd(app):  
  
    ruta_base = app.config['DB_PATH']  
    ruta_sql = os.path.join(ruta_base, "data.sql")  
  
    with app.app_context():  
  
        with open(ruta_sql, 'r', encoding='utf-8') as f:  
            sql_script = f.read()  
  
        conn = db.engine.raw_connection()  
        cursor = conn.cursor()  
        cursor.executescript(sql_script)  
        conn.commit()  
        conn.close()  
  
    poblar_peticiones()
```

Figura 5.5: Código de la función de inicialización de la base de datos.

Para gestionar la base de datos dentro de la aplicación se utiliza SQLAlchemy, lo que permite definir clases Python equivalentes a cada tabla y trabajar con los registros como objetos. La Figura 5.6 muestra un ejemplo de consulta dinámica sobre la tabla de peticiones, en la que se aplican filtros, ordenación y paginación en función de los parámetros recibidos. El resultado se almacena en la variable “peticiones” como una lista de objetos de la clase correspondiente, permitiendo acceder a sus atributos y métodos directamente.

```

stmt = select(Peticion)
if id:
    stmt = stmt.where(Peticion.id.like(f"%{id}%"))
if telefono:
    stmt = stmt.where(Peticion.telefono.like(f"%{telefono}%"))
if idEstado:
    stmt = stmt.where(Peticion.idEstadoActual==idEstado)
if idTramite:
    stmt = stmt.where(Peticion.idTramite==idTramite)
if empleado:
    if empleado=='-':
        stmt = stmt.where(Peticion.idEmpleadoAsignado.is_(None))
    else:
        subq = select(Empleado).where(or_(
            Empleado.username.ilike(f"%{empleado}%"),
            Empleado.nombre.ilike(f"%{empleado}%"),
            Empleado.apellido1.ilike(f"%{empleado}%"),
            Empleado.apellido2.ilike(f"%{empleado}%")
        )).subquery()
        stmt = stmt.join(subq, Peticion.idEmpleadoAsignado == subq.c.id)

direc = asc if direccion == "ascendente" else desc

if orden in ("id", "telefono"):
    stmt = stmt.order_by(direc(getattr(Peticion, orden)))
elif orden == "estado":
    stmt = stmt.join(Estado, Peticion.idEstadoActual==Estado.id).order_by(direc(func.lower(Estado.valor)))
elif orden == "tramite":
    stmt = stmt.join(Tramite, Peticion.idTramite==Tramite.id).order_by(direc(func.lower(Tramite.valor)))
elif orden == "creacion":
    stmt = stmt.join(Hito, (Peticion.id==Hito.idPeticion) & (Peticion.idEstadoActual == Hito.idEstado)).order_by(direc(Hito.updated_at))
elif orden == "asignacion":
    stmt = stmt.outerjoin(Empleado, Peticion.idEmpleadoAsignado==Empleado.id).order_by(direc(func.lower(Empleado.username)).nulls_last(), direc(Peticion.id))

peticiones = db.session.scalars(stmt.offset((page-1)*per_page).limit(per_page))

```

Figura 5.6: Fragmento de código de la consulta de peticiones.

5.3. Implementación del backend

En esta sección se va a tratar el desarrollo del backend de la aplicación web, centrándolo en las rutas que ejercen como controladores de las vistas. Por modularidad, se han definido distintos blueprints (módulos de Flask que permiten agrupar rutas relacionadas) para cada parte de la aplicación, lo que facilita una organización más clara del código al separar su funcionalidad en distintos archivos Python, así como un mejor mantenimiento.

El primer blueprint es el de inicio, que se encarga de redireccionar a los empleados no identificados en el sistema al inicio de sesión. También incluye un endpoint encargado de renderizar dinámicamente la plantilla con el formulario específico de cada trámite, ya que, en función del trámite seleccionado por el empleado al crear una petición, se deberá mostrar el formulario correspondiente.

El segundo blueprint es el de autenticación, que incluye las rutas para iniciar y cerrar sesión. Las sesiones se gestionan mediante la extensión Flask-Login, que incluye funciones que simplifican la gestión de usuarios identificados, como el decorador `@login_required`, que se escribe en las funciones de las rutas para bloquear el acceso a usuarios no autenticados. Además, se ha implementado un sistema de permisos para cada rol, Figura 5.7, que se almacenan en un archivo JSON y se cargan en la sesión de cada usuario cuando se identifican. La Figura 5.8 muestra el código de las funciones que gestionan los permisos, incluido un decorador personalizado, `@permiso_requerido`, que actúa de forma análoga a `@login_required`, bloqueando el acceso a las rutas a las que el usuario no tenga permiso.

```
{
  "Administrador":[
    "ver_peticiones",
    "ver_empleados",
    "crear_empleado",
    "editar_empleado",
    "eliminar_empleado",
    "ver_tramites",
    "crear_tramite",
    "editar_tramite",
    "eliminar_tramite",
    "ver_estadisticas"
  ],
  "Secretario":[
    "ver_peticiones",
    "crear_peticion",
    "asignar_peticion",
    "ver_estadisticas"
  ]
}
```

Figura 5.7: JSON con los permisos de cada rol.

```
#Devuelve la lista de permisos del usuario. Rol = current_user.rol.valor
def cargar_permisos(rol):

    ruta = os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "data\\permisos.json")

    with open(ruta, "r", encoding="utf-8") as f:
        session["permisos"] = json.load(f)[rol]

#Comprueba si un usuario tiene el permisos necesario con un decorador
def permiso_requerido(permiso):
    def decorator(f):
        @wraps(f)
        def decorated_function(*args, **kwargs):
            if not current_user.is_authenticated:
                abort(401)
            else:
                if permiso not in session.get("permisos", []):
                    abort(403)
                return f(*args, **kwargs)
            return decorated_function
        return decorator

#Comprueba si un usuario tiene el permisos necesario
def verificar_permiso(permiso):
    if not current_user.is_authenticated:
        abort(401)
    else:
        if permiso not in session.get("permisos", []):
            abort(403)
    return
```

Figura 5.8: Código de las funciones para la gestión de permisos en la aplicación.

El tercer blueprint es el más importante, incluye las rutas que gestionan las vistas y funcionalidades principales de la aplicación. En este módulo se encuentran las funciones que gestionan las peticiones, empleados, trámites y la información del perfil de cada empleado. Estas funciones incluyen principalmente la lógica para recuperar la información necesaria de la base de datos, como por ejemplo, renderizar las plantillas correspondientes y gestionar la información recibida de los formularios (Figura 5.6).

Tanto las peticiones como los empleados y trámites se gestionan de forma similar en el backend, ya que comparten la forma en la que se presentan en el frontend. En los tres casos se ha implementado una ruta para renderizar la página con la tabla y un endpoint para recuperar la información de la tabla en formato JSON. En el supuesto de las peticiones se ha añadido una ruta para mostrar el resumen de cada petición, con la funcionalidad correspondiente para asignarse la petición, completarla, cancelarla, editarla o desasignársela, y otra ruta independiente para crear nuevas peticiones. Sin embargo, en el caso de los empleados y trámites se simplifica la lógica reutilizando la misma ruta y plantilla tanto para mostrar los datos individuales y editarlos, como para crear nuevos registros, diferenciando ambos casos mediante la presencia o ausencia de un ID en la URL.

El cuarto blueprint sirve para gestionar la carga y subida de archivos en la web. Hasta el momento, esta funcionalidad se utiliza únicamente para recuperar la imagen de perfil de cada empleado que se muestra tanto en el perfil como en el historial de cada petición. También permite modificar la imagen de perfil, almacenando la imagen nueva y borrando la antigua.

Por último, hay un blueprint de errores, que se encarga únicamente de renderizar una plantilla personalizada en caso de error 404, es decir, cuando el empleado intenta acceder a un recurso inexistente, y de error 403, cuando el empleado intenta acceder a un recurso para el que no tiene permiso. También incluye un error 501, recurso no implementado, para el caso concreto en el que se intente crear una petición para un trámite del que aún no se ha implementado la plantilla con su formulario correspondiente.

Adicionalmente, se ha implementado otro blueprint independiente para modularizar la lógica de las APIs. Por el momento, este incluye un único endpoint que actúa como webhook para procesar los eventos de Dialogflow, cuya integración se comentará con mayor detalle en la sección siguiente. En este contexto, un webhook consiste en un mecanismo de comunicación basado en eventos que permite a una aplicación enviar información a otra en tiempo real mediante peticiones HTTP.

5.4. Implementación e integración de servicios

Esta sección se centra en explicar la implementación de los cuatro servicios integrados en la aplicación, poniendo especial atención en aquellos relacionados con la gestión de las llamadas, ya que constituyen el eje central del proyecto.

5.4.1. Implementación e integración del servicio de correo electrónico

Para la gestión del envío de correos electrónicos dentro de la aplicación web se ha utilizado la extensión Flask-Mail, que permite integrar fácilmente un sistema de notificaciones por correo electrónico dentro del framework Flask. Este servicio se utiliza para comunicar automáticamente a los empleados distinta información relacionada con su cuenta, como el envío de sus credenciales de acceso, el establecimiento de una nueva contraseña temporal o la activación y desactivación del acceso al sistema.

Teniendo en cuenta el alcance del proyecto, no se ha configurado un servidor de correo real. En su lugar, se ha utilizado un servidor SMTP local de pruebas ejecutado en localhost mediante la herramienta

aiosmtplib. De esta forma, los correos generados por la aplicación se muestran por consola, lo que permite verificar su correcto funcionamiento sin necesidad de utilizar un servidor real.

Finalmente, la configuración actual del servicio de correo electrónico se encuentra en el archivo `config.py`, donde se definen los parámetros necesarios para su funcionamiento, siendo fácilmente modificable para la futura integración del servidor real que se utilice en producción. El envío de correos se realiza de forma centralizada mediante una única función, Figura 5.9, lo que permite generar distintos tipos de mensajes ajustando únicamente los parámetros de entrada.

```
def enviar_mail(subject, sender=None, recipients=None, cc=None, body=''):

    if recipients is None:
        recipients = []
    if cc is None:
        cc = []

    msg = Message(
        subject=subject,
        sender=sender if sender else current_app.config['MAIL_DEFAULT_SENDER'],
        recipients=recipients,
        cc=cc,
        body=body
    )

    mail.send(msg)
```

Figura 5.9: Código de la función de envío de correos electrónicos.

5.4.2. Implementación e integración de los servicios de Google Cloud

La configuración de los servicios de Google se lleva a cabo desde la consola de Google Cloud, donde se crea un proyecto y se habilitan las APIs correspondientes al servicio de Speech-to-Text y Dialogflow. Una vez habilitadas las APIs, se descarga el archivo de credenciales en formato JSON y se almacena en las variables de entorno de la aplicación.

Hay dos puntos del proyecto en los que se integran estos servicios, el bot de Discord y el webhook de Dialogflow.

En el caso del bot de Discord, se integran los servicios mediante las bibliotecas oficiales de Google Cloud para Python, lo que permite utilizar las funciones de cada servicio de forma sencilla. El servicio de Speech-to-Text se utiliza únicamente para transcribir el audio de las llamadas VoIP. Nada más recibir el audio en la cola de entrada se envía al cliente SpeechClient de Google Cloud, que lo procesa y devuelve la transcripción. El servicio de Dialogflow se integra en el bot de Discord para procesar las transcripciones de las llamadas, enviando el texto recibido en una cola a través de la función de la Figura 5.10. Cada llamada se asocia a una sesión única de Dialogflow, de forma que se mantiene el contexto de la conversación durante toda la llamada, enviando el texto transcrito del usuario y recibiendo la respuesta en texto y audio generada por Dialogflow. También se reciben las acciones configuradas en Dialogflow, lo que permite conocer al bot el fin de cada llamada para poder colgar automáticamente.

```

def enviar_texto(session_id, user_text=None, bienvenida=False):

    session = dialogflow_client.session_path(PROJECT_ID, session_id)

    audio_config = dialogflow_v2.OutputAudioConfig(
        audio_encoding=dialogflow_v2.OutputAudioEncoding.OUTPUT_AUDIO_ENCODING_LINEAR_16,
        sample_rate_hertz = 16000,
        synthesize_speech_config=dialogflow_v2.SynthesizeSpeechConfig(
            voice=dialogflow_v2.VoiceSelectionParams(
                name="es-ES-Neural2-G"
            ),
            speaking_rate=1.0
        ),
    )

    if bienvenida:
        event_input = dialogflow_v2.EventInput(name="WELCOME", language_code="es")
        query_input = dialogflow_v2.QueryInput(event=event_input)
    elif user_text:
        text_input = dialogflow_v2.TextInput(text=user_text, language_code="es")
        query_input = dialogflow_v2.QueryInput(text=text_input)
    else:
        return

    response = dialogflow_client.detect_intent(
        request={
            "session":session,
            "query_input":query_input,
            "output_audio_config":audio_config
        }
    )

    texto = response.query_result.fulfillment_text
    audio = response.output_audio
    accion = response.query_result.action
    return texto, audio, accion

```

Figura 5.10: Código de la función de envío de transcripciones a Dialogflow.

En el caso del webhook de Dialogflow, se implementa un endpoint en el backend de la aplicación web para que Dialogflow pueda comunicarse con el sistema. Se utiliza para poder generar las peticiones de forma automática una vez el agente extrae los datos necesarios de la conversación, y en el caso de solicitar una cita en la Agencia Tributaria, para poder consultar si el servicio elegido por el usuario dispone de cita telefónica o no. Dialogflow envía la información de cada evento en formato JSON, que se procesa en el endpoint para extraer los datos necesarios y realizar las acciones correspondientes y enviar la respuesta de vuelta a Dialogflow. El webhook se configura en la consola de Dialogflow, Figura 5.11, indicando la URL del endpoint, que debe ser accesible desde Internet para que Dialogflow pueda comunicarse con él. Por ello, se utiliza la herramienta ngrok, que expone el servidor local de desarrollo a través de una URL pública, lo que permite el correcto funcionamiento del webhook sin desplegar la aplicación.

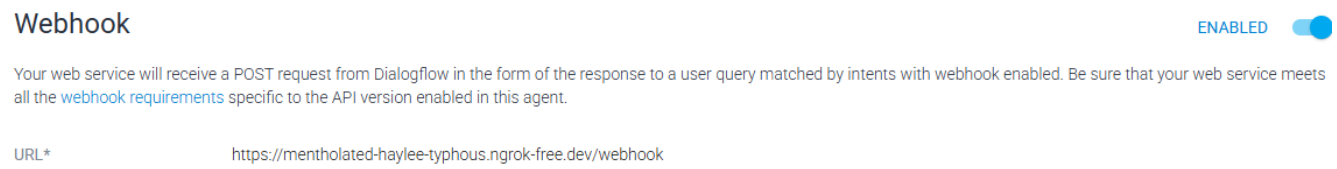


Figura 5.11: Configuración del webhook en la consola de Dialogflow.

5.4.3. Entrenamiento del agente de Dialogflow

El entrenamiento del agente de IA se ha llevado a cabo desde la consola de Dialogflow, donde se ha definido el flujo de conversación mediante la creación de intents y entities y su unión mediante contextos y acciones. Dialogflow oculta el modelo o algoritmo interno de Machine Learning que utiliza para procesar el lenguaje natural; solo permite establecer el umbral de confianza para el reconocimiento de cada intent.

Cada intent puede activarse mediante el lanzamiento de un evento, como el evento de bienvenida al iniciar la llamada, o mediante el reconocimiento de una frase de entrenamiento, Figura 5.12. Por ello, el entrenamiento del agente se ha centrado principalmente en establecer frases de entrenamiento significativas para cada intent, pues cuantas más frases de entrenamiento se establezcan menor será la probabilidad de error en el reconocimiento de los intents y en la extracción de los datos.



Figura 5.12: Establecimiento de frases de entrenamiento en Dialogflow.

Dialogflow también ofrece una página de entrenamiento, Figura 5.13, donde se muestran las conversaciones procesadas por el agente, indicando los intents reconocidos y los datos extraídos, permitiendo validar el correcto funcionamiento del agente y modificar los posibles errores, y asociando automáticamente las frases validadas a cada intent para su reentrenamiento.

WELCOME

Feb 19 7 REQUESTS 1 NO MATCH

USER SAYS quiero el papel padronal

PARAMETER NAME	ENTITY	RESOLVED VALUE	
tramite	@tramite	papel padronal	✕

INTENT tramite_certificado_empadronamiento

CONTEXT OUT esperando_tramite certificado_empadronamiento tramite_certificado_empadronamiento-followup

USER SAYS me llamo ana

INTENT tramite_certificado_empadronamiento - obtener datos

CONTEXT OUT tramite_certificado_empadronamiento-obtenerdatos-followup

USER SAYS sanz perez

INTENT tramite_certificado_empadronamiento - obtener datos

CONTEXT OUT tramite_certificado_empadronamiento-obtenerdatos-followup

USER SAYS 12345678Z

INTENT tramite_certificado_empadronamiento - obtener datos

CONTEXT OUT tramite_certificado_empadronamiento-obtenerdatos-followup

CLOSE
APPROVE

Figura 5.13: Página de entrenamiento en Dialogflow.

En algunos intents también hay que definir un contexto, Figura 5.14, lo que permite establecer una relación entre intents, de forma que un intent con un contexto de entrada solo se podrá activar si un intent anterior ha establecido ese contexto como contexto de salida. Esto es especialmente útil para establecer un flujo de conversación, como responder solo a las preguntas relacionadas con el trámite solicitado o recuperar los datos necesarios para ese trámite concreto. Para recuperar los datos de la conversación hay que definir los parámetros en cada intent, Figura 5.15, indicando el tipo de dato esperado mediante entidades predefinidas o personalizadas.

tramite_certificado_empadronamiento - obtener datos

Contexts ?

tramite_certificado_empadronamiento-followup ✕ Add input context

2 tramite_certificado_empadronamiento-obtenerdatos-followup ✕ Add output context

Figura 5.14: Contexto de un intent en Dialogflow.

Action and parameters

Enter action name

REQUIRED ?	PARAMETER NAME ?	ENTITY ?	VALUE	IS LIST ?	PROMPTS ?
☒	nombre	@sys.given-name	\$nombre	☐	<speak>¿Cuál es...
☒	apellidos	@sys.person	\$apellidos	☐	<speak>¿Podría ...
☒	dni_pasaporte	@dni_pasaporte	\$dni_pasaporte	☐	<speak>Por favo...
☒	telefono	@telefono_es	\$telefono	☐	<speak>Dígame s...
☒	motivo	@sys.any	\$motivo	☐	<speak>¿Cuál es...

Figura 5.15: Parámetros de un intent en Dialogflow.

Para definir entidades personalizadas se puede establecer una lista de sinónimos para cada valor, lo que permite mejorar el reconocimiento de los datos por el agente, o definir una expresión regular, como en la Figura 5.16, lo que es especialmente útil para datos como el dni o el número telefónico que siguen un formato específico.

telefono_es

☒ Define synonyms 
☒ Regexp entity 
☐ Allow automated expansion
 ☐ Fuzzy matching 

```
(?:\+34|0034)?[6789][0-9]{8}
```

Figura 5.16: Entidad personalizada en Dialogflow.

5.4.4. Implementación e integración del bot de Discord

La implementación del bot de Discord para la gestión de llamadas VoIP es la más compleja técnicamente de entre todos los servicios integrados en la aplicación.

La configuración del bot se realiza desde la página de desarrolladores de Discord, Figura 5.17, desde donde se crea una nueva aplicación, se añade un bot a esta y se configuran los permisos necesarios para que el bot pueda actuar correctamente en el servidor.

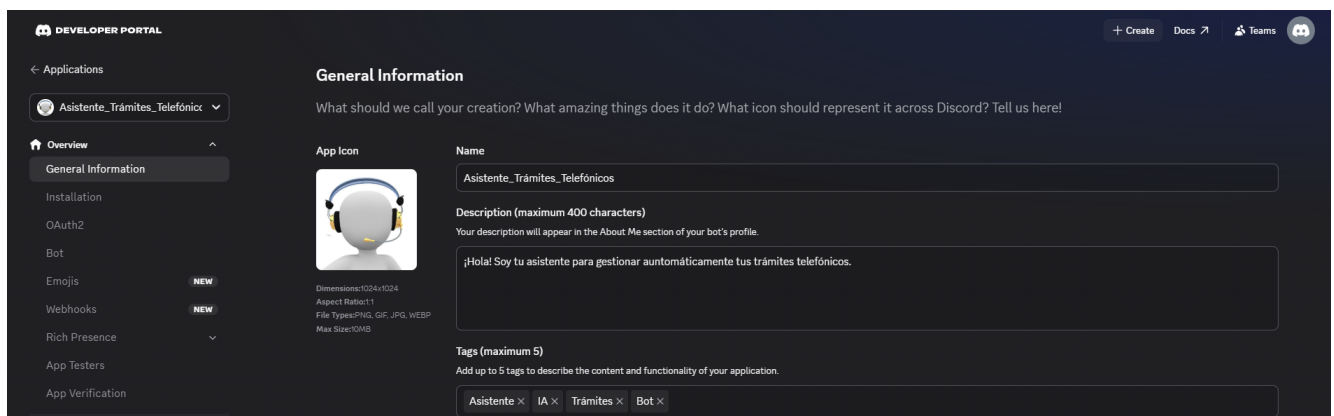


Figura 5.17: Página de desarrolladores de Discord.

Del mismo modo, se crea un servidor de Discord, se invita al bot a este y se configura el servidor para que tenga un canal de voz específico para las llamadas, limitado a dos personas para que solo pueda haber una llamada a la vez, Figura 5.18.

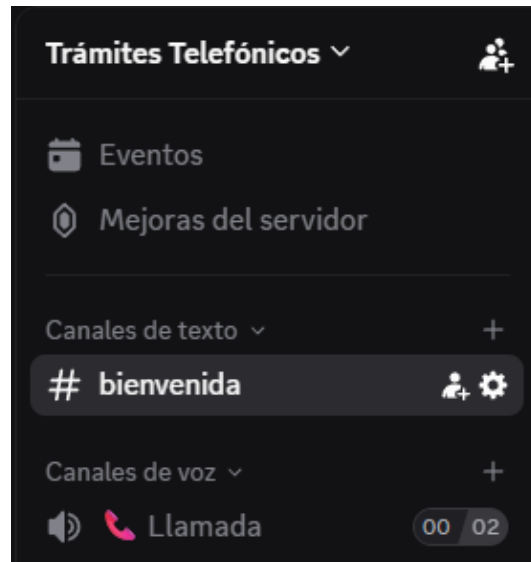


Figura 5.18: Servidor de Discord.

Una vez habilitado el bot en el servidor de Discord, se extrae el token de la página de desarrolladores y se añade como variable de entorno en la aplicación. Con todo configurado, se implementa la lógica para gestionar las llamadas utilizando la biblioteca oficial `dircord.py`, que permite gestionar los eventos de Discord relacionados con los canales de voz y los miembros. Se utiliza también la extensión `discord_ext_voice_recv`, que permite extraer el audio de los canales de voz, aunque no es una biblioteca oficial de Discord, sino un proyecto de código abierto [17], ya que Discord no facilita la recuperación de audio de los canales de voz.

La lógica de las llamadas se ejecuta de forma simultánea en distintos hilos, concurriendo la captura, procesamiento y reproducción de audio, utilizando colas para la comunicación entre ellos.

El hilo principal corresponde al bucle de eventos de Discord, que se encarga de gestionar los eventos asíncronos de la plataforma, así como la conexión al canal de voz y la recepción de los paquetes de audio. Cuando un usuario entra en el canal de voz, activa un evento que hace que el bot se una al canal, inicializando la conexión al cliente de voz. Tras establecer la conexión, se crean tres colas sincronizadas, una para el audio de entrada, otra para el texto transcrito y la última para el audio de salida. Estas colas siguen un patrón productor-consumidor, permitiendo que cada hilo se encargue de una tarea específica, comunicándose entre ellos sin bloquear el flujo de ejecución. Por último, se inicializan las clases encargadas de gestionar cada una de estas colas, las ejecuta en hilos independientes y comienza la escucha del canal de voz.

El primer componente que interviene es la clase `VozSink`, que se encarga de recibir los paquetes de audio del canal de voz, decodificarlos y almacenarlos en la cola de entrada de audio. La decodificación del audio se realiza mediante una clase `OpusDecoder` (Figura 5.19), que convierte los paquetes de audio recibidos por Discord a un formato entendible por los servicios de transcripción. En este caso, Discord transmite el audio utilizando el códec Opus con una frecuencia de muestreo de 48kHz, que es el estándar utilizado por la plataforma para las comunicaciones de voz en tiempo real. Este audio se decodifica a formato Pulse Code Modulation (PCM), el estándar global para la representación de audio digital sin compresión, y posteriormente se remuestrea a 16kHz con la biblioteca `numpy`, ya que es requerido por los servicios de transcripción para poder ser procesado correctamente. El audio se recibe en fragmentos de 20ms, correspondientes al tamaño de frame habitual de Opus. Dado que la frecuencia original es de 48kHz, cada fragmento contiene 960 muestras ($48kHz * 0,02s = 960$), lo cual define el tamaño de los bloques que deben procesarse en cada iteración de decodificación. Finalmente, se almacenan estos fragmentos en la cola de entrada de audio como un array de bytes.

```
class OpusDecoder:
    def __init__(self, rate=48000, channels=1):
        self.decoder = opuslib.Decoder(rate, channels)

    def decode(self, opus_bytes):
        if not opus_bytes:
            return None
        try:
            pcm48 = self.decoder.decode(opus_bytes, frame_size=960, decode_fec=True)
            pcm48 = np.frombuffer(pcm48, dtype=np.int16).astype(np.float32)
            return resample_poly(pcm48, up=1, down=3).astype(np.int16).tobytes()
        except opuslib.exceptions.OpusError:
            return None
```

Figura 5.19: Código de la clase OpusDecoder.

A continuación, actúa el hilo de transcripción STTWorkerGoogle, que se encarga de consumir en bucle el audio de la cola de entrada. Según va consumiendo los fragmentos de audio, los va almacenando en un buffer para completar la frase completa del usuario. Cuando se detecta un silencio prolongado por parte del usuario, 1.2 segundos sin recibir audio, se considera que el usuario ha terminado de hablar y se envía el audio almacenado en el buffer al cliente de Speech-to-Text de Google Cloud. Una vez recibida la transcripción, se almacena en la cola de texto. También se ha implementado una clase de transcripción alternativa, STTWorkerVosk, que utiliza un modelo de reconocimiento de voz local, para transcribir el audio en el caso de que el servicio de google no esté disponible o que los créditos sean insuficientes. Este modelo es menos preciso que el de Google, pero permite mantener la funcionalidad, ya que funciona de forma similar.

Posteriormente, el hilo de procesamiento AgentWorker consume el texto de la cola de transcripciones y lo envía al agente de Dialogflow como se explicó en la subsección anterior, añadiendo la respuesta de audio en la cola de salida. También detecta las acciones de fin de llamada, que llaman a una función asíncrona que espera a que se consuma el audio de la cola de salida para colgar la llamada, expulsando al usuario del canal de voz.

Finalmente, el hilo de TTSTWorker se encarga de consumir el audio de la cola de salida y reproducirlo en el canal de voz. Para poder reproducir, hay que remuestrear el audio de 16kHz a 48kHz, aunque no es necesario recodificarlo a formato Opus, ya que Discord acepta reproducirlo en formato PCM. Además, se debe duplicar el contenido del array de audio, ya que el audio recibido de Dialogflow es monoaural y hay que reproducirlo en estéreo. Al tener la reproducción de audio en un hilo independiente, se puede ir reproduciendo sin bloquear la escucha del canal de voz, permitiendo que no se pierda ningún fragmento de audio del usuario mientras que el bot responde.

Por último, el bot también detecta cuando el usuario abandona el canal de voz, procediendo a vaciar las colas, parar los hilos y liberar los recursos para esperar a la siguiente llamada.

Toda esta lógica se recoge en el esquema de la Figura 5.20 a modo de resumen, mostrando el flujo de ejecución de cada hilo y la comunicación entre ellos mediante las colas.

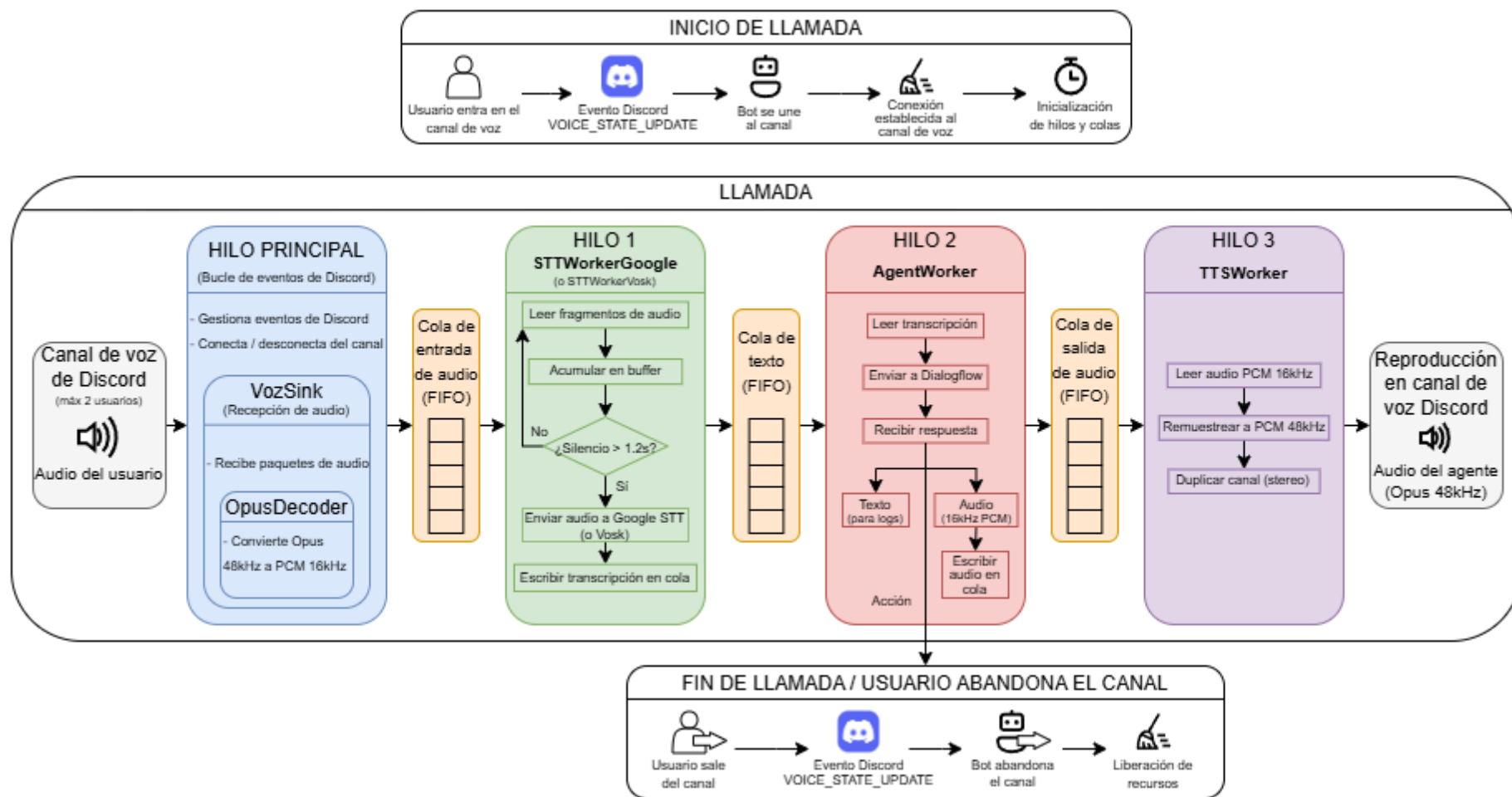


Figura 5.20: Resumen de la lógica del bot de Discord.

5.4.5. Implementación e integración de las funciones de RPA

Para la automatización de los trámites se han implementado distintas funciones de RPA utilizando la biblioteca de Selenium, que permite controlar el ratón y el teclado para interactuar con la interfaz de las aplicaciones. Estas funciones se integran en el backend de la aplicación web, de forma que se podrían ejecutar automáticamente nada más crear la petición. No obstante, para el proyecto y por motivos de seguridad, se ha decidido que el empleado ejecute manualmente la petición, para que pueda revisar cómo se rellena automáticamente la información necesaria de cada trámite, pero sin que se llegue a completar ningún trámite real. La Figura 5.21 muestra el código de la función correspondiente a la automatización del trámite de solicitud del certificado de empadronamiento en el Ayuntamiento de Valladolid. Se puede observar que se utiliza Selenium para abrir el navegador, acceder a la página oficial donde se completa el trámite, rellenar los campos necesarios con la información de la petición y simular su completitud sin llegar a enviar realmente el formulario.

```
def rpa_certificado_empadronamiento(informacion):  
  
    driver = webdriver.Chrome()  
    try:  
        driver.get("https://www.valladolid.es/es/temas/hacemos/padron-habitantes/solicitud-volante-certificado-empadronamiento-certificado-c")  
  
        driver.find_element(By.ID, "principal.gr_datos.nombre").send_keys(informacion.get("nombre",""))  
        driver.find_element(By.ID, "principal.gr_datos.apellidos").send_keys(informacion.get("apellidos",""))  
        driver.find_element(By.ID, "principal.gr_datos.dni_nie").send_keys(informacion.get("dni",""))  
        driver.find_element(By.ID, "principal.gr_datos.telefono").send_keys(informacion.get("telefono",""))  
        driver.find_element(By.ID, "principal.gr_datos.email").send_keys(informacion.get("email",""))  
        driver.find_element(By.ID, "principal.gr_datos.motivo").send_keys(informacion.get("motivo",""))  
  
        old_url = driver.current_url  
  
        try:  
            WebDriverWait(driver, 60).until(  
                expected_conditions.url_changes(old_url)  
            )  
        except:  
            pass  
        finally:  
            print("Certificado de empadronamiento solicitado correctamente.")  
            driver.quit()  
            return True  
  
    except:  
        print("Error al solicitar el certificado de empadronamiento.")  
        driver.quit()  
        return False
```

Figura 5.21: Código de la función de RPA para solicitar el certificado de empadronamiento.

El resto de trámites automatizados siguen una lógica similar; se localizan los elementos necesarios de cada página web mediante su ID o XPath, se rellenan o clican para ir completando los menús y formularios necesarios, y se simula la completitud del trámite sin llegar a enviar realmente nada.

Capítulo 6

Pruebas

Con el objetivo de validar el correcto funcionamiento del sistema implementado, se han realizado una serie de pruebas funcionales manuales sobre los distintos módulos que componen el proyecto. Aunque durante el desarrollo se llevaron a cabo verificaciones parciales para comprobar el progreso y corregir errores, las pruebas descritas en este capítulo se realizaron con el desarrollo finalizado.

Las pruebas se han realizado en un entorno local de desarrollo, utilizando el navegador Google Chrome para el acceso a la aplicación web y la aplicación de escritorio de Discord como medio de comunicación por voz con el sistema. Se ha empleado la versión de escritorio de Discord para garantizar una mayor estabilidad en la transmisión de audio, ya que la versión móvil presenta una conexión más inestable que puede acarrear pérdidas puntuales de paquetes.

6.0.1. Pruebas del sistema web

WEB01	Inicio de sesión exitoso.
Descripción	El empleado se identifica en la plataforma web con sus credenciales.
Resultado esperado	Se redirige a la página de inicio, con el empleado identificado en el sistema y con los permisos correspondientes cargados en la sesión.
Resultado obtenido	Funcionamiento conforme al resultado esperado.

Tabla 6.1: Caso de prueba WEB01 - Inicio de sesión exitoso

WEB02	Inicio de sesión denegado.
Descripción	El empleado introduce unas credenciales erróneas en la plataforma web o las de una cuenta desactivada.
Resultado esperado	El sistema deniega el inicio de sesión y avisa al empleado que las credenciales son incorrectas o la cuenta está desactivada.
Resultado obtenido	Funcionamiento conforme al resultado esperado.

Tabla 6.2: Caso de prueba WEB02 - Inicio de sesión denegado

WEB03	Buscar una petición.
Descripción	El empleado busca una petición en la tabla de peticiones mediante los filtros.
Resultado esperado	La tabla muestra las peticiones coincidentes con los criterios de búsqueda.
Resultado obtenido	Funcionamiento conforme al resultado esperado.

Tabla 6.3: Caso de prueba WEB03 - Buscar una petición

WEB04	Crear una petición manualmente.
Descripción	Un secretario crea una petición rellenando un formulario manualmente desde la plataforma web.
Resultado esperado	Se genera una nueva petición en el sistema con los datos introducidos.
Resultado obtenido	Funcionamiento conforme al resultado esperado.

Tabla 6.4: Caso de prueba WEB04 - Crear una petición manualmente

WEB05	Asignarse una petición.
Descripción	Un secretario se asigna una petición cuyo estado es pendiente.
Resultado esperado	Se actualiza el estado de la petición a asignada y se muestra el menú de edición de la petición al empleado asignado.
Resultado obtenido	Funcionamiento conforme al resultado esperado.

Tabla 6.5: Caso de prueba WEB05 - Asignarse una petición

WEB06	Desasignarse una petición.
Descripción	Un secretario se desasigna una petición que tenía asignada.
Resultado esperado	Se actualiza el estado de la petición a pendiente y se oculta el menú de edición de la petición al empleado.
Resultado obtenido	Funcionamiento conforme al resultado esperado.

Tabla 6.6: Caso de prueba WEB06 - Desasignarse una petición

WEB07	Editar los datos de una petición.
Descripción	El secretario asignado a una petición modifica su información y la guarda.
Resultado esperado	Se actualiza la información de la petición en la base de datos.
Resultado obtenido	La modificación de los datos se realiza correctamente conforme al resultado esperado en las peticiones de certificado de empadronamiento y tarjeta sanitaria. En el caso de las peticiones de cita en la AEAT, se detectó una incidencia al modificar el tipo de cita de presencial a telefónica, debido a que dicha opción no estaba restringida correctamente en función del servicio seleccionado. Tras aplicar la corrección, se repitió la prueba, obteniendo un resultado conforme a lo esperado.

Tabla 6.7: Caso de prueba WEB07 - Editar los datos de una petición

WEB08	Cancelar una petición.
Descripción	El secretario asignado a una petición la cancela.
Resultado esperado	Se actualiza el estado de la petición a cancelada y no se completa el trámite.
Resultado obtenido	Funcionamiento conforme al resultado esperado.

Tabla 6.8: Caso de prueba WEB08 - Cancelar una petición

WEB09	Completar una petición de certificado de empadronamiento.
Descripción	El secretario asignado a una petición de certificado de empadronamiento la completa.
Resultado esperado	Se abre un navegador en la página del Ayuntamiento de Valladolid donde se completa el trámite del certificado de empadronamiento, se rellena el formulario automáticamente con la información de la petición y se actualiza el estado de la petición a completada.
Resultado obtenido	Funcionamiento conforme al resultado esperado.

Tabla 6.9: Caso de prueba WEB09 - Completar una petición de certificado de empadronamiento

WEB10	Completar una petición de cita en la AEAT.
Descripción	El secretario asignado a una petición de cita en la AEAT la completa.
Resultado esperado	Se abre un navegador en la página de citas de la Agencia Tributaria, se completan automáticamente los menús y formularios con la información de la petición y el sistema comprueba la disponibilidad de citas. Si existe disponibilidad, se actualiza el estado de la petición a completada. En caso contrario, se informa por consola de la siguientes fechas disponibles.
Resultado obtenido	Funcionamiento conforme al resultado esperado.
Observaciones	Durante la prueba se comprobó que el sistema detecta correctamente la ausencia de citas para la fecha solicitada, mostrando por consola la siguiente fecha ofrecida por el portal. Actualmente, esta notificación se utiliza únicamente con fines de depuración y validación del flujo automatizado, quedando como trabajo futuro su integración en mecanismos más adecuados para el usuario.

Tabla 6.10: Caso de prueba WEB10 - Completar una petición de cita en la AEAT

WEB11	Completar una petición de tarjeta de SACYL.
Descripción	El secretario asignado a una petición de tarjeta sanitaria la completa.
Resultado esperado	Se abre un navegador en la página de SACYL donde se solicita la tarjeta sanitaria, se rellena el formulario automáticamente con la información de la petición y se actualiza el estado de la petición a completada.
Resultado obtenido	Se abre un navegador en la página de SACYL donde se solicita la tarjeta sanitaria y el sistema rellena automáticamente el formulario con la información de la petición. El flujo finaliza correctamente y la petición se actualiza como completada, funcionando conforme al resultado esperado. No obstante, en un número elevado de ejecuciones el proceso se detiene en un campo de tipo select, siendo necesario interactuar manualmente con dicho elemento para que la automatización continúe. Este comportamiento depende de la carga dinámica de la página web de Sanidad de Castilla y León.

Tabla 6.11: Caso de prueba WEB11 - Completar una petición de tarjeta de SACYL

Las Tablas 6.9, 6.10 y 6.11 recogen los resultados obtenidos en las distintas pruebas sobre los flujos automatizados mediante RPA, donde se observa un comportamiento correcto en la ejecución de los trámites.

Cabe destacar que este enfoque basado en técnicas de RPA implica una dependencia directa de la estructura y disposición de las interfaces web de los portales correspondientes. Por ello, aunque los resultados obtenidos validan el correcto funcionamiento del sistema en el momento de realización de las pruebas, no se garantiza su validez futura sin un mantenimiento continuo.

Además, para el caso concreto de la Agencia Tributaria, es habitual la realización de modificaciones en los servicios ofrecidos sobre las citas, lo que incrementa la probabilidad de que este flujo automatizado requiera ajustes en el futuro.

WEB12	Buscar un empleado.
Descripción	Un administrador busca un empleado en la tabla de empleados mediante los filtros.
Resultado esperado	La tabla muestra los empleados coincidentes con los criterios de búsqueda.
Resultado obtenido	Funcionamiento conforme al resultado esperado.

Tabla 6.12: Caso de prueba WEB12 - Buscar un empleado

WEB13	Crear un empleado.
Descripción	Un administrador crea una cuenta de empleado rellenando un formulario en la plataforma web.
Resultado esperado	Se genera una nueva cuenta de empleado en el sistema con los datos introducidos, comprobando que no exista otra cuenta con el mismo correo, y se envía un email a la cuenta creada con su usuario y su contraseña temporal.
Resultado obtenido	Funcionamiento conforme al resultado esperado.

Tabla 6.13: Caso de prueba WEB13 - Crear un empleado

WEB14	Modificar información de un empleado.
Descripción	Un administrador modifica la información de un empleado y la guarda.
Resultado esperado	El sistema actualiza la información de la cuenta y si se ha modificado el correo comprueba que no haya otra cuenta con el mismo.
Resultado obtenido	Funcionamiento conforme al resultado esperado.

Tabla 6.14: Caso de prueba WEB14 - Modificar información de un empleado

WEB15	Desactivar una cuenta de empleado.
Descripción	Un administrador desactiva la cuenta de un empleado.
Resultado esperado	Se deniega el acceso de la cuenta al sistema, se desasignan todas las peticiones asignadas por este empleado cambiando el estado a pendiente y se notifica por correo al empleado que su cuenta ha sido desactivada.
Resultado obtenido	Funcionamiento conforme al resultado esperado.

Tabla 6.15: Caso de prueba WEB15 - Desactivar una cuenta de empleado

WEB16	Activar una cuenta de empleado.
Descripción	Un administrador activa la cuenta de un empleado.
Resultado esperado	Se permite el acceso de la cuenta al sistema y se notifica por correo al empleado que su cuenta ha sido reactivada.
Resultado obtenido	Funcionamiento conforme al resultado esperado.

Tabla 6.16: Caso de prueba WEB16 - Activar una cuenta de empleado

WEB17	Reestablecer contraseña de un empleado.
Descripción	Un administrador restablece la contraseña de un empleado.
Resultado esperado	Se modifica la contraseña del empleado a una contraseña temporal generada automáticamente y se envía la contraseña por correo al empleado.
Resultado obtenido	Funcionamiento conforme al resultado esperado.

Tabla 6.17: Caso de prueba WEB17 - Reestablecer contraseña de un empleado

WEB18	Buscar un trámite.
Descripción	Un administrador busca un trámite en la tabla de trámites mediante los filtros.
Resultado esperado	La tabla muestra los trámites coincidentes con los criterios de búsqueda.
Resultado obtenido	Se detectó una incidencia al filtrar los trámites por el campo del id. Tras aplicar la corrección, se repitió la prueba, obteniendo un resultado conforme a lo esperado.

Tabla 6.18: Caso de prueba WEB18 - Buscar un trámite

WEB19	Crear un trámite.
Descripción	Un administrador genera un nuevo trámite rellenando un formulario.
Resultado esperado	Se genera un nuevo trámite en el sistema, comprobando que no existe otro trámite con el mismo título.
Resultado obtenido	Funcionamiento conforme al resultado esperado.

Tabla 6.19: Caso de prueba WEB19 - Crear un trámite

WEB20	Modificar el título de un trámite.
Descripción	Un administrador edita la información de un trámite.
Resultado esperado	Se actualiza la información correspondiente al título del trámite en el sistema, comprobando que no existe ningún otro trámite con el mismo título.
Resultado obtenido	Funcionamiento conforme al resultado esperado.

Tabla 6.20: Caso de prueba WEB20 - Modificar el título de un trámite

WEB21	Desactivar un trámite.
Descripción	Un administrador desactiva un trámite activo.
Resultado esperado	Se cancelan automáticamente todas las peticiones no finalizadas de ese trámite y no deja crear nuevas peticiones para esta gestión.
Resultado obtenido	Funcionamiento conforme al resultado esperado.

Tabla 6.21: Caso de prueba WEB21 - Desactivar un trámite

WEB22	Activar un trámite.
Descripción	Un administrador activa un trámite desactivado.
Resultado esperado	Se permite la creación de nuevas peticiones para esta gestión.
Resultado obtenido	Funcionamiento conforme al resultado esperado.

Tabla 6.22: Caso de prueba WEB22 - Activar un trámite

WEB23	Cambiar la foto de perfil.
Descripción	Un empleado modifica la foto de perfil de su cuenta.
Resultado esperado	Se borra la foto de perfil antigua para almacenar la nueva en su lugar, refrescando la web para mostrar la foto nueva.
Resultado obtenido	Funcionamiento conforme al resultado esperado.

Tabla 6.23: Caso de prueba WEB23 - Cambiar la foto de perfil

WEB24	Cambiar la contraseña de la cuenta.
Descripción	Un empleado modifica la contraseña de su cuenta introduciendo la contraseña antigua y la nueva.
Resultado esperado	Se comprueba que la contraseña antigua es correcta y se actualiza la contraseña de la cuenta con la nueva.
Resultado obtenido	Funcionamiento conforme al resultado esperado.

Tabla 6.24: Caso de prueba WEB24 - Cambiar la contraseña de la cuenta

WEB25	Cerrar sesión.
Descripción	Un empleado cierra la sesión de su cuenta.
Resultado esperado	Se cierra la sesión del empleado y se redirige a la página de inicio de sesión.
Resultado obtenido	Funcionamiento conforme al resultado esperado.

Tabla 6.25: Caso de prueba WEB25 - Cerrar sesión

6.0.2. Pruebas del agente conversacional

AC01	Detección del trámite.
Descripción	Un usuario indica mediante voz el trámite que desea ejecutar en la llamada.
Resultado esperado	El agente conversacional identifica correctamente el trámite deseado por el usuario.
Resultado obtenido	Funcionamiento conforme al resultado esperado.

Tabla 6.26: Caso de prueba AC01 - Detección del trámite

AC02	Extracción de datos para el certificado de empadronamiento.
Descripción	El usuario responde a las preguntas del agente.
Resultado esperado	El agente extrae de la conversación con el usuario la información necesaria para llevar a cabo el trámite del certificado de empadronamiento.
Resultado obtenido	Funcionamiento conforme al resultado esperado.

Tabla 6.27: Caso de prueba AC02 - Extracción de datos para el certificado de empadronamiento

AC03	Extracción de datos para la cita de la AEAT.
Descripción	El usuario responde a las preguntas del agente.
Resultado esperado	El agente extrae de la conversación con el usuario la información necesaria para llevar a cabo el trámite de la cita en la AEAT.
Resultado obtenido	Funcionamiento conforme al resultado esperado.

Tabla 6.28: Caso de prueba AC03 - Extracción de datos para la cita de la AEAT

AC04	Extracción de datos para la tarjeta de SACYL.
Descripción	El usuario responde a las preguntas del agente.
Resultado esperado	El agente extrae de la conversación con el usuario la información necesaria para llevar a cabo el trámite de la tarjeta de SACYL.
Resultado obtenido	Funcionamiento conforme al resultado esperado.

Tabla 6.29: Caso de prueba AC04 - Extracción de datos para la tarjeta de SACYL

Las Tablas 6.27, 6.28 y 6.29 recogen los resultados obtenidos en las distintas pruebas sobre la extracción de datos por parte del agente conversacional para los diferentes trámites, mostrando en líneas generales un comportamiento acorde al esperado. No obstante, se ha observado que pueden producirse errores puntuales de interpretación en el reconocimiento de algunos datos proporcionados por el usuario. Estas incidencias se consideran razonables dentro del margen de error inherente a las tecnologías basadas en

inteligencia artificial. Asimismo, este tipo de imprecisiones pueden reducirse mediante un proceso continuo de entrenamiento.

AC05	Confirmar los datos extraídos.
Descripción	El usuario confirma los datos extraídos por el agente.
Resultado esperado	El agente finaliza la llamada y se genera la petición en la base de datos.
Resultado obtenido	Funcionamiento conforme al resultado esperado.

Tabla 6.30: Caso de prueba AC05 - Confirmar los datos extraídos

AC06	Modificar uno de los datos extraídos.
Descripción	El usuario corrige en la llamada uno de los datos extraídos por el agente.
Resultado esperado	El agente modifica el valor del dato correctamente.
Resultado obtenido	La funcionalidad se ha validado satisfactoriamente en el trámite de certificado de empadronamiento, permitiendo la corrección de cualquier dato sin interrumpir el flujo.

Tabla 6.31: Caso de prueba AC06 - Modificar uno de los datos extraídos

AC07	Responder preguntas sin perder el flujo.
Descripción	El usuario interrumpe el flujo de la llamada para hacer una de las preguntas relacionadas programadas.
Resultado esperado	El agente responde a la pregunta y reanuda el flujo.
Resultado obtenido	La funcionalidad se ha validado satisfactoriamente sobre el flujo del trámite de certificado de empadronamiento, donde el agente responde a las preguntas predefinidas, retomando posteriormente la conversación.

Tabla 6.32: Caso de prueba AC07 - Responder preguntas sin perder el flujo

6.0.3. Pruebas del bot de Discord

DISC01	Conexión al canal de voz.
Descripción	El usuario se une al canal de llamada de Discord.
Resultado esperado	El bot de Discord se une al canal, establece la conexión de voz con el usuario e inicia el flujo del agente.
Resultado obtenido	Funcionamiento conforme al resultado esperado.

Tabla 6.33: Caso de prueba DISC01 - Conexión al canal de voz

DISC02	Recepción de audio del canal de voz.
Descripción	Se comprueba la recuperación de paquetes de audio de la llamada y su conversión para ser interpretables por el transcriptor.
Resultado esperado	Se extraen y decodifican los paquetes de audio correctamente, recuperando las respuestas del usuario.
Resultado obtenido	Funcionamiento conforme al resultado esperado.

Tabla 6.34: Caso de prueba DISC02 - Recepción de audio del canal de voz

DISC03	Reproducción de audio en el canal de voz.
Descripción	Se comprueba la reproducción del audio recibido del agente en el canal de Discord.
Resultado esperado	La reproducción es correcta y el audio se escucha con claridad.
Resultado obtenido	Funcionamiento conforme al resultado esperado.

Tabla 6.35: Caso de prueba DISC03 - Reproducción de audio en el canal de voz

DISC04	Colgar la llamada.
Descripción	El usuario abandona la llamada o la finaliza tras completar o cancelar un trámite.
Resultado esperado	El bot detecta la acción del usuario, libera los recursos y abandona el canal de Discord.
Resultado obtenido	Funcionamiento conforme al resultado esperado.

Tabla 6.36: Caso de prueba DISC04 - Colgar la llamada

A partir de las pruebas realizadas, se concluye que el sistema cumple satisfactoriamente con los requisitos funcionales establecidos, permitiendo a los usuarios solicitar los trámites de forma telefónica y a los empleados gestionar las solicitudes de manera eficaz. Respecto a los requisitos no funcionales, los resultados obtenidos muestran un comportamiento adecuado en términos de rendimiento y usabilidad, aunque algunos aspectos requieren una validación más exhaustiva en un entorno real futuro para poder afirmarlo de forma concluyente.

Capítulo 7

Conclusiones y trabajo futuro

7.1. Conclusiones

El objetivo principal de este proyecto ha sido desarrollar un sistema accesible que contribuya a reducir las barreras que la brecha digital impone a determinados sectores de la población. En particular, se ha planteado una solución que permita realizar distintos trámites administrativos mediante una llamada telefónica, ofreciendo una alternativa a los métodos digitales actuales para aquellos usuarios con dificultades para utilizarlos.

A lo largo del desarrollo del proyecto se ha logrado implementar un prototipo funcional capaz de atender y gestionar las solicitudes administrativas. El sistema permite recibir llamadas VoIP a través de un canal de Discord, interpretar las solicitudes del usuario mediante un agente conversacional y tramitar las peticiones a través de una plataforma de gestión para empleados. De esta manera, se ha conseguido cumplir con los objetivos y requisitos establecidos inicialmente, demostrando la viabilidad de la solución como alternativa para reducir la brecha digital.

Los resultados obtenidos tras la finalización del proyecto muestran que es posible integrar tecnologías de inteligencia artificial en la Administración pública para facilitar la interacción con los usuarios. Este enfoque permite simplificar procesos que actualmente pueden resultar complejos para determinados sectores de la población, poniendo de manifiesto que todavía existen nuevas vías para abordar el problema de la brecha digital. En particular, el aprovechamiento de tecnologías emergentes como la inteligencia artificial abre nuevas oportunidades para mejorar la accesibilidad de los servicios digitales.

No obstante, durante el desarrollo del proyecto se han identificado algunas limitaciones y dificultades que podrían suponer una barrera en un escenario de despliegue a gran escala. Por un lado, la utilización de servicios basados en inteligencia artificial y sistemas de llamadas telefónicas implica un coste económico asociado al uso de plataformas externas, que podría aumentar exponencialmente en función del número de solicitudes tramitadas, además de los costes derivados del personal y del mantenimiento del propio sistema. En este contexto, su aplicación práctica podría requerir apoyo institucional o financiación pública para ser factible, ya que está planteado como un servicio orientado a cubrir una necesidad social más que a generar beneficios como un modelo de negocio. Por otro lado, la incorporación de nuevos trámites en el sistema requiere un proceso de configuración y adaptación que puede resultar costoso en términos de tiempo y esfuerzo. Estas cuestiones evidencian la necesidad de continuar investigando y optimizando soluciones de este tipo, con el objetivo de encontrar un equilibrio entre la accesibilidad del servicio y los recursos necesarios para su mantenimiento.

En conclusión, el trabajo realizado demuestra que es posible desarrollar alternativas que mejoren el acceso al entorno digital mediante el uso de sistemas basados en inteligencia artificial y comunicación

por voz. Aunque se trata de un prototipo y todavía sería necesario evaluar su funcionamiento en un entorno real, la propuesta presentada plantea un enfoque prometedor que podría contribuir a mejorar la accesibilidad de los servicios digitales en el futuro.

7.2. Trabajo futuro

Como líneas de trabajo futuro, se plantean diversas mejoras y ampliaciones que permitirían reforzar la funcionalidad del sistema y ampliar su alcance, especialmente en el contexto de la posible integración de este proyecto en los sistemas actuales de las Administraciones públicas.

Ampliación de trámites. El núcleo del proyecto consiste en la realización de trámites administrativos, por lo que sería indispensable incluir el mayor número de trámites posibles en el sistema para intentar cubrir todas las necesidades de los usuarios. Habría que añadir nuevas gestiones de distintas instituciones y localidades, priorizando los más relevantes para el público objetivo. Esto implicaría modificar los permisos de la plataforma web creando nuevos roles, modularizando aún más los accesos a las peticiones por trámites, instituciones o ámbito territorial.

Integración de APIs oficiales. El caso idóneo para completar los trámites implicaría el uso de APIs internas para comunicarse directamente con el backend de las Administraciones correspondientes. Esto evitaría el uso de RPA en los portales web públicos para completarlos, evitando errores, posibles brechas de seguridad y mejorando el mantenimiento de la aplicación.

Mejoras en seguridad y autenticación. Con las llamadas telefónicas surge una necesidad en la producción futura del sistema y es la forma de identificar al llamante. La mayoría de las gestiones son personales y exigen autenticar al emisor, ya sea por protección de datos o para evitar estafas. Para la simulación actual no era necesario, ya que se trata de un entorno controlado, con trámites que no exigen esta identificación, pero la mayoría de los casos requieren identificación mediante DNI electrónico o certificado digital.

Por lo tanto, habría que crear un método de identificación telefónica. Una opción podría ser una autenticación de dos factores, basada en algo conocido, una contraseña pedida por el agente al inicio de la conversación, y algo poseído, el número telefónico con el que se realiza la llamada. El punto negativo de esta medida es que sería necesario vincular el teléfono a la contraseña, lo que implicaría que los usuarios tengan que realizar una gestión previa para registrarlo y poder disfrutar de los beneficios de este sistema.

Vista de estadísticas. Sería de gran utilidad generar una vista con estadísticas y datos en la plataforma web para empleados. Se podría utilizar para realizar análisis sobre las peticiones y así conocer información de valor para las instituciones. Por ejemplo, se podría sacar información sobre la cantidad de llamadas recibidas por localidad o Administración o las veces que se realiza un trámite concreto, para saber el número de empleados necesarios para gestionarlos o si sería útil programar un trámite basado en los datos.

Mejoras en el agente conversacional. Para tener un agente más completo, se podría complementar con una IA generativa que actúe cuando el mensaje del usuario no coincida con ninguna de las intenciones entrenadas en el agente. Esto daría más libertad y naturalidad a la conversación, permitiendo contestar a cualquier duda o aportar información de forma más sencilla, rompiendo con un flujo estricto y una conversación más robotizada.

En la misma línea, se podría ampliar la configuración del agente para que sea capaz de realizar llamadas y no solo recibirlas. De esta forma, se podría mantener al usuario informado ante cualquier cambio en el estado de su petición, para confirmar su completitud, avisarle si se ha cancelado por si quiere volver a intentarlo o incluso llamarle en caso de error en algún dato para volver a solicitárselo.

Portal web público. Como concepto para ampliar el alcance del proyecto y el público objetivo, se podría habilitar un formulario web público para tener otro método de solicitud de los trámites fuera de las llamadas telefónicas. Esta idea queda fuera del objetivo del proyecto, pero no sería difícil de

realizar ya que ya existe un formulario privado para empleados para generar peticiones y ayudaría a un gran sector que desea unificar todos los trámites en un portal único [4].

Bibliografía

- [1] S. Gómez, “La brecha digital: las tres dimensiones de las desigualdades sociodigitales.” <https://www.ferrerguardia.org/blog/articulos-8/la-brecha-digital-las-tres-dimensiones-de-las-desigualdades-sociodigitales-288>. Último acceso: 2025-10-7.
- [2] BBVA Innovación, “Superando la brecha digital: soluciones para conectar a las personas mayores con la tecnología.” <https://www.bbva.com/es/innovacion/superando-la-brecha-digital-soluciones-para-conectar-a-las-personas-mayores-con-la-tecnologia/>, 2023. Último acceso: 2025-10-7.
- [3] Instituto Nacional de Estadística (INE), “Encuesta sobre Equipamiento y Uso de Tecnologías de la Información y Comunicación (TIC) en los Hogares. Año 2024.” <https://www.ine.es/dyngs/Prerensa/TICH2024.htm>, 2024. Último acceso: 2025-10-7.
- [4] Fundación Ferrer Guardia, “La Brecha Digital y la Administración Digital en España.” https://www.ferrerguardia.org/web/content/22588?access_token=3db14e1b-3b9f-4076-a0fe-b29ae3ddf07e&unique=fd632c2b526838e0ca10224177fd268090ef3dd6, 2023. Último acceso: 2025-10-7.
- [5] Cóginiti. <https://cogniti.es/>. Último acceso: 2025-10-7.
- [6] U. de Valladolid, “Guía docente del trabajo de fin de grado 2025–2026 (mención computación).” https://apps.stic.uva.es/guias_docentes/uploads/2025/545/46978/1/Documento.pdf, 2025. Último acceso: 2026-2-3.
- [7] Jinja, “Jinja Documentation.” <https://jinja.palletsprojects.com/en/stable/>. Último acceso: 2025-10-23.
- [8] Flask, “Flask Documentation.” <https://flask.palletsprojects.com/en/stable/>. Último acceso: 2025-11-21.
- [9] Flask SQLAlchemy, “Flask SQLAlchemy Documentation.” <https://flask-sqlalchemy.readthedocs.io/en/stable/>. Último acceso: 2025-11-21.
- [10] Selenium, “Selenium Documentation.” <https://selenium-python.readthedocs.io/getting-started.html>. Último acceso: 2025-11-26.
- [11] Vapi. <https://dashboard.vapi.ai/>. Último acceso: 2025-11-17.
- [12] Google Cloud, “Dialogflow ES Documentation.” <https://docs.cloud.google.com/dialogflow/es/docs?hl=es>. Último acceso: 2025-12-15.
- [13] Synthflow. <https://synthflow.ai/>. Último acceso: 2025-11-17.
- [14] Twilio. <https://www.twilio.com/>. Último acceso: 2025-11-17.

- [15] Ngrok. <https://ngrok.com/>. Último acceso: 2025-12-22.
- [16] Iconify.Design. <https://iconify.design/>. Último acceso: 2025-10-23.
- [17] imayhaveborkedit, “discord-ext-voice-recv.” <https://github.com/imayhaveborkedit/discord-ext-voice-recv>. Último acceso: 2025-12-1.

Apéndice A

Manuales

A.1. Manual de instalación y despliegue

Este manual describe el proceso necesario para la instalación, configuración y despliegue del sistema desarrollado. Se detallan los requisitos previos, las dependencias externas y los pasos necesarios para ejecutar correctamente la aplicación en un entorno local de desarrollo.

A.1.1. Dependencias del sistema

El sistema desarrollado depende de varios servicios externos que permiten el funcionamiento del agente conversacional y la comunicación entre los distintos componentes del sistema. Concretamente, el sistema requiere el uso de los siguientes servicios:

- Discord, utilizado para realizar las llamadas VoIP, capturando y reproduciendo audio de un canal de voz.
- Dialogflow, encargado de interpretar la intención del usuario y recopilar la información necesaria para realizar el trámite.
- Google Cloud, necesario para la comunicación entre el bot de Discord y Dialogflow y el uso del servicio de reconocimiento de voz.
- Ngrok, utilizado para exponer el servidor local de desarrollo a Internet, permitiendo la comunicación entre Dialogflow y la plataforma web.

Además, para ejecutar el sistema es necesario disponer del siguiente software:

- Python 3.13 o superior.
- Gestor de paquetes pip.
- Navegador web moderno.
- Cuenta en Ngrok.
- Cuenta en Google Cloud.
- Cuenta en Discord.

Todas las dependencias necesarias para ejecutar el sistema se encuentran definidas en el archivo **requirements.txt**.

A.1.2. Instalación del proyecto

Para instalar el proyecto es necesario clonar el repositorio y preparar el entorno de ejecución de Python.

En primer lugar, se debe clonar el repositorio mediante el comando `git clone https://github.com/humaral/tfg.git`.

Una vez descargado, se accede al directorio del repositorio con `cd tfg`.

Posteriormente, se crea un entorno virtual de Python para aislar las dependencias del proyecto con `python -m venv venv`.

Una vez creado el entorno virtual, se activa con `venv\Scripts\activate` en sistemas Windows, y con `source venv/bin/activate` en Linux o MacOS.

Finalmente, se instalan las dependencias necesarias para ejecutar el proyecto con `pip install -r requirements.txt`.

Además, es necesario crear un archivo `.env` que contenga las variables de entorno utilizadas por la aplicación. Debe incluir al menos las siguientes variables:

- `DIALOGFLOW_CREDENTIALS`
- `DISCORD_TOKEN`
- `VOSK_MODEL_PATH` (*opcional*).

A.1.3. Configuración del agente de Dialogflow

El repositorio incluye una copia exportada del agente conversacional utilizado por el sistema, ubicada en `/scripts_aux/services/models/Operador.zip`. Pasos para importar este agente en Dialogflow se deben seguir los siguientes pasos:

1. Acceder a la consola de Dialogflow - <https://dialogflow.cloud.google.com>.
2. Crear un nuevo agente.
3. Seleccionar la opción **Import from ZIP** en la configuración.
4. Subir el archivo **Operador.zip**.

A.1.4. Configuración de Google Cloud

Para que la aplicación pueda comunicarse con Dialogflow y para poder usar los servicios de STT es necesario disponer de credenciales de Google Cloud. La configuración de este servicio se consigue siguiendo los siguientes pasos:

1. Acceder a la consola de Google Cloud - <https://console.cloud.google.com/>.
2. Crear un proyecto de Google Cloud.
3. Activar el servicio de facturación.
4. Vincular el agente de Dialogflow al proyecto.
5. Activar el servicio de Speech-to-Text en el proyecto. (*opcional*)
6. Crear una **Cloud Resource Manager API** asociada a los servicios de Dialogflow y STT y descargar el archivo JSON con sus credenciales.
7. Añadir en el archivo `.env` la variable de entorno `DIALOGFLOW_CREDENTIALS` con la ruta donde se almacene el archivo JSON con las credenciales.

A.1.5. Configuración de Vosk

En caso de preferir usar un modelo local para realizar el Speech-to-Text en vez del servicio de Google, se puede conseguir siguiendo estos pasos:

1. Descargar el modelo Vosk en <https://alphacephei.com/vosk/models>.
2. Descomprimir la carpeta y añadirla al proyecto.
3. Añadir en el archivo `.env` la variable de entorno `VOSK_MODEL_PATH` con la ruta donde se almacene la carpeta con el modelo.
4. Acceder al archivo `scripts_aux\services\discord_bot.py` y cambiar la variable global `USE_LOCAL_STT=True`.

A.1.6. Configuración del bot de Discord

El código del bot se encuentra en `scripts_aux\services\discord_bot.py` y para utilizarlo hay que seguir los siguientes pasos:

1. Acceder a Discord Developer Portal - <https://discord.com/developers>.
2. Crear una nueva aplicación.
3. Añadir un bot a la aplicación.
4. Copiar el token de autenticación del bot y añadirlo en `.env` en la variable `DISCORD_TOKEN`.
5. Crear un servidor de Discord.
6. Generar un enlace de invitación desde el portal de desarrolladores e invitar al bot al servidor de Discord.
7. Crear un canal de voz en el servidor de Discord, limitado a 2 usuarios.

Para que el bot pueda funcionar correctamente, deberá tener permisos en el servidor de Discord para conectarse a canales de voz, hablar y mover miembros.

A.1.7. Configuración de Ngrok

Antes de iniciar Ngrok por primera vez, hay que realizar los siguientes pasos:

1. Acceder al dashboard de Ngrok - <https://dashboard.ngrok.com/>.
2. Copiar el Authtoken y añadirlo al entorno con el comando `ngrok config add-authtoken TU_TOKEN_NGROK`.
3. Ir a la pestaña Domains en el dashboard de Ngrok y copiar la URL pública donde se levantará la aplicación web.
4. En la consola de Dialogflow, acceder a la pestaña de fulfillment y añadir la URL del webhook de la aplicación web - `https://TU_URL_NGROK/webhook`.

A.1.8. Despliegue del sistema

Para ejecutar el sistema es necesario iniciar los distintos componentes del sistema en un orden determinado.

En primer lugar, se debe iniciar el servidor de correo SMTPD utilizado para simular el envío de correos electrónicos mediante el comando `py -m aiosmtpd -n -l localhost:1025`, recibiendo los correos en texto plano en el propio terminal de ejecución.

Posteriormente, se inicia el servidor Flask con **py app.py**, quedando la aplicación web disponible desde un navegador en la dirección **http://localhost:5000**. El acceso inicial a la aplicación se realiza mediante una cuenta de administrador por defecto (usuario: **admin.admin**, contraseña: **admin**), desde la cual es posible crear nuevas cuentas de usuario. Por motivos de seguridad, se recomienda modificar la contraseña tras el primer acceso o desactivar dicha cuenta una vez creada una propia.

Una vez iniciada la aplicación web, se debe exponer el servidor local a Internet mediante el servicio de Ngrok, ejecutando **ngrok http 5000**.

Por último, se inicializa el bot de Discord ejecutando **py .\scripts_aux\services\discord_bot.py**. El bot se conectará al servidor de Discord y, cuando el usuario acceda al canal de voz, dará comienzo la llamada.

A.2. Manual de usuario

Este manual describe el uso de la aplicación web de gestión de trámites telefónicos desde el punto de vista del empleado, detallando las principales funcionalidades disponibles.

A.2.1. Login en la aplicación

El formulario de inicio de sesión se presenta en un recuadro con un fondo gris claro y bordes redondeados. En el centro, el título "Identificación de Empleado" está escrito en una tipografía sans-serif, con "Identificación de" en negro y "Empleado" en un color más oscuro. Debajo del título, hay un campo de entrada de texto con un fondo azul claro y el texto "hugo.martin" en gris. Justo debajo de este campo, hay otro campo de entrada de texto con un fondo blanco y los caracteres "...." en gris, acompañado a la derecha de un icono de ojo con una diagonal roja, lo que indica que la contraseña está oculta. Al final del formulario, hay un botón rectangular con un fondo azul oscuro y el texto "Iniciar Sesión" en blanco.

Figura A.1: Formulario de inicio de sesión.

Al acceder a la aplicación, el empleado es recibido por una pantalla de login, donde se muestra un formulario de acceso, Figura A.1. Se introduce el usuario o correo electrónico del empleado y su contraseña privada y el sistema te redirige a la página principal con la sesión iniciada.

A.2.2. Funcionalidades del administrador

Portal de Trámites Telefónicos
Sistema Simulado de La Administración Pública

Hugo Martín
Administrador

Peticiones Empleados Trámites Estadísticas (1)

(2) ID Teléfono --Selecciona un Estado-- --Selecciona un Trámite-- Empleado Asignado

ID	Teléfono	Estado	Trámite	Fecha de Creación	Asignado A
1	627899040	Pendiente	Certificado de Empadronamiento	13/02/2026 17:15:25	-
2	722643980	Pendiente	Cita AEAT	13/02/2026 17:15:25	-
3	834521234	Pendiente	Cita AEAT	13/02/2026 17:15:25	-
4	952439538	Completada	Tarjeta Sanitaria SACYL	13/02/2026 17:15:25	-
5	706365383	Pendiente	Cita AEAT	13/02/2026 17:15:25	-
6	806582096	Pendiente	Cita AEAT	13/02/2026 17:15:25	-
7	718425749	Pendiente	Cita AEAT	13/02/2026 17:15:25	-
8	980552937	Pendiente	Certificado de Empadronamiento	13/02/2026 17:15:25	-
9	895125344	Pendiente	Cita AEAT	13/02/2026 17:15:25	-
10	741435983	Pendiente	Tarjeta Sanitaria SACYL	13/02/2026 17:15:25	-

(5) 10 Peticiones por página (6) 1 2 ... 4 >

© 2026 Plataforma de Transmisión Telefónica Interna - Proyecto Universitario
Versión: 0.1.0 | Última actualización: 23/03/2026
Soporte: soporte@tramitestelefonicos.es | Tel. XXXXXXXXXX

Figura A.2: Página con la tabla de peticiones para el rol de administrador.

Al acceder en el sistema con una cuenta de administrador, se muestra como pantalla de inicio la página de peticiones, Figura A.2. Dicha página, muestra como módulo principal una tabla con información básica de todas las peticiones del sistema y el empleado puede interactuar con los siguientes componentes:

1. Barra de navegación. En la parte superior izquierda se muestra un menú para navegar entre las páginas principales de la aplicación. El administrador puede usar este menú para cambiar entre la tabla de peticiones, empleados o trámites.
2. Filtros de búsqueda. Se pueden usar para filtrar el contenido de la tabla de peticiones, pudiendo localizar rápidamente una petición por su ID, el teléfono del llamante, el estado o trámite de la petición o el nombre del empleado asignado.
3. Filtros de ordenación. Al clicar sobre el encabezado de una columna de la tabla, se ordenarán las peticiones según ese campo. Si se vuelve a clicar sobre el mismo campo, se cambiará de orden de ascendente a descendente o viceversa.
4. Redirección al resumen de cada petición. Si se clicca sobre el ID de una petición de la tabla, se carga la página con la información completa de una petición. Los administradores solo pueden ver la información, no tienen permiso para la modificación o ejecución.
5. Filtro de paginación. Al clicar sobre este elemento se puede modificar el número de peticiones que aparecen en cada página de la tabla.
6. Menú de paginación. Interactuando con este menú se puede cambiar la página de peticiones que se muestra en la tabla.
7. Información del empleado. Este botón muestra el nombre, rol y foto de perfil del empleado registrado. Al clicarlo, se redirecciona a la página con la información completa del perfil del empleado.

Figure A.3 shows a web interface for managing employees. At the top, there is a search bar with filters for 'Usuario', 'Nombre', 'Email', 'Rol', and 'Activo'. A table below lists two employees: 'hugo.martin' (Administrador) and 'juan.sanz' (Secretario). A pagination control shows '10 Empleados por página'. A 'Nuevo Empleado' button is in the top right.

Usuario	Nombre	Email	Rol	Activo
hugo.martin	Hugo Martín	hugo.martin@tramitestelefonicos.es	Administrador	✓
juan.sanz	Juan Sanz	juan.sanz@tramitestelefonicos.es	Secretario	✓

Figura A.3: Página con la tabla de empleados.

Al clicar sobre el botón de empleados del menú de navegación se accede a la página con la tabla de empleados, Figura A.3. Esta tabla muestra la información de todos los empleados registrados en el sistema, con una funcionalidad similar a la tabla de peticiones:

1. Filtros de búsqueda. Sirven para localizar rápidamente un empleado filtrando por su usuario, su nombre o apellidos, su email, su rol o si la cuenta está activada o desactivada.
2. Botón de creación de empleados. Al clicarlo, se carga la página con el formulario para añadir nuevos empleados al sistema, Figura A.4.
3. Filtros de ordenación. Al igual que en la tabla de peticiones, si se clicca sobre el encabezado de una columna se ordenará el contenido de la tabla según ese campo. Se podrá cambiar entre orden ascendente y descente clicando varias veces sobre el mismo encabezado.
4. Filtro de paginación. Para cambiar el número de empleados que aparecen en cada página de la tabla.
5. Menú de paginación. Para cambiar la página de la tabla mostrada.
6. Botón de edición de empleado. Este botón tiene una doble función, muestra si la cuenta está activada o desactivada con un icono y al mismo tiempo sirve para redirigir a la página de edición de empleado, Figura A.5.

Figure A.4 shows a form titled 'Registrar Empleado'. It contains input fields for 'Nombre', 'Primer Apellido', 'Segundo Apellido', and 'Email'. Below these are radio buttons for 'Administrador' and 'Secretario', with 'Secretario' selected. There is a checkbox for 'Activo' which is checked. At the bottom are 'Cancelar' and 'Crear' buttons.

Figura A.4: Formulario para añadir un nuevo empleado al sistema.

El formulario de creación de empleados, Figura A.4, se compone de varios campos obligatorios que hay que rellenar, salvo el segundo apellido que se puede dejar en blanco. El email es único por cada empleado y no se podrá crear la cuenta si ya está registrado en el sistema. El usuario y la contraseña temporal se generan automáticamente por el sistema y se envían al correo electrónico del nuevo empleado nada más pulsar el botón de crear.

Editar Empleado

Hugo

Martín

Alonso

hugo.martin@tramitestelefonicos.es

Administrador ☒ Secretario ☐

Activo: ☒

Cancelar Resetear Contraseña Editar

Figura A.5: Formulario para editar la información de un empleado existente.

El formulario de edición de empleados, Figura A.5, se compone de varios campos rellenos con la información actual del empleado elegido para editar. Se puede modificar cualquier campo y se actualizará el valor en el sistema al clicar sobre el botón editar. Si se modifica el email, se comprobará que no está registrado ya en el sistema, en cuyo caso no se modificaría la información del empleado. También hay un botón de restablecimiento de contraseña, al clicarlo se generará automáticamente una contraseña temporal, que se enviará al correo electrónico del empleado. Por último, no se puede eliminar una cuenta de empleado para mantener una trazabilidad en el sistema, solo se puede desactivar para anular su acceso.

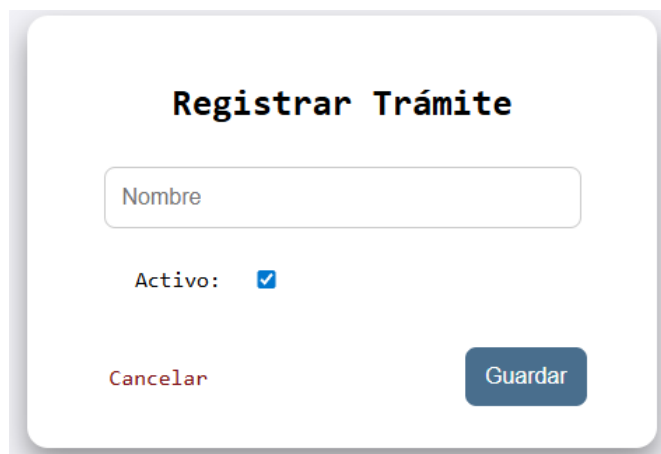
id Nombre Activo: ☒ Nuevo Trámite

ID	Nombre	Activo
1	Certificado de Empadronamiento	☒
2	Cita AEAT	☒
3	Tarjeta Sanitaria SACYL	☒

10 Trámites por página 1

Figura A.6: Página con la tabla de trámites.

Al clicar sobre el botón de trámites del menú de navegación se accede a la página con la tabla de trámites, Figura A.6. Esta tabla muestra la información de todos los trámites registrados en el sistema. La funcionalidad y disposición de la información es igual a la tabla de empleados.



El formulario 'Registrar Trámite' tiene un título centralizado. Debajo de él hay un campo de texto con el placeholder 'Nombre'. Más abajo, la etiqueta 'Activo:' está seguida de un icono de casilla marcada. En la parte inferior, el texto 'Cancelar' está a la izquierda y el botón 'Guardar' está a la derecha.

Figura A.7: Formulario para añadir un nuevo trámite al sistema.

Para registrar un trámite en el sistema únicamente hay que escribir su nombre en el formulario, Figura A.7, teniendo en cuenta que no exista otro trámite con el mismo nombre en el sistema.



El formulario 'Editar Trámites' tiene un título centralizado. Debajo de él hay un campo de texto con el placeholder 'Certificado de Empadronamiento'. Más abajo, la etiqueta 'Activo:' está seguida de un icono de casilla marcada. En la parte inferior, el texto 'Cancelar' está a la izquierda y el botón 'Guardar' está a la derecha.

Figura A.8: Formulario para editar la información de un trámite existente.

El formulario de edición, Figura A.8, permite modificar el nombre del trámite, teniendo en cuenta la unicidad; y al igual que el formulario de empleados, no se permite eliminar un trámite del sistema, solo se puede desactivar para evitar su uso.

A.2.3. Funcionalidades del secretario

ID	Teléfono	Estado	Trámite	Fecha de Creación	Asignado A
1	627899040	Pendiente	Certificado de Empadronamiento	13/02/2026 17:15:25	-
2	722643980	Pendiente	Cita AEAT	13/02/2026 17:15:25	-
3	834521234	Pendiente	Cita AEAT	13/02/2026 17:15:25	-
4	952439538	Completada	Tarjeta Sanitaria SACYL	13/02/2026 17:15:25	-
5	706365383	Pendiente	Cita AEAT	13/02/2026 17:15:25	-
6	806582096	Pendiente	Cita AEAT	13/02/2026 17:15:25	-
7	718425749	Pendiente	Cita AEAT	13/02/2026 17:15:25	-
8	900552937	Pendiente	Certificado de Empadronamiento	13/02/2026 17:15:25	-
9	895125344	Pendiente	Cita AEAT	13/02/2026 17:15:25	-
10	741435983	Pendiente	Tarjeta Sanitaria SACYL	13/02/2026 17:15:25	-

Figura A.9: Página con la tabla de peticiones para el rol de secretario.

Al acceder en el sistema con una cuenta de secretario, se muestra como pantalla de inicio la tabla de peticiones, Figura A.9. Dicha página, muestra una tabla con la información básica de todas las peticiones del sistema, tiene la misma disposición y funcionalidad explicada ya en la sección de administradores, Figura A.2. A mayores, incluye un botón en la parte superior derecha de la tabla que redirecciona a la página con el formulario para crear peticiones, Figura A.10.

Figura A.10: Formulario para la creación de peticiones manualmente.

El formulario de creación de peticiones se compone de un campo obligatorio para añadir el teléfono del llamante y otro para la selección del trámite a realizar. Dependiendo del trámite elegido se despliegan los formularios correspondientes para completarlos, Figuras A.11, A.12 y A.13.

Crear Petición

Teléfono

Certificado de Empadronamiento ▾

Introduce los Datos

DNI:

Nombre:

Apellidos:

Teléfono:

Motivo de la solicitud:

Cancelar Crear

Figura A.11: Formulario para la creación de una petición del trámite de solicitud del certificado de empadronamiento.

El formulario para solicitar el certificado de empadronamiento consta de cinco campos textuales obligatorios. Se rellenan con la información proporcionada por el usuario y se clicca en crear.

Crear Petición

Teléfono

Cita AEAT

Introduce los Datos

DNI:

Nombre:

Servicio:

Modalidad:

Oficina:

Fecha:

Cancelar

Figura A.12: Formulario para la creación de una petición del trámite de cita en la AEAT.

El formulario de solicitud de una cita en la agencia tributaria consta de múltiples campos obligatorios a rellenar, pero hay que seleccionar primero el servicio que desea el usuario. Porque dependiendo del servicio elegido, puede estar disponible la modalidad de cita telefónica o no. Una vez rellenados todos los campos se clicca en el botón de crear.

Crear Petición

Teléfono

Tarjeta Sanitaria SACYL

Introduce los Datos

Nombre:

Localidad:

Primer Apellido:

Centro de Salud:

Fecha de Nacimiento:

Calle:

Motivo de la solicitud:

Pérdida ☐ Deterioro ☐

Nº: **Piso:** **Pta.:**

Cancelar

Figura A.13: Formulario para la creación de una petición del trámite de solicitud de la tarjeta de SACYL.

El formulario para solicitar la tarjeta sanitaria de SACYL consta también de múltiples campos obligatorios a rellenar en cualquier orden y dos campos de la dirección, el piso y la puerta, que no lo son y por tanto pueden dejarse en blanco. Al rellenarlos se clicca en el botón de crear.

Petición #1, Certificado de Empadronamiento
Teléfono: 627899040

23 marzo 2026 12:28:15 Creada

23 marzo 2026 12:28:15 Pendiente

Información

DNI:
12312345A

Nombre:
Manuel

Apellidos:
Pérez

Teléfono:
627899040

Motivo de la solicitud:
Para realizar otros trámites.

Asignar

Figura A.14: Página con la información de una petición pendiente para el rol de secretario.

De la misma forma que para los administradores, al clicar en el ID de una petición en la tabla se redirige a la página con la información completa de dicha petición (Figura A.14). Para los secretarios, cuando la petición está en estado “pendiente”, se muestra un botón en la parte superior derecha que, al clicarlo, asignará automáticamente dicha petición al usuario. Esta acción activa las opciones para gestionar la petición, Figura A.15.

Petición #1, Certificado de Empadronamiento
Teléfono: 627899040

13 febrero 2026 17:15:25 Creada

13 febrero 2026 17:15:25 Pendiente

23 marzo 2026 12:19:22 Asignada Juan Sordo

Información

DNI:
12312345A

Nombre:
Manuel

Apellidos:
Pérez

Teléfono:
627899040

Motivo de la solicitud:
Para realizar otros trámites.

Desasignar

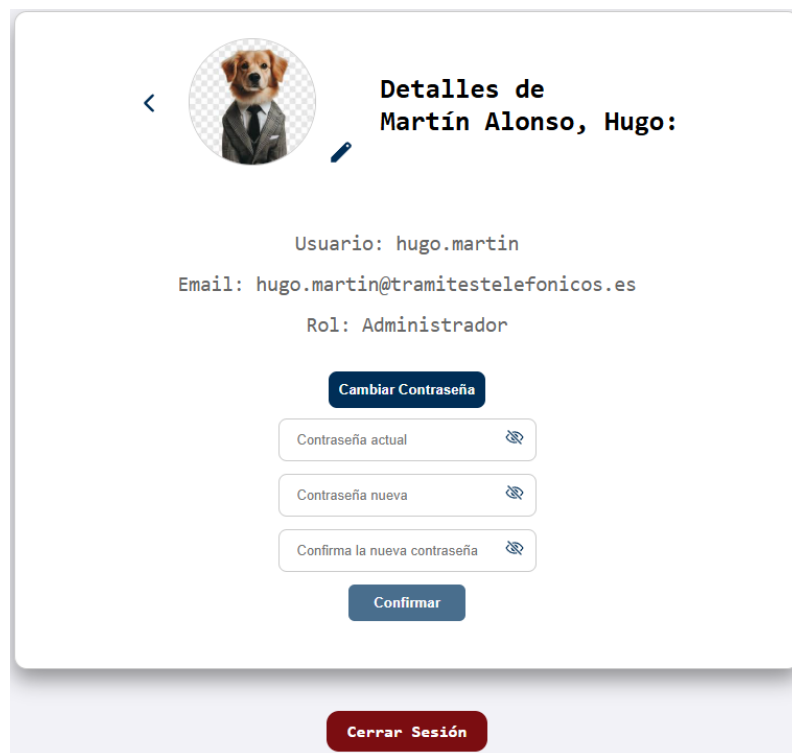
Cancelar Actualizar Completar

Figura A.15: Página con la información de una petición asignada para el rol de secretario.

Una vez asignada una petición, los campos del formulario donde se muestra la información se vuelven editables. Se puede modificar cualquier campo y se debe clicar en el botón actualizar para guardar la información modificada. Hay también otros tres botones:

- Desasignar. Se encuentra en la parte superior derecha de la página y desasigna la petición, reestableciéndola al estado “pendiente”.
- Cancelar. Finaliza la petición sin realizar el trámite.
- Completar. Inicia el script de RPA, ejecutando el trámite en la página administrativa correspondiente.

A.2.4. Funcionalidades comunes



The screenshot shows a user profile interface. At the top left is a back arrow and a circular profile picture of a dog in a suit. To the right of the picture is the title 'Detalles de Martín Alonso, Hugo:' and a small edit icon. Below this, the user's details are listed: 'Usuario: hugo.martin', 'Email: hugo.martin@tramitestelefonicos.es', and 'Rol: Administrador'. A dark blue button labeled 'Cambiar Contraseña' is positioned above three input fields: 'Contraseña actual', 'Contraseña nueva', and 'Confirma la nueva contraseña'. Each input field has an eye icon for toggling visibility. A dark blue 'Confirmar' button is below the input fields. At the bottom of the page, centered, is a red button labeled 'Cerrar Sesión'.

Figura A.16: Cuadro de detalles del perfil del empleado.

Al clicar sobre el botón del perfil de arriba a la derecha, se muestra una vista con la información detallada de la cuenta registrada, Figura A.16. Desde esta vista se puede cambiar la foto de perfil clicando en la foto actual. También se puede modificar la contraseña de la cuenta, necesario para cambiar la contraseña temporal por una propia. Por último, en la parte de abajo de esta página se encuentra el botón para cerrar la sesión del empleado.

Apéndice B

Resumen de enlaces adicionales

Los enlaces útiles de interés en este Trabajo Fin de Grado son:

- Repositorio del código: <https://github.com/humaral/tfg.git>.
- Servidor de Discord de ejemplo: <https://discord.gg/TQTEaXw>.